M.Sc. Engg. (CSE) Thesis

Agentic Graph Reasoning: Autonomous Knowledge Graph Navigation for Fact Verification and Hallucination Mitigation in Large Language Models

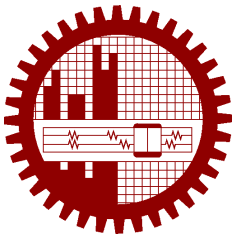
Submitted by

Md. Sakif Khan

0421052099

Supervised by

Dr. Sadia Sharmin



Submitted to

Department of Computer Science and Engineering

Bangladesh University of Engineering and Technology

Dhaka, Bangladesh

in partial fulfillment of the requirements for the degree of

Master of Science in Computer Science and Engineering

October 2026

Candidate's Declaration

I, do, hereby, certify that the work presented in this thesis, titled, "Agentic Graph Reasoning: Autonomous Knowledge Graph Navigation for Fact Verification and Hallucination Mitigation in Large Language Models", is the outcome of the investigation and research carried out by me under the supervision of Dr. Sadia Sharmin, Associate Professor, Department of CSE, BUET.

I also declare that neither this thesis nor any part thereof has been submitted anywhere else for the award of any degree, diploma or other qualifications.

Md. Sakif Khan

0421052099

The thesis titled "**Agentic Graph Reasoning: Autonomous Knowledge Graph Navigation for Fact Verification and Hallucination Mitigation in Large Language Models**", submitted by Md. Sakif Khan, Student ID 0421052099, Session April 2021, to the Department of Computer Science and Engineering, Bangladesh University of Engineering and Technology, has been accepted as satisfactory in partial fulfilment of the requirements for the degree of Master of Science in Computer Science and Engineering and approved as to its style and contents on October 7, 2026.

Board of Examiners

1. _____

   Dr. Sadia Sharmin — Chairman
   Associate Professor — (Supervisor)
   Department of CSE, BUET, Dhaka

2. _____

   Dr. Tanzima Hashem — Member
   Professor and Head — (Ex-Officio)
   Department of CSE, BUET, Dhaka

3. _____

   Dr. Muhammad Masroor Ali — Member
   Professor
   Department of CSE, BUET, Dhaka

4. _____

   Dr. Shaheena Sultana — Member
   Professor — (External)
   Department of Computer Science and Engineering
   Notre Dame University, Dhaka

Acknowledgement

I am thankful to my supervisor, Dr. Sadia Sharmin, for her invaluable guidance, continuous support, and profound expertise. Her mentorship was instrumental in shaping my research on Agentic Graph Reasoning and Large Language Models, and her precise feedback consistently pushed me to refine my methodologies.

I am thankful to the faculty and my colleagues in the Department of Computer Science and Engineering at the Bangladesh University of Engineering and Technology for creating an environment that fosters rigorous intellectual exploration.

I am thankful to my parents and family for their unwavering belief in my academic pursuits. Their encouragement provided a crucial foundation during the most demanding phases of this project.

I am especially thankful to my wife. Witnessing her dedication and resilience as she successfully pursued her own rigorous medical specialty certifications has been a constant source of inspiration to me. Her patience, understanding, and support during the long hours dedicated to this research made this achievement possible.

Finally, I owe my deepest gratitude to the Almighty for providing the strength, resources, and clarity needed to see this thesis through to completion.

Dhaka
October 7, 2026

Md. Sakif Khan
0421052099

Contents

Candidate's Declaration i

Board of Examiners ii

Acknowledgement iii

List of Figures vii

List of Tables viii

List of Algorithms ix

Abstract x

1 Introduction 1

1.1 Hallucination in Large Language Models . . . 1
1.2 From Retrieval Augmentation to Graph-Structured Grounding . . . 2
1.3 Limitations of Existing Agentic KGQA Systems . . . 3
1.4 Problem Statement . . . 4
1.5 Research Questions . . . 5
1.6 Our Contribution . . . 5
1.7 Scope and Delimitations . . . 6
1.8 Thesis Organization . . . 7

2 Background and Related Work 8

2.1 Knowledge Graphs . . . 8
2.2 Large Language Models as Reasoning Engines . . . 11
2.3 Static Graph-Augmented Generation . . . 13
2.4 Agentic Graph Exploration . . . 15
2.5 Comparative Summary of Agentic KGQA Systems . . . 18

3 The Knowledge Environment 21

3.1 Design Goals and Source Data . . . 21
3.2 Graph Construction . . . 23

3.3 Environment Validation Gate . . . 27

4 The AGR Framework: Architecture, Navigation, and Verification 30
4.1 Architectural Overview . . . 30
4.2 The Graph Tool API . . . 31
4.3 The Planner . . . 35
4.4 The Explorer . . . 36
4.5 The Evaluator and Routing Policy . . . 38
4.6 The Answerer . . . 40
4.7 Budgets and Termination Guarantees . . . 40
4.8 Instrumentation and Reproducibility . . . 42
4.9 Design Validation on the Smoke Set . . . 42
4.10 The Structural Verification Layer . . . 43
4.11 Claim Decomposition of the Draft Answer . . . 44
4.12 Two-Stage Claim Checking . . . 45
4.13 Repair Policies for Unsupported Claims . . . 49
4.14 Output Contract: Answer Paired with Supporting Triples . . . 52
4.15 What the Verification Layer Establishes, and What It Does Not . . . 53

5 Evaluation: Setup, Results, and Error Analysis 54
5.1 Experimental Setup . . . 54
5.2 Evaluation Datasets and Test Sets . . . 56
5.3 Backbone Model Selection and Qualification . . . 59
5.4 Baseline Systems . . . 60
5.5 Metrics and the Groundedness Judge . . . 64
5.6 Ablation Conditions and the Development Set . . . 67
5.7 Main Results . . . 69
5.8 Where the Margin Over the Agentic Baseline Comes From . . . 72
5.9 Positioning Against Published RoG Results . . . 74
5.10 Accuracy Broken Down by Hop Count . . . 76
5.11 Groundedness and Hallucination Rate Across Systems . . . 78
5.12 Efficiency and Cost . . . 79
5.13 Ablation Study . . . 81
5.14 Error Analysis . . . 85
5.15 Failures in Planning, Navigation, and Verification . . . 87
5.16 Graph Gaps, the Echo Attractor, and Benchmark Defects . . . 92
5.17 Discussion . . . 94
5.18 Threats to Validity . . . 97

6 Conclusion **100**
6.1 Summary of Contributions 100
6.2 Limitations 101
6.3 Future Work 102
6.4 Closing Remark 104

References **105**

Index **110**

List of Figures

4.1 The AGR state machine 32
4.2 The path of one claim through the verification layer 46

5.1 Hits@1 across hop strata 76
5.2 Accuracy against token cost 81

List of Tables

2.1 Mechanism comparison of the seven closest prior systems . . . 18
2.2 Reported evaluation settings and Hits@1 as published . . . 19

3.1 The knowledge environment as constructed and imported . . . 25
3.2 Entity resolution over all topic-entity mentions . . . 27
3.3 Environment coverage over the full test sets . . . 28

4.1 The graph tool API . . . 33
4.2 The agent state . . . 34
4.3 Budget configuration and the two tool-layer caps . . . 41

5.1 The two certified test samples . . . 57
5.2 Gold-label adjudication . . . 58
5.3 The development-set sweep . . . 69
5.4 Main results on both test sets . . . 70
5.5 Paired McNemar tests against each baseline . . . 72
5.6 The agentic-baseline comparison, split on budget clipping . . . 73
5.7 AGR against RoG's published figures . . . 74
5.8 Hits@1 and F1 per hop stratum . . . 76
5.9 Two-tier groundedness . . . 78
5.10 Cost per question . . . 80
5.11 Ablation conditions on stratified halves . . . 82
5.12 Component attribution . . . 84
5.13 The merged failure histogram . . . 86
5.14 The entity-extraction bug . . . 88

List of Algorithms

1 Structural verification of a draft 47

Abstract

Large language models answer factual questions fluently. But, they tend to sound just as fluent when they do not hold the facts. To fix this, researchers introduced Retrieval-Augmented Generation (RAG), which connects the LLM to an external data source. This grounds the model's answers in retrieved text. But, the retrieval happens once and up front, before any reasoning starts. So, a question that spans several facts has to be answered from whatever that first query brought back. Systems that navigate a knowledge graph do better by interleaving retrieval with reasoning. Even they, however, send out whatever their final generation call produces, and nothing checks the answer's claims before it reaches the user.

To close that gap we propose Agentic Graph Reasoning (AGR): a knowledge-graph question-answering framework. Here, an agent plans sub-objectives and walks the graph through a constrained, deterministic tool API. Its distinguishing component is a Structural Verification Layer. Before the answer is emitted, it splits the draft into atomic claims and checks each one against the (head, relation, tail) triples the agent actually traversed. Claims it cannot ground are re-explored or withdrawn. So, where it cannot ground a claim, AGR hedges rather than asserts. And the grounded claims come paired with the triples that support them.

We evaluate AGR on a Freebase-derived environment of 2.59 million entities and 8.31 million triples. We test it on 400 certified questions from each of WebQSP and ComplexWebQuestions. We compare it against four baselines: parametric answering, vector retrieval, GraphRAG, and an agentic navigator. All five systems share one frozen backbone and the same call budget. So, any difference between them comes from architecture, not model capacity.

AGR reaches a Hits@1 of 0.755 on WebQSP and 0.522 on ComplexWebQuestions. It beats every baseline on both datasets, at half the agentic navigator's language-model calls. The navigator does lead AGR on the questions its own budget lets it finish. But that budget runs out on 29% and 44% of questions on the two datasets. AGR's own budget never does, and AGR grounds every entity it asserts, all $1{,}709$ of them. The parametric control leaves 221 of its $1{,}001$ asserted entities ungrounded (22.1%). And on ComplexWebQuestions, AGR is the only system whose accuracy rises with hop count. Every other system ends up below where it started.

We performed an ablation study. Each condition removes exactly one component and re-runs the identical questions. The four components are the planner that decomposes a question into sub-objectives, the backtracker, the claim verification layer, and the model-based edge scorer. We tested the re-runs against the unmodified system with paired significance tests. Only the planner shows a detectable effect, and

it runs the opposite way from what we expected. Removing it *improves* WebQSP accuracy by 0.083 F1 ($p = 0.006$) and cuts token use by 31%, while it trends the other way on ComplexWebQuestions. So, decomposition should be gated on question structure, not applied unconditionally.

We read all 256 remaining failures by hand. The characteristic error is not invention but what we call the *echo attractor*. It is a real, grounded entity one hop from the answer, which any grounding check will pass because it is genuinely true, and which different systems fall into together. The same reading confirms 57 questions where the benchmark, not the system, was wrong.

Chapter 1

Introduction

Large language models answer factual questions fluently. But often enough they give wrong answers, and those errors are not random. They are systematic, and the model states them with the same confidence it gives a correct answer. That makes them hard to catch by reading the output alone. A fabricated answer looks exactly like a correct one. Things get worse when the answer requires putting several facts together rather than recalling just one. This thesis takes on that gap in two steps. First, we give the model a symbolic knowledge graph to walk through. Then, before it asserts anything, we check what it is about to say.

The rest of this chapter motivates the problem and explains why the obvious remedies still fall short. It then states the problem and the research questions, summarises our contributions, and sets out how the rest of the document is organised.

1.1 Hallucination in Large Language Models

A *hallucination* is a generated statement that reads fluently and is well formed, but backed by no source the model can point to. Very often it is simply false. Huang et al. [1] survey the phenomenon at length. They argue that hallucination comes from how these models are built, not from some incidental defect that could be patched away. That framing matters here. This is because it determines what kind of remedy is on the table at all. If hallucination were a bug, we would fix it. But it is structural, so the response has to be architectural.

1.1.1 Where Hallucination Originates: Training Data, Model, and Prompt

Hallucinations arise at three distinct points, each calling for a different kind of intervention.

One is the *training data*. A model trained on a corpus that carries errors, staleness, or thin coverage of a domain will reproduce those defects faithfully. Fixing them means curating the corpus and retraining. That is something only the model's producer can do.

A second is the *model* itself, both its parameterisation and its decoding procedure. What a model knows is stored diffusely across its weights, with no explicit index. So it holds no reliable internal signal that separates a fact it has memorised from a plausible continuation it is simply constructing. Intervening here means changing the architecture or the training objective. That, again, is largely out of reach.

The third is the *prompt*, which is the one this thesis takes on. When the context supplied alongside a question already carries the facts needed to answer it, the model does not have to fall back on parametric recall. Structured context can override defects inherited from the training data without touching the model at all. Retrieval augmentation [2] rests on that premise, and so does this work. Our contribution is in *how* that context is assembled, and in *what gets checked* before the answer goes out.

1.1.2 Why Multi-Hop Questions Are the Hard Case

Single-fact questions are easy, as "Where was Ada Lovelace born?" takes one lookup. The model either knows the answer or it does not.

Multi-hop questions, such as "Who directed the film that won Best Picture in 1998?", need a chain of lookups in a fixed order. Each step depends on the result of the one before it. Two things make these questions genuinely hard. The first is that errors compound. If the first hop lands on the wrong entity, nothing later in the chain will know this. The rest of the chain confidently carries on from a false premise. The second is that a check on the final answer alone never sees the intermediate results at all. So a system that gets the right answer by luck looks no different from one that gets it by sound reasoning.

These questions also tend to carry temporal, categorical, or ordinal *constraints* that have to be satisfied together rather than one at a time. Dropping one silently is one of the commonest ways a system gets such a question wrong instead of simply declining to answer it.

1.2 From Retrieval Augmentation to Graph-Structured Grounding

1.2.1 Vector Retrieval and Its Structural Blind Spot

Retrieval-Augmented Generation [2] attaches the model to an external corpus. The system embeds the question and retrieves passages that are semantically similar to it. It lets the model answer from those. Where the answer lives in a single retrievable passage, this works well.

Its weakness, though, is structural. Vector retrieval treats a corpus as flat chunks of text and ranks them by similarity to the question. For a multi-hop question, that signal is actively misleading.

The passage holding the *intermediate* entity may bear little lexical or semantic resemblance to the original question. At that point, the system does not yet know which entity to look for. Flat retrieval also has no way to represent the relationships between entities. They survive, at best, as adjacency inside a retrieved chunk. But relationships are exactly what a compositional question has to traverse. Surveys of the intersection between graphs and language models single this out as the central limitation of flat retrieval [3].

1.2.2 Knowledge Graphs and Static GraphRAG

A *knowledge graph* makes those relationships explicit. It stores facts as typed edges between identified entities. So composing two facts becomes a traversal rather than an inference over prose. GraphRAG [4] exploits exactly this. It builds a graph over a corpus and retrieves structured context in place of raw passages. Related systems carry the idea into long-term memory [5] and temporally-qualified facts [6].

What these systems still do is retrieve *statically*. They fetch a subgraph around the linked entities in one shot and hand it to the model. Retrieval runs first and runs only once. So it cannot know what the next reasoning step will need. Take a question whose second hop depends on the answer to its first. A fixed-radius neighbourhood is then either too small to contain that answer, or so large that the relevant edges are buried among thousands of irrelevant ones.

1.2.3 Agentic Retrieval: Reasoning as Navigation

The natural response is to interleave retrieval with reasoning. The model takes one step, looks at what it found, and decides where to look next. Retrieval then turns into *navigation*. The system becomes an agent operating on the graph rather than a pipeline reading from it.

Think-on-Graph [7] set the pattern with LLM-guided beam search over the graph. Plan-on-Graph [8] added sub-objective decomposition and reflection-driven backtracking. Reasoning-on-Graphs [9] generates grounded relation paths as explicit plans. Other systems add tool abstractions [10], cope with incomplete graphs [11], or generalise across structured sources [12]. Recent surveys treat this meeting of language models and knowledge graphs as a research programme in its own right [13, 14].

Agentic navigation is the right shape for the problem, and this thesis adopts it. The question is what it still leaves unsolved.

1.3 Limitations of Existing Agentic KGQA Systems

Nothing checks what the final answer asserts. Agentic systems ground their answers in the subgraph they traverse. Our own measurements confirm that this works. The entities they

assert do exist in the graph, and they are reachable along the paths the systems actually walked (Section 5.11). The problem is not fabrication but *precision*. Every system surveyed in Chapter 2 simply emits whatever its final generation call produces from the traversed context. Nothing checks that each asserted entity actually answers the question asked. An entity can sit in the retrieved subgraph, have been retrieved correctly, and still be the wrong answer. It may be an intermediate node from the reasoning chain, a container where the question asked for a member, or one of several plausible neighbours picked more or less at random. A system can equally assert a crowd of entities where the question admits only a few, inflating recall at the cost of precision. Both show up as a measurable precision gap between systems (Section 5.7.2). That gap motivates a check without being evidence that the check works. Section 5.13 is where this thesis asks whether the check proposed here is what produces it. Chain-of-Verification [15] and multi-agent debate [16] show that post-hoc self-checking reduces factual error in free-form generation. Hu et al. [17] demonstrate a self-correcting agentic graph pipeline in a clinical setting. What none provides is a claim-level check against the retrieved triples, carried out *before* the answer is emitted, inside a graph-navigating agent.

The contribution of individual agentic mechanisms is unquantified. Agentic systems are assemblies of mechanisms such as decomposition, scoring, backtracking, and self-correction. The literature tends to report them as wholes. When one beats a static baseline, the published work rarely establishes which mechanism produced the gain, or whether some contribute nothing at all. So later work inherits the entire assembly on faith along with its cost. Component-level ablation is the standard remedy, but this literature underuses it.

Accuracy is reported without cost. Navigation is expensive, because every exploration step costs at least one model call. Agentic systems routinely burn an order of magnitude more tokens per question than a single-shot baseline. An accuracy table that leaves this out makes agentic methods look better across the board. The honest picture is a trade-off. And that trade-off is exactly what a practitioner needs to see before deciding whether to deploy such a system.

1.4 Problem Statement

We address the following problem. Given a natural-language question q and a knowledge graph $G = (E, R, T)$ with entities E, relation types R, and triples $T \subseteq E \times R \times E$, produce an answer set $A \subseteq E$ together with a supporting triple set $S \subseteq T$, such that S justifies every entity in A and contains only triples the system actually traversed in reaching A.

Three properties set this apart from standard knowledge graph question answering. The output is a *pair*, not just an answer, and its supporting triples are part of the contract. That is what makes the reasoning auditable rather than merely asserted. The support also has to be *traversed*. It is not enough that the triples could be retrieved after the fact. That requirement closes off the possibility of justifying, post hoc, an answer the system reached some other way. And the system

checks the justification *before* it emits the answer. So it can drop or hedge unsupported content instead of merely flagging it.

1.5 Research Questions

RQ1: Does agentic navigation improve multi-hop factual accuracy?

Does interleaving retrieval with reasoning over a knowledge graph yield more accurate answers than static retrieval, and does the advantage grow with the number of hops a question requires? Answering it calls for a no-retrieval control. The standard benchmarks predate current models and may be partly memorised.

RQ2: What does pre-generation verification contribute beyond graph navigation?

Given a system that already navigates the graph, what does a claim-level check against the traversed triples add? It is posed as *what does it contribute* rather than *does it reduce hallucination*. Its answer has several parts that a yes-or-no framing would obscure (Section 5.17.1).

RQ3: Which components contribute what, at what token cost?

Decomposition, backtracking, verification, and learned candidate scoring each cost model calls. Which of them measurably improve accuracy or groundedness, by how much, and at what price in tokens and latency?

1.6 Our Contribution

We present Agentic Graph Reasoning (AGR), a knowledge-graph question-answering framework built as an explicit state machine over a constrained, deterministic graph tool API. It carries a *Structural Verification Layer* that decomposes the draft answer into atomic claims and checks each one against the traversed subgraph before the system emits anything. *Structural* names what that check establishes and, in the same word, bounds it. The two graph routes ask whether an edge joins a claim's endpoints, not whether it is the edge the claim names, and not in which direction. So a claim that X is Y's mother is certified by an edge recording that X is Y's child. That is the price of allowing free-text relations. Section 4.15 explains why, and reports how large the exposure is. When a claim turns out to be unsupported, the design either sends the agent back for a targeted round of re-exploration or drops the claim from the answer. AGR returns what survives paired with the triples that support it.

Both repair routes are reported at the frequency we actually measured rather than the one we intended. That frequency is low, and the re-exploration route never fired on the development set. The depth budget is already exhausted by the time the verifier is reached. And the entity filter ran on three of eighty questions (Section 4.13.4). On the vast majority of questions the layer passes

through and certifies a draft at no extra model call. That is a finding about when verification can act rather than a defect in it. Section 5.17.1 treats it as one.

Four further contributions follow from evaluating that framework. **A component-level ablation** of the planner, the backtracking mechanism, the verification layer, and the learned component of candidate scoring reports accuracy, groundedness, and token cost for each with paired significance testing, closing for one architecture the attribution gap of Section 1.3. **Stratum-dependent decomposition** is what that ablation finds. The literature treats question decomposition as straightforwardly beneficial. But removing the planner *improves* accuracy on the shallower benchmark while trending toward harming it on the deeper one. The asymmetry, not the pooled average, is the result. Only the improving arm reaches significance ($p = 0.006$). The harming arm does not ($p = 0.088$). So it is reported as a direction rather than an effect. **The echo attractor** names a recurring failure in which several systems sharing a graph and a backbone converge on the same wrong answer. That answer is typically the question's own topic entity, an intermediate node from the reasoning chain, or a coarser-granularity container. No evaluation treating those systems as independent can see that convergence. It accounts for 13 of the 256 failures read in Section 5.16.2, though the contribution is the named mechanism and what it means for consensus-based evaluation rather than the frequency. And **quantified benchmark defect rates** for WebQSP and CWQ come from a consensus pre-pass followed by per-item adjudication, separating the two kinds of label error by their opposite evidence signatures and reporting the accuracy tables as measured, with the direction in which the defects cut stated beside them rather than rescored away.

The evaluation protocol itself fixed four decisions before any test data was touched. They were the scoring hyperparameters, the sample size, the baseline-certification diagnostic, and the judge-agreement bar. How each turned out is reported, including the one narrowly missed.

1.7 Scope and Delimitations

Our knowledge environment is a static, curated subset of Freebase [18], specifically the union of per-question subgraphs, each pre-extracted so as to contain its own question's answer. That makes it a friendlier place to search than the full knowledge base. It also puts answer reachability above 97% by construction (Section 3.3). The absolute accuracies we report are therefore not comparable to evaluations over full Freebase. The comparisons between systems are comparable, though, because every system navigates the very same environment. Section 5.18 states the limit in full. We do not take on graph construction from unstructured text, knowledge-base completion, or the temporal evolution of facts. And we draw only English factoid questions, from WebQSP [19] and ComplexWebQuestions [20]. Long-form generation and conversational settings are outside our scope.

All systems share one backbone model. We hold it constant so that none of the differences we

report can be credited to model capability rather than to architecture. What we establish, then, is how these architectures compare on a single backbone, not how they scale across backbones. Finally, AGR is a research prototype that we evaluate offline. We treat latency as a cost measure rather than as evidence of production readiness.

How to read the title. This thesis is submitted under the title registered with CASR at the proposal stage. That title is fixed at registration and is not revised here. Three of its terms are broader than what the chapters below actually establish. They are better read against the findings than taken as claims in their own right. *Hallucination mitigation* does occur, but it is not this work's contribution. Section 5.11 attributes the elimination of structural hallucination to graph navigation, with the agentic baseline reaching the same zero without any verification layer at all. *Autonomous* names a design this thesis argues against. Under the control pattern of Section 4.1, the program orchestrates the loop. The model is consulted only at constrained decision points. And *fact verification* is narrower here than the phrase suggests. The check establishes structural adjacency rather than the relation a claim asserts. It never reaches answer relevance (Section 4.15). Put in the terms the measurements actually support, what follows is a claim-level structural check placed before emission, together with a component-level attribution of the mechanisms around it.

1.8 Thesis Organization

Chapter 2 lays the groundwork that the technical chapters build on. It then surveys the relevant prior work. The survey covers knowledge graphs and their Freebase-specific idiosyncrasies, knowledge graph question answering and its evaluation conventions, language models as reasoning engines, retrieval augmentation, graph search, and the statistical tools used throughout. It ends with a comparison of agentic knowledge graph question answering systems that pinpoints the gap this thesis addresses and justifies the choice of baselines.

Chapter 3 describes how the knowledge environment is built and validated, including the answer-reachability gate that bounds the accuracy attainable inside it. Chapter 4 specifies the AGR framework. It covers the tool API, the state machine, the planner, the scoring function, the backtracking policy, and the budgets that guarantee termination. Section 4.10 treats the Structural Verification Layer in detail. Chapter 5 reports the evaluation in three parts. They are the experimental design fixed in advance (Section 5.1), the measurements and the answers to the three research questions (Section 5.7), and a hand reading of every failure that remained (Section 5.14). Chapter 6 sums up the contributions, states the limitations, and points to future work.

Chapter 2

Background and Related Work

This chapter lays out the concepts and notation that the technical chapters rely on. It also surveys the work that AGR builds on and competes with. The chapter is kept short on purpose and covers only material that is used later. Standard results are stated, not derived. The background comes first. It covers knowledge graphs, question answering over them, language models as reasoning engines, retrieval augmentation, and search. The survey follows. It moves from static graph-augmented generation, through path retrieval and agentic exploration, to verification and factuality measurement. The chapter closes with a structured comparison of the seven closest systems. That comparison does two things. It points to the gap this thesis addresses. And it justifies the choice of baselines in Section 5.4.

2.1 Knowledge Graphs

2.1.1 Triples, Entities, and Relations

A *knowledge graph* $G = (E, R, T)$ consists of a set of entities E, a set of relation types R, and a set of triples $T \subseteq E \times R \times E$. A triple (h, r, t) asserts that relation r holds from head entity h to tail entity t. One example is (Ada Lovelace, place_of_birth, London). Relations are directed and typed, and both directions carry meaning. The inverse of place_of_birth is a legitimate but distinct relation.

Two properties matter for what follows. Triples are *atomic*, each asserting exactly one fact. So a claim can be checked against the graph by testing for the existence of specific edges. They are also *composable*. A chain $(e_0, r_1, e_1), (e_1, r_2, e_2), \ldots$ expresses a multi-step relationship that no single triple states. That composability is what makes a knowledge graph the natural substrate for multi-hop questions.

2.1.2 Freebase: MIDs, Schema, and Mediator Nodes

Freebase [18] is a large, collaboratively-built knowledge base, frozen since 2016. It is also the substrate for the standard knowledge graph question answering benchmarks. Three of its characteristics affect any system built on it. They concern how it identifies entities, how it names relations, and how it stores a fact with more than two participants.

A *machine identifier* (MID) is the permanent key Freebase assigns to a node. It is a short, opaque code prefixed m. or g., such as m.02mjmr. Nothing in the code says what the node is. A node's human-readable name is stored apart from its MID, as the value of an ordinary property. Names are neither unique nor stable. Two entities can share one, and editors could change one. So entity identity and entity surface form must be handled separately.

The *schema* is the vocabulary Freebase states facts in, and it has three levels. A *domain* is a subject area, such as people. A *type* is a class of entity within a domain, such as people.person, and one entity may carry several types. A *property* is an attribute that a type defines. A relation is named by the full path to its property. For example, people.person.place_of_birth is the property place_of_birth of the type person in the domain people. Many properties come with a *reverse* that states the same fact from the other end, here location.location.people_born_here. The vocabulary runs to tens of thousands of relations, and many of them carry no factual content. Among these are the *type* predicates, which record an entity's types, and the *key* predicates, which list its identifiers in other namespaces, such as Wikipedia's.

Most consequentially, a *mediator node*, which Freebase calls a *compound value type* (CVT), is an unnamed node standing for one n-ary fact. An n-ary fact has more than two participants, but a triple joins only two entities. So the fact becomes a node of its own, with one edge to each participant, and each edge's relation names that participant's role. That Rainn Wilson played Dwight Schrute in *The Office* is stored as the mediator m.02l00g_. Its actor, character, and series edges, all properties of the type tv.regular_tv_appearance, lead to the actor, the character, and the show. So a relationship a user would describe as one hop is two edges in the graph. Any system reporting hop counts over Freebase must state how it treats these nodes. The choice changes every depth measurement. Systems that ignore them lose the facts they encode [11].

2.1.3 Property Graphs, Neo4j, and Cypher

The triple model above is that of the *Resource Description Framework* (RDF), the W3C standard for data stated as triples. The alternative is the *property graph* model, in which nodes and edges are both typed and may carry arbitrary key–value properties. The graph database Neo4j [21] implements it. Cypher [22] queries it, with a MATCH clause that expresses graph patterns directly. A triple store maps onto it naturally. Entities become nodes carrying their MID and name, and relations become typed edges. Traversal patterns that would require recursive joins in a relational

schema are expressed as literal path patterns instead. That is the practical reason for choosing a graph store here.

2.1.4 Knowledge Graph Question Answering

Multi-Hop Questions and Constraint Satisfaction Knowledge graph question answering (KGQA) takes a natural-language question and a graph and returns the entity or entities that answer it. A question is k-*hop* if answering it requires traversing k edges from the entities named in the question to the answer. Those entities are its *topic entities*. Beyond hop count, questions vary in their *constraints*. A constraint is a condition that filters candidates rather than extending the chain. "Which of Tolstoy's novels was published after 1875?" is one hop plus a temporal constraint. A correct answer has to satisfy both at once. A system that walks the hop correctly but quietly drops the constraint returns every novel, not just the ones published after 1875. The result is a superset of the answer, a failure that hop-count analysis alone never exposes.

Two families of approach have historically divided this field. *Semantic parsing* methods translate the question into a formal query and execute it. That approach is exact and interpretable, but it requires the parse to be exactly right [19, 23]. *Information retrieval* methods instead pull out a relevant subgraph and rank candidate answers inside it. They fail more gently, but they are harder to see into. The agentic systems considered here sit closer to the second family. But they get much of the first family's transparency back by logging the traversal itself.

Evaluation Conventions: Hits@1 and F1 Two metrics are standard. *Hits@1* scores a question 1 if any entity the system asserts is in the gold answer set, and 0 otherwise. It is a simple right-or-wrong score per question. It is also the headline number in most of this literature. The name is historical and slightly misleading. It was coined for systems that returned a *ranked* list, where only the top entry counted. It has since been carried over to systems that return an unordered set. There it becomes the set-intersection test just stated. This thesis uses the set-intersection form throughout, because that is what the systems it compares against report. Section 5.5 pins down the string-matching rule exactly.

Many questions have more than one correct answer. Hits@1 alone rewards a system that names one correct entity and ignores the rest. *F1* fills that gap. It is computed per question between the predicted answer set A and the gold set A^*:

$$P = \frac{|A \cap A^*|}{|A|}, \qquad R = \frac{|A \cap A^*|}{|A^*|}, \qquad F_1 = \frac{2PR}{P+R},$$

and then averaged over questions. The difference between P and R matters a great deal in this thesis. A system that asserts many entities can score high recall while its precision falls. Section 1.3 argues that exactly this failure goes unmeasured when only Hits@1 is reported.

2.2 Large Language Models as Reasoning Engines

2.2.1 In-Context Learning, Chain-of-Thought, and the ReAct Loop

Sufficiently large language models perform tasks specified entirely in their prompt, without gradient updates. This is *in-context learning* [24]. Prompting the model to produce intermediate reasoning steps before its final answer, *chain-of-thought* prompting [25], substantially improves performance on multi-step problems. Both are used here. The agent's planner, scorer, evaluator, and verifier are prompted components of a single frozen model, which this thesis calls the *backbone*. No component is *fine-tuned*, that is, trained further on examples of the task. That matters for comparability, since several systems surveyed in Section 2.5 are fine-tuned and therefore not directly comparable to prompting methods.

A language model can also be given *tools*. A tool is a named operation the model may invoke, whose result re-enters its context [26]. The *ReAct* pattern [27] interleaves reasoning and action in a loop. The model reasons about what to do, acts, observes the result, and reasons again. That loop is the control structure underlying agentic graph navigation. Where the tools are graph operations, each iteration extends a traversal. The sequence of tool calls is a complete record of how the answer was reached.

2.2.2 A Working Definition of Hallucination for This Thesis

"Hallucination" means different things in different papers. This thesis adopts a narrow, operational definition tied to the knowledge environment. An *asserted* entity is one the system names in its answer. It is *ungrounded* if it does not exist in the graph, or exists but lies more than four hops from every one of the question's topic entities. A *claim* is a single statement drawn from the answer, relating two entities. It is *unsupported* if the traversed triples do not entail it.

Two consequences of this definition should be stated plainly. The definition is relative to the environment. An entity absent from the graph is ungrounded even if true in the world. And groundedness is not correctness. An answer can be perfectly grounded, every entity real and reachable, and still be the wrong answer to the question. Section 5.7 shows that this gap is not hypothetical, and Section 1.3 is built on it.

2.2.3 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) [2] gives a language model material retrieved from an external collection at the moment it answers. The answer can then rest on that evidence rather than on *parametric* knowledge, the facts absorbed into the model's weights during training. Dense retrieval encodes queries and documents into a shared vector space and retrieves by

nearest neighbour, capturing semantic similarity where lexical matching fails [28]. Sentence-level encoders such as Sentence-BERT [29] produce embeddings suitable for direct cosine comparison. In this thesis embeddings serve two bounded purposes. They link question phrases to graph entities, and they pre-rank relation candidates during exploration. They never establish that a fact is true, a job that belongs to the symbolic triples alone. Graph-based retrieval swaps flat passages for structured context [4, 5]. What gets retrieved is a subgraph rather than a document. So the relationships between facts come along explicitly. The distinction that matters here is not graph versus vector but *static versus interleaved*. Does retrieval happen once, before reasoning begins, or again and again as reasoning proceeds?

2.2.4 Search over Graphs

Best-first search keeps a *frontier* of nodes it has reached but not yet expanded. To *expand* a node is to generate its neighbours. A heuristic scores each frontier node by how promising it looks, and the search expands the most promising one first [30, 31]. *Beam search* trims the frontier to the b highest-scoring candidates at each depth. That gives up *completeness*, the guarantee of finding a goal whenever one is reachable, in exchange for a fixed memory and computation budget. Here the scoring function is a language model call, which is expensive. So the beam width is also the principal cost control. Bounded search commits to choices that may prove wrong. *Backtracking* recovers by abandoning the current branch and resuming from a previously unexplored alternative. That recovery requires maintaining a stack of such alternatives and a trigger condition for when to pop it. Both are specified in Section 4.5.

Relation to Monte Carlo Tree Search: A Terminological Clarification The original proposal for this work described its navigation strategy as "inspired by Monte Carlo Tree Search". That description is withdrawn here, for a reason worth stating. Monte Carlo Tree Search [32, 33] is defined by four phases. They are selection, expansion, *simulation* (a rollout to a terminal state), and *backpropagation* of the rollout's value to the ancestors of the expanded node. The two phases that give MCTS its statistical character are simulation and backpropagation. Together, they let node values be estimated from sampled outcomes rather than from a fixed heuristic.

The method used in this thesis has neither phase, so it performs no rollouts. No value is propagated back up the search tree. Each candidate is scored once, by a fixed function combining embedding similarity with a language-model judgement. That score is never revised in light of what is found later. What the method is, is *guided best-first search with a bounded beam and backtracking*. It is described that way throughout. The contrast is not hypothetical, since genuine tree search has been applied to this exact task. KBQA-o1 [34] drives an agentic knowledge base question answering process over Freebase with full MCTS, including policy and reward models, rollouts, and value backpropagation. Reasoning with Trees [35] likewise builds an explicit search

tree. That those systems exist is precisely why the terminology matters here. Describing a single-pass scoring heuristic as MCTS-inspired would invite comparison with machinery this framework does not implement.

2.2.5 Statistical Tools Used in This Thesis

Three tools recur, each chosen for a specific property of the data.

Bootstrap confidence intervals [36] put a number on sampling uncertainty. The evaluated questions are resampled with replacement, and the metric is recomputed on each resample. The 2.5th and 97.5th percentiles of the resulting distribution give a 95% interval. That is the *percentile* method, used here rather than the bias-corrected variants. The intervals are read as a sense of scale for a reported difference, not as a hypothesis test. They also make no distributional assumption about metrics such as per-question F1.

McNemar's test [37] asks whether two systems differ in accuracy when both are run on the *same* questions. It looks only at the discordant pairs. These are the questions that one system gets right and the other gets wrong. The test asks whether the split between the two discordant counts is more lopsided than chance would produce. Paired testing is essential here, because every system is run on the same question samples. An unpaired test would throw the pairing away and lose power.

Cohen's κ [38] measures agreement between two annotators on a categorical judgement, correcting for agreement expected by chance:

$$\kappa = \frac{p_o - p_e}{1 - p_e},$$

where p_o is observed agreement and p_e is the agreement expected if both annotators labelled independently at their observed marginal rates. It is used to validate the automatic groundedness judge against human annotation. Conventional interpretation follows Landis and Koch [39], on whose scale values between 0.61 and 0.80 denote substantial agreement.

2.3 Static Graph-Augmented Generation

2.3.1 GraphRAG and Query-Focused Summarization

GraphRAG [4] constructs a knowledge graph from a text corpus using a language model. It partitions the graph into *communities*, which are clusters of densely interlinked entities nested in a hierarchy. It pre-computes a summary of each community. A query is then answered from those summaries rather than from retrieved passages. GraphRAG was designed for *query-focused summarization*, in which a corpus is summarised with respect to one question. Its strength is

global questions. The answer to such a question depends on the corpus as a whole rather than on any single passage. Flat retrieval handles these poorly.

The relevant limitation is that retrieval remains a single step preceding generation. The context is assembled before the model has begun reasoning. So it cannot be conditioned on intermediate findings. GraphRAG also builds its graph by language-model extraction. This is appropriate for unstructured corpora, but it introduces extraction error into the reference itself. That consideration shapes the environment design in Section 3.1.

2.3.2 HippoRAG and Memory-Structured Retrieval

HippoRAG [5] draws on the hippocampal indexing theory of human memory. It combines a knowledge graph with *personalised PageRank*. This is a random walk over the graph that keeps restarting from the entities named in the query, so that nodes are ranked by how readily the walk reaches them. A single query can then integrate evidence distributed across many passages. It substantially improves multi-hop retrieval over flat baselines at low cost.

Its retrieval is nonetheless a single graph-propagation step, not an agent-directed traversal. The propagation is fixed, and the model does not choose where to look next. Temporal extensions such as T-GRAG [6] address a different axis. They handle conflicting and redundant facts across time. This is orthogonal to the navigation question. So it is noted here for completeness rather than compared against.

2.3.3 Path-Retrieval and Semantic-Parsing KGQA

Reasoning-on-Graphs Reasoning-on-Graphs (RoG) [9] adopts a planning–retrieval–reasoning pipeline. A fine-tuned model generates *relation paths*. A relation path is a sequence of schema relations that names no entity, such as a place of birth followed by the country containing it. Each path is then *instantiated*. It is matched against the graph as a chain of actual triples, starting from the question's topic entities. The model reasons over the chains it retrieves. Because every answer comes from a retrieved chain, RoG is *faithful* by construction. The path it presents as an explanation is the path that produced the answer.

Two properties distinguish it from the present work. It is *fine-tuned*. This places its reported numbers outside direct comparison with prompting methods. And its plan is generated in one shot before retrieval. If the generated relation path does not instantiate in the graph, there is no mechanism to revise it mid-course. The pre-generation verification question does not arise here, because grounding is guaranteed structurally rather than checked.

RoG is nonetheless the one surveyed system this thesis shares a data distribution with. The knowledge environment of Section 3.1.3 is built from its published subgraphs. So the comparison is taken up directly, figures and all, in Section 5.9.

StructGPT and KG-Agent StructGPT [12] provides a general interface for reasoning over structured data such as knowledge graphs, tables, and databases. It works through an invoke–linearise–generate loop. Specialised interfaces extract evidence that is linearised into text for the model. StructGPT is deliberately general rather than graph-specific. It performs no open-ended graph search.

KG-Agent [10] is closer to the agentic setting. It combines a multifunctional toolbox, a knowledge-graph executor, and a dynamic memory, all driven by an *instruction-tuned* model that autonomously selects tools across iterations. An instruction-tuned model is one fine-tuned on examples that pair task instructions with worked solutions. KG-Agent demonstrates that a small fine-tuned model with good tool abstractions is competitive with much larger prompted ones. It plans implicitly through tool selection rather than committing to explicit sub-objectives. It provides no backtracking mechanism and performs no verification of the final answer.

2.4 Agentic Graph Exploration

2.4.1 Think-on-Graph

Think-on-Graph (ToG) [7] established the template for *training-free* agentic KGQA, in which the model is prompted and never fine-tuned. The model performs iterative beam search over the graph. At each step it is shown candidate relations and entities and prunes them itself. Beam width and maximum depth are both set to three. Exploration terminates when the model judges the retrieved paths sufficient. At that point it answers directly.

ToG is the most directly comparable prior system to AGR. It is training-free and prompting-based, it operates on Freebase, and it is evaluated on the same benchmarks. It is therefore the agentic baseline in Section 5.4. Its limitations are instructive. There is no explicit decomposition, so multi-constraint questions are handled implicitly. Low-scoring branches are pruned within the beam. But there is no mechanism to return to an abandoned node once the beam has moved on. And the transition from exploration to answering is a single model judgement. Nothing checks the answer that follows.

2.4.2 Plan-on-Graph

Plan-on-Graph (PoG) [8] is the closest system to AGR in ambition. It decomposes the question into *sub-objectives*. These are smaller questions, each resolvable by a short search, with the question's constraints attached. It explores adaptively, with a breadth that varies by sub-objective, under three mechanisms. *Guidance* uses the sub-objectives to steer exploration. *Memory* records the subgraph and reasoning paths found so far. *Reflection* decides both whether to self-correct

and which entity to backtrack to. That dual role makes PoG the one surveyed system with genuine backtracking.

PoG's self-correction operates on the *search process*, reconsidering where to explore. It does not decompose the draft answer into claims and check each against the retrieved triples before responding. That difference is the gap this thesis addresses, and Section 2.5.1 states it precisely.

One naming hazard needs a warning here. A second, unrelated system, Paths-over-Graph [40], also goes by PoG. It reports considerably higher numbers. Survey tables that cite "PoG" without saying which one are a known source of muddled comparisons. This thesis therefore always writes Plan-on-Graph or Paths-over-Graph in full.

2.4.3 Generate-on-Graph and Reasoning with Trees

Generate-on-Graph (GoG) [11] targets the incomplete-graph setting through a thinking–searching–generating loop. When the graph lacks a required fact, the model generates candidate triples from its parametric knowledge and continues. It also expands subgraphs dynamically to handle the mediator nodes described in Section 2.1. ToG tends to skip these. The trade-off is directly opposed to this thesis's. GoG deliberately admits model-generated facts into the evidence, whereas AGR admits only graph triples.

Reasoning with Trees [35] applies explicit tree search to knowledge graph question answering. It is cited in Section 2.2.4 as an example of genuine tree-search machinery. That machinery contrasts with the guided best-first search implemented here.

2.4.4 KBQA-o1 and Genuine Tree Search

KBQA-o1 [34] is the closest system to the search strategy the present work's proposal originally described, and therefore the most important one to distinguish from it. It performs agentic knowledge base question answering with full Monte Carlo Tree Search. A policy model proposes actions, a reward model scores them, and the values of rollouts propagate back through the tree. The search is wrapped around a ReAct-style agent that generates *logical forms* step by step against a Freebase environment. A logical form is a query in a formal language that the knowledge base can execute. The exploration additionally generates annotations used for incremental fine-tuning. So the search improves the model that drives it.

Two aspects bear on this thesis. First, KBQA-o1 is what the withdrawn "MCTS-inspired" description of Section 2.2.4 would have claimed. It has the rollouts and value backpropagation that AGR does not. Saying so plainly is what keeps the two from being confused.

Second, its ablation speaks directly to the methodological argument of Section 2.5.1. Removing MCTS from KBQA-o1, with every other component left in place, drops overall GrailQA F1 from 78.5 to 48.5. That is thirty points attributed to a single mechanism. The authors could make

that attribution only because they ablated components rather than reporting the whole assembly as one number. This thesis applies the same instrument in Section 5.13. That the two studies agree here is evidence that the instrument tells us something real.

KBQA-o1 is not a baseline here, for three reasons. It is fine-tuned rather than prompted. It is evaluated under a low-resource few-shot protocol. And it does not report results on ComplexWebQuestions. So there is no way to run a controlled comparison against it in this environment.

2.4.5 Verification and Self-Correction in Language Models

Chain-of-Verification Chain-of-Verification (CoVe) [15] has the model draft a response, plan verification questions about it, answer those on their own, and then regenerate a corrected response. It establishes the core premise behind this thesis's verification layer. If a draft is broken into parts and each part is checked, factual errors go down. But CoVe runs its checks against the model's own knowledge, not against an external symbolic source. Multi-agent debate [16] gets a related effect by having several model instances critique one another.

That distinction matters. There is evidence that language models cannot reliably correct their own reasoning without outside feedback [41]. A verification step whose ground truth is the very model that made the error is limited by construction. AGR's verification is grounded in traversed triples, putting the check outside the generator.

Self-Correcting Graph RAG A self-correcting agentic graph RAG system for clinical decision support in hepatology [17] shows agentic retrieval and a correction loop working together in a high-stakes domain. It backs up the practical case for verification, though in one specialised domain rather than on the open-domain benchmarks used here. Reflexion [42], in which an agent writes a critique of its own failed attempt and conditions the next attempt on it, is the general agentic pattern. The hepatology system is a graph-specific instance of it.

2.4.6 Measuring Factuality

Claim-Decomposition Metrics FActScore [43] evaluates long-form generation by breaking the text into *atomic facts*. An atomic fact is a short statement that conveys a single piece of information. FActScore checks each one against a reference source and reports the proportion that are supported. This decompose-then-verify protocol is the basis for the groundedness metric in Section 5.5.3, with the knowledge graph as the reference. The one methodological requirement that carries over is that the judge doing the checking must be independent of the system that wrote the text.

Table 2.1: Mechanism comparison of the seven closest prior systems. No system performs claim-level verification of the final answer against retrieved triples before responding.

System	Exploration strategy	Decomposes query?	Backtracks?	Verifies before answering?
ToG [7]	Training-free LLM-guided iterative beam search; beam width and depth both 3	No	No (prunes within beam, cannot return)	No
PoG [8]	Self-correcting adaptive planning with Guidance, Memory, Reflection	Yes, into sub-objectives with constraints	Yes (Reflection selects the entity to return to)	Partial (corrects the search, not the answer)
RoG [9]	Planning–retrieval–reasoning over generated relation paths	Partial (relation-path plans)	No	Faithful by construction; no explicit check
KG-Agent [10]	Toolbox with KG executor and dynamic memory; autonomous tool selection	Implicit, via tool calls	No	No
GoG [11]	Thinking–searching–generating; generates triples when the KG is incomplete	Yes	No (expands subgraph instead)	No
StructGPT [12]	Iterative reading-then-reasoning via invoke–linearise–generate interfaces	No	No	No
KBQA-o1 [34]	Monte Carlo Tree Search with policy and reward models over a ReAct agent; fine-tuned	Implicit, stepwise logical form	Via MCTS tree revisiting	No
AGR (this work)	Guided best-first beam search with explicit backtracking	Yes, into ordered sub-objectives	Yes (threshold, dead end, or evaluator veto)	**Yes (claim-level, pre-generation)**

Factuality Benchmarks and Their Limits FactBench [44] is a dynamic benchmark for factuality evaluation in the wild. The original proposal for this thesis meant to use it twice, as a source for the knowledge environment and as the hallucination evaluator. That plan was dropped for a methodological, not practical, reason. If the evaluator's facts sit inside the graph the system retrieves from, a low hallucination rate follows automatically. It says nothing about merit. The evaluation in Section 5.1 therefore measures claim grounding against the graph with a human-validated judge. That keeps the evaluator's ground truth outside the system's evidence.

2.5 Comparative Summary of Agentic KGQA Systems

Table 2.1 compares the seven closest systems on the mechanisms relevant to this thesis. Table 2.2 records their evaluation settings and published accuracy.

Three caveats attach to Table 2.2 and are carried into every comparison in this thesis.

Table 2.2: Reported evaluation settings and Hits@1 as published. These figures are *not* directly comparable to one another. Backbones differ, some papers evaluate on sampled subsets, and the backbone accounts for much of the variation.

System	Backbone	Datasets	WebQSP	CWQ
ToG	GPT-3.5	CWQ, WebQSP, GrailQA [45], QALD10-en, SimpleQuestions, WebQuestions, T-REx, Zero-Shot RE, Creak	76.2	57.1
	GPT-4		82.6	67.6
PoG	GPT-3.5	CWQ, WebQSP, GrailQA	82.0	63.2
	GPT-4		87.3	75.0
RoG	Fine-tuned LLaMA2-7B	WebQSP, CWQ	85.7	62.6
KG-Agent	Fine-tuned 7B	WebQSP, CWQ, plus other KBs	83.3	72.2
GoG	GPT-3.5	WebQSP, CWQ (complete and incomplete KG)	78.7	55.7
	GPT-4		84.4	75.2
StructGPT	GPT-3.5	WebQSP, MetaQA, TableQA, Text-to-SQL	72.6	—
KBQA-o1	Llama-3.1-8B	GrailQA, WebQSP, GraphQ; 100-shot low-resource protocol	59.8†	—
	Llama-3.3-70B		67.0†	—

† KBQA-o1 reports **F1**, not Hits@1, and evaluates under a 100-shot low-resource protocol. These cells are therefore *not* comparable with the rest of the column and are shown only to situate the system. It does not evaluate CWQ.

Every figure is the one its own authors published. That this is not the same as a comparison is visible inside the table. Think-on-Graph's own paper reports a second variant, ToG-R, at 58.9 and 69.5 on CWQ against the 57.1 and 67.6 shown, on the same two backbones. One paper's own variants differ by 1.8 and 1.9 points on the same benchmark. That gap is the order of many claimed improvements in this literature. It is also the reason Section 5.1 reimplements its baseline rather than citing a number.

First, the figures are drawn from the original publications and reflect different backbones. RoG and KG-Agent are fine-tuned. Their numbers are not comparable with prompting methods at all. Among the prompting methods, the backbone accounts for much of the spread. The same methods drop sharply when run on smaller *open-weight* models, whose trained weights are publicly released. That sensitivity is the direct motivation for holding the backbone constant across every system in Section 5.1. The literature does not hold it constant. So a cross-paper table cannot settle which architecture is better.

Second, evaluation protocols differ, with some papers evaluating on sampled test subsets. And answer-matching conventions are not uniform across implementations.

Third, and most concretely, one paper's own variants differ by the order of a claimed improvement. Think-on-Graph reports 57.1 and 67.6 on CWQ for the method in Table 2.2 and 58.9 and 69.5 for ToG-R, a variant that explores relation chains rather than triple paths, on the same two backbones. Plan-on-Graph's own comparison table quotes the first pair exactly. The gap between the variants is 1.8 and 1.9 points, the same order as many improvements claimed in this literature, and a reimplementation on a different substrate is a further variant of the same kind, which is the situation this thesis is in with respect to its own agentic baseline (Section 5.4.3). A cross-paper

table cannot distinguish a real advance from a variant or reproduction difference of that size. That is why no claim in this thesis rests on one.

2.5.1 Positioning of This Work

Table 2.1 makes the gap explicit. Decomposition is well established, in both PoG and GoG. Backtracking exists in PoG, and faithful grounding by construction exists in RoG. But in the final column, *no surveyed system decomposes its draft answer into atomic claims and checks each against the retrieved triples before responding*. PoG's Reflection corrects the search while it is running. RoG's answers are grounded because of how they are produced, not because anything examined them. The slot is open, and Section 1.6 states what this thesis puts in it. It is the Structural Verification Layer (Section 4.10), with a measurement of what that layer buys. Alongside it come a component ablation pricing each agentic mechanism in accuracy and tokens (Section 5.13) and the controlled agentic comparison itself (Section 5.17.1).

Two features of that positioning belong here rather than there. Both are claims about the literature. The first is that these mechanisms are typically reported as bundles. So which of them earns its cost is not established. That is the gap the ablation fills, and where the planner's stratum dependence appears. The second is that the thesis does not depend on beating prior systems on raw Hits@1. The comparison that matters is against the same architecture with the verification layer removed, on the same graph, with the same backbone. That is why Section 5.13 rather than Table 2.2 carries the argument. Where AGR is compared against a prior system, that comparison is run under controlled conditions in this thesis's own environment rather than quoted across papers.

Chapter 3

The Knowledge Environment

Everything the agent can know, it knows from the graph. So the environment sets a ceiling on how accurate any system in this thesis can be. A result reported without that ceiling is impossible to interpret. This chapter describes how the environment was built and what it holds. Just as important, it describes what the environment cannot express.

3.1 Design Goals and Source Data

3.1.1 Decoupling the Graph Source from the Question Sets

The original proposal for this work described loading WebQSP and FactBench into a graph database. That description mixed up two different roles. The correction is worth spelling out, since it shaped everything that came after.

WebQSP [19] is not a knowledge graph. It is a set of questions, each annotated with a semantic parse over Freebase [18]. The graph is Freebase, and WebQSP supplies questions and gold answers against it. The two roles are kept strictly apart here. Freebase-derived triples make up the environment. WebQSP and ComplexWebQuestions [20] supply questions and gold answers, and they contribute no facts to it.

FactBench [44] was dropped entirely. It had been meant to serve twice, as a source of graph content and as the hallucination evaluator. That double role is circular. If the evaluator's facts sit inside the graph the system retrieves from, a low hallucination rate follows automatically. Groundedness is instead measured against the graph by an independent judge, as Section 5.5.5 describes.

3.1.2 Why a Curated Subgraph Rather Than LLM-Extracted Triples

The proposal also called for a language-model extractor to pull entities and relations out of text and build the graph from them. That approach makes sense when the source is unstructured text. It is what GraphRAG does [4], but here it would defeat the purpose. The graph is the reference that model output is checked against. Building that reference with a language model would introduce the very errors the thesis sets out to measure. And no claim about hallucination reduction would survive the observation that the ground truth was itself generated.

The environment is therefore assembled exclusively from curated Freebase triples. Language models are used to *navigate* the graph and to *judge* answers, never to populate it.

3.1.3 Source Datasets

Every dataset this thesis consumes is public, standard, and third-party. None of it was collected, authored, or annotated here. Three artifacts are used, and the subsections below describe each in turn. A Freebase snapshot is the knowledge source, and the WebQSP and ComplexWebQuestions question sets are the benchmarks. All three arrive through one distribution. It is the set of per-question subgraphs published with Reasoning-on-Graphs [9], released as rmanluo/RoG-webqsp and rmanluo/RoG-cwq. So the graph and the questions evaluated over it are guaranteed to be mutually consistent. And any system that has been measured on those benchmarks has been measured on these questions.

What *is* built here is the knowledge environment derived from them. It consists of the union graph of Section 3.2, its property-graph store, and its vector index. That derivation is deterministic and scripted. The environment it produces is recorded in Table 3.1. The distinction is worth stating plainly, since the two are easily conflated. The datasets are standard and unmodified. The environment assembled from them is this thesis's own, and Section 3.3 measures what the assembly costs. Which questions are actually scored, and how they are sampled from these datasets, is a separate matter specified in Section 5.2.

The Freebase Snapshot Freebase has been frozen since 2016. That freeze makes it stable and reproducible. But it also dates it. That property cuts both ways, and it motivates the no-retrieval control in Section 5.4. Two routes to a working Freebase environment were considered.

The first is a full SPARQL endpoint, served from the community-standard preprocessed Virtuoso image. It ships a database of over 53 GB. Its maintainers recommend on the order of 100 GB of RAM for acceptable query latency. Its advantage is a complete and realistic search space, including the hub entities of very high degree that make naive traversal explode.

The second is the per-question subgraph distribution published alongside Reasoning-on-Graphs [9], in which each record pairs a question with its topic entities, gold answers, and

the triples of a multi-hop neighbourhood around those topic entities.

The second route was chosen, for a reason that should be recorded plainly. The hardware for the first was not available. The consequence is a scoped environment rather than all of Freebase. Section 3.3 quantifies what that costs.

WebQSP and ComplexWebQuestions as Question Sets WebQSP [19] contains questions whose answers lie predominantly one or two hops from the topic entity. WebQSP descends from WebQuestions [23]. ComplexWebQuestions [20] is constructed by composing WebQSP queries into deeper chains with conjunctions, superlatives, and comparatives. Of the two, CWQ is the real multi-hop benchmark, but both are reported on purpose. WebQSP makes the results comparable with prior work. CWQ is where the multi-hop claim is actually put to the test.

3.2 Graph Construction

3.2.1 Union of Per-Question Subgraphs from the RoG Distribution

The environment is the *union* of every per-question subgraph in the RoG distribution, across both datasets and all three splits. Each record's triple list is read, whitespace-normalised, and added to one global deduplicated set. Entity and relation strings are interned to integers, so the union and its adjacency structure fit in memory.

Taking the union is not a minor detail. A per-question subgraph on its own is close to an oracle. It was extracted to contain the answer, so an agent searching it meets almost no distractors. Merging every question's neighbourhood into one global graph brings the distractors back. An agent exploring one question's topic entity now runs into edges that unrelated questions contributed. WebQSP contributes 3,791,302 unique triples over 1,298,304 entities. Adding CWQ brings the union to 8,309,194 triples over 2,592,892 entities.

Two consequences must be disclosed. First, the RoG subgraphs were pre-extracted so as to guarantee answer reachability. That guarantee makes the resulting environment friendlier than raw Freebase. It has fewer irrelevant hub explosions, and a higher answer-reachability rate than a BFS extraction from the full store would produce. That friendliness is a threat to external validity, revisited in Section 5.18. The mitigating argument is that *every system compared in this thesis navigates this identical environment*. Relative comparisons therefore remain sound even where absolute numbers are optimistic.

Second, the distribution's triples are keyed by human-readable entity names rather than by Freebase machine identifiers. The node identifier is therefore the name string. As a result, distinct real-world entities sharing a surface form are merged into a single node. That merging is inherent to name-keying and is also revisited in Section 5.18.

3.2.2 Relation Filtering and Noise Removal

An extraction from raw Freebase would require aggressive predicate filtering. It would have to remove administrative namespaces, type and key predicates, and above all type.type.instance, which contributes millions of edges per type node. Degree capping would likewise be mandatory, since uncapped breadth-first expansion from a hub entity explodes at the second hop.

No such filtering was applied here, and the omission is structural rather than an oversight. The RoG distribution ships subgraphs that have already been scoped around topic entities. So the administrative predicates and hub explosions that filtering exists to remove are largely absent from the source. The only transformation applied to relation names is sanitisation into Cypher-legal identifiers, described in Section 3.2.4. What survives is a vocabulary of 7,058 distinct relation types. That vocabulary is the relation search space the agent faces at every exploration step.

3.2.3 Treatment of Mediator Nodes and Its Effect on Hop Counts

Freebase reifies n-ary facts through mediator nodes, as described in Section 2.1. In the name-keyed distribution these surface distinctively. Named entities appear as their names, while unnamed mediators survive as raw machine identifiers of the form m.0d05w3 or g.11b62t84jq. Nodes are flagged as mediators by matching that pattern. The flag is stored as an is_cvt property.

Mediators are *retained*, not collapsed into hyper-edges. Retaining them preserves the facts they encode, and systems that skip mediators lose exactly those facts [11]. The cost is inflated raw hop counts, since one logical hop through a mediator is two graph edges. The conversion is roughly two raw hops per logical hop.

Two conventions follow, and they are not the same one. Every statement about *distance in the graph* is in *raw* hops, because that is what the graph actually contains. This covers the reachability gate of Section 3.3, the hop strata of Section 5.2, and the path-length bounds quoted anywhere in this thesis. The agent's depth budget is different, since its max_depth counts explorer expansions. One expansion crosses a mediator transparently. So it may consume two raw edges while costing one unit of depth. That discount is deliberate. The agent should not be charged twice for an artifact of the encoding (Section 4.2.3). But the discount also means max_depth $= 4$ does not bound the search at four raw hops. The two numbers should not be compared without the conversion.

The scale of this decision is easy to understand. **Of the 2,592,892 nodes, 1,652,618 (63.7%) are mediator-form nodes carrying no human-readable name.** Nearly two thirds of the environment is connective tissue rather than nameable entities. That imbalance is why the tool layer in Section 4.2 must traverse through mediators transparently rather than present them to the agent as candidate answers.

Table 3.1: The knowledge environment as constructed and imported.

Property	Value
Entities (nodes)	2,592,892
Triples (relationships)	8,309,194
Node and relationship properties	16,087,870
Distinct relation types	7,058
Mediator-form nodes (is_cvt)	1,652,618 (63.7%)
Triples contributed by WebQSP subgraphs	3,791,302
Entities after WebQSP alone	1,298,304
Rows skipped during import	0
Import wall-clock time	36.4 s
Peak import memory	1.09 GiB
Neo4j version	Community 5.26.28

3.2.4 Graph Store Construction

Property-Graph Schema Entities become :Entity nodes carrying an integer id, the name string, and the boolean is_cvt flag. Triples become typed relationships whose type is the sanitised relation name, with the original dotted string retained as an fb_name property. Sanitising replaces non-alphanumeric characters with underscores, since Freebase predicates contain dots that are illegal in unquoted Cypher relationship types.

Native relationship types were chosen over a single generic type carrying a name property. The alternative simplifies import but slows filtered expansion. Filtered expansion is the agent's hottest operation. Every exploration step retrieves the relations of an entity and then the neighbours along one of them.

Integer identifiers were chosen over names as the import key. Entity names contain commas, quotation marks, and unicode, all of which invite escaping bugs and silent collisions in bulk-import CSVs. The name survives as an ordinary property and is what the entity resolver matches against.

Import used Neo4j's offline bulk importer, with the uniqueness and index constraints created afterwards. Appendix E.18 records the versions and the constraint set.

Graph Statistics Table 3.1 records the environment as built. These figures, together with the dataset versions and the hop cap, constitute the reproducibility record for the environment.

The zero skipped rows deserve emphasis. The import was deliberately configured to tolerate skips. Node and relationship counts in the database match the CSV row counts exactly. So the count-reconciliation check of Section 3.3 passes without any unexplained gap to account for.

3.2.5 The Vector Index

Entity names and the relation vocabulary are both embedded with all-MiniLM-L6-v2 [29], a 384-dimensional sentence encoder. Appendix E.17 records how the two indexes were built and checked. One property of the relation index is load-bearing for Section 4.4. Relation names are *verbalised* before embedding. So people.person.place_of_birth is encoded as the phrase it means rather than as a namespaced path. That verbalisation is what lets a question phrase score against a schema identifier at all.

The role of embeddings in this thesis is deliberately narrow. They link question phrases to graph entities and pre-rank relation candidates during exploration. They never establish that a fact is true, a role that belongs exclusively to the symbolic triples. Every claim the verification layer certifies is certified against an edge, not against a cosine.

3.2.6 Entity Resolution and Surface-Form Linking

The Exact, Lexical, and Vector Cascade Mapping a question's surface mention to a graph node uses a three-stage cascade, each stage attempted only if the previous one fails.

Exact matching queries for a node whose name equals the normalised mention. Because the environment is name-keyed and the benchmarks supply topic entities as names, this resolves nearly everything.

Lexical matching queries the full-text index, handling typographical variation and partial names. Mentions are escaped for Lucene syntax before the query. Benchmark entity names contain parentheses, colons, and slashes, all of which are Lucene operators that would otherwise crash or silently distort the query.

Vector matching encodes the mention and queries the entity vector index, handling paraphrases and misspellings that survive the first two stages.

The cascade is referred to by these names throughout, never as "tiers". This thesis reserves "tiers" for the two-tier groundedness metric of Section 5.5.3. That naming rule is about this prose, not about the code, which does call them tiers. The resolver returns a tier field taking the values 1, 2 and 3. A reader moving between the two will meet the collision. Appendix B declares it where the field is specified, rather than leaving the disclaimer above to look like a claim about the implementation.

Threshold Selection on Development Data The lexical stage accepts its top hit only when the full-text score exceeds 1.0, falling through to vector matching otherwise. The threshold exists because the full-text index returns a ranked list for almost any input, including inputs that match nothing meaningfully. Without a floor, the lexical stage would confidently return the least-bad token overlap. The vector stage would never be reached.

Table 3.2: Entity resolution over all topic-entity mentions in both test sets. A mention counts as resolved only when the returned node's name equals the mention exactly.

Dataset	Mentions	Exact stage	Unresolved
WebQSP	1,661	1,661 (100%)	0
CWQ	5,488	5,478 (99.8%)	10
Total	**7,149**	**7,139 (99.86%)**	**10 (0.14%)**

Resolution was measured over every topic-entity mention in both test sets. Table 3.2 reports the outcome.

Surface forms supplied by the benchmarks therefore resolve to nodes in this environment essentially without loss. The limits of that measurement should be read alongside it. The graph is keyed on the same name strings the benchmarks supply, both coming from the same RoG distribution. The measurement therefore establishes only that the *resolver* does not lose entities the environment contains. It does not show that entity linking would be easy against an independently-built graph, or that linking from free text would work at all. Topic entities are given rather than linked throughout (Section 5.18). What follows for the error analysis of Section 5.14 is the narrow version. A failure in this environment cannot be attributed to the resolver having failed to find the question's topic.

3.3 Environment Validation Gate

No experiment was run until the environment was certified. The gate has three parts of increasing strength.

3.3.1 Answer-Reachability Protocol

For every test question, two properties are computed over the union graph. *Existence* asks whether the gold answer entities appear as nodes at all. *Reachability* asks whether at least one gold answer is connected to at least one topic entity within a bounded number of hops.

Reachability is computed by breadth-first search treating edges as *undirected*. The agent's neighbour-retrieval tool looks in both directions, so the gate has to do the same. It must mirror how the agent actually moves through the graph. The bound is four raw hops. By Section 3.2, that works out to roughly two logical hops once mediators are counted. That is enough for all of WebQSP and most of CWQ. A question whose topic entity is itself a gold answer counts as reachable at zero hops.

The reachability rate is the **accuracy ceiling**. If a gold answer cannot be reached, no navigating system can find it, no matter how well it is designed.

Table 3.3: Environment coverage over the full test sets, at a bound of four raw hops. The reachability column is the Hits@1 ceiling this environment induces over the whole benchmark. The evaluation draws 400-question samples from these splits, whose own ceilings differ slightly by sampling and are the operative bound on the reported results (Table 5.1).

Dataset	Test questions	All answers exist	≥1 exists	≥1 reachable
WebQSP	1,628	94.7%	97.3%	**97.3%**
CWQ	3,531	99.4%	99.7%	**99.7%**

3.3.2 Coverage Results and the Induced Accuracy Ceiling

Table 3.3 reports what the gate found.

Three things stand out. Every question in both datasets has at least one topic entity present in the graph. So no question is lost at the entry point.

More strikingly, *existence and reachability are identical* in the ≥ 1 column. Both are 1,584 of 1,628 for WebQSP and 3,519 of 3,531 for CWQ. Every question with at least one gold answer present in this environment has at least one reachable within four hops. The two columns agree question for question, not merely in total. The claim is deliberately the weaker of the two available. The *all answers exist* column is lower, at 94.7% against 97.3% on WebQSP. So on 42 WebQSP questions some but not all gold entities are present, on top of the 44 where none are. And this measurement says nothing about whether the ones present are all reachable. What the measurement does say is that reachability is never the binding constraint on *answerability*. When a question is unanswerable here, it is because entities are absent altogether. That pattern is a direct consequence of the construction in Section 3.2. The subgraphs were extracted around topic entities precisely to guarantee reachability. The measurement confirms that the property survived unioning.

Finally, CWQ's coverage exceeds WebQSP's, which inverts the expected ordering. The deeper benchmark has the better-covered answers. Two things account for this inversion, and they are not competing. The larger is constructional. CWQ contributes far more questions and therefore far more subgraphs to the union. So its own answers are better represented. The residual WebQSP shortfall that this leaves is semantic. It is dominated by answers that are not entities at all, a gap Section 3.3.3 takes up.

The ten unresolved mentions of Table 3.2 are not a random residue. Every one is a bare machine identifier of the form g.123852dg that the benchmark carries in place of a name. So no surface form exists for the resolver to match. Appendix E.23 reads them. The strongest check the gate makes is not the linking rate but the re-verification of reachability directly in Cypher, independently of the Python that computed it. That independent re-check is what makes Table 3.3 a certification rather than a self-report.

3.3.3 What the Environment Cannot Express

Certifying reachability establishes that answers which *are* entities can be found. That certification says nothing about answers that are not entities. This is where the environment's real limits lie. The import schema of Section 3.2.4 gives every node a single :Entity label. There is no typed literal node, no datatype, and no distinction between an entity and a value. A date or a number can therefore exist in the graph only incidentally, as an entity whose *name* happens to be a numeric string. Measured over all 2,592,892 nodes by scripts/count_literals.py, which writes results/phase1/literal_census.json:

- **Dates** are effectively absent. Only 77 nodes carry a full YYYY-MM-DD name and 175 a bare four-digit year. Together they are under 0.01% of the graph, and untyped. So these dates cannot be compared, ordered, or filtered by range.
- **Numeric values** appear on 12,799 nodes (0.49%) under the strict rule that the entire name is a single number. But the population is dominated by identifiers such as ISBNs rather than by measurable quantities. That count is sensitive to its own definition. That sensitivity is why the definition is committed alongside it. Admitting names that merely carry no letters ("(1955)", "12-4") raises it to 13,888. The argument does not depend on which count is used, since both are under 0.6%. But the figure is not a property of the graph by itself and depends on how it was counted.
- **Ordinals and superlatives** cannot be expressed at all. That gap comes from the tool layer, not the data. Section 4.2 exposes no aggregation, sorting, or comparison operator. So "the smallest" or "the earliest" cannot be computed even when every candidate is present in the graph.

Together, these three gaps define a class of questions that is *unanswerable in this environment*. It contains any question whose gold answer is a date, a measured quantity, or a superlative selection over candidates. Such a question is not a reasoning failure when the system misses it. The environment could not have supplied the answer. The class is referenced by name in Section 5.16.1, where it accounts for a substantial share of the residual failures. The class is also a known consequence of a design decision. Modelling literals as typed nodes was considered and not adopted, in exchange for a uniform schema and a simpler tool API.

Chapter 4

The AGR Framework: Architecture, Navigation, and Verification

This chapter specifies the Agentic Graph Reasoning framework. It sets out how the agent is structured, what operations it may perform on the graph, how it decides where to look next, how it recovers from wrong turns, and what guarantees bound its cost. The verification layer is specified in Section 4.10. Everything before it is the machinery that produces the traversal it verifies. That layer is the component this thesis set out to contribute. Section 5.7 measures its contribution as narrower than intended. The contribution is auditability rather than accuracy. Even the precision advantage over the agentic baseline belongs to the architecture as a whole rather than to the layer. So the layer is introduced here as one part of the architecture rather than as the part the rest exists to serve.

4.1 Architectural Overview

4.1.1 The Controlled-Agent Pattern

There are two ways to build a graph-navigating agent. The first hands the model a set of tools through native function-calling and lets it drive the loop itself, ReAct-style [27]. The second inverts control. The *program* orchestrates the loop, calls the graph tools deterministically, and consults the model only at decision points, through constrained prompts that must return JSON. AGR uses the second, a choice deliberate enough to be worth defending. It fixes what the word "agentic" means in this thesis. Four things follow from it:

- **Every graph operation is logged by construction.** The traversal is not pieced together from the model's own account of what it did. It is the literal sequence of tool calls the program made.

- **Budgets are enforced in code, not requested in prompts.** A prompt that asks the model to stop after four hops is a suggestion. A counter checked in a router is a guarantee (Section 4.7).
- **Failures are reproducible.** A malformed model response can spoil one decision, but it cannot corrupt the control flow. Every call goes through one wrapper, which parses the response as JSON. If that fails, it spends a second metered call asking the model to repair its own output. So when a node below is said to "make one model call", that is the normal case, not an upper bound. The second call is charged to the budget like any other. The call counts in Section 5.12 therefore already include whatever repairs occurred.
- **Each model call is small, single-purpose, and cacheable.** That combination is what makes the response cache of Section 4.8 viable.

The agency in AGR therefore lives in the *decision policy*, not in unconstrained tool invocation. That policy decides which sub-objectives to pursue, which edges look promising, whether the current path is a dead end, and whether the draft answer is supported. Prior systems in this space make the same choice [7, 8]. Stating it explicitly forecloses the objection that AGR is merely a prompt around a function-calling loop.

4.1.2 The State Machine

The framework is a state machine. It is a directed graph of six nodes over a shared typed state, built with LangGraph. Figure 4.1 gives the topology.

Two properties of this topology matter. The topology contains *cycles*. That is why a state-machine framework is used rather than a linear pipeline. Exploration loops until the evaluator is satisfied. Verification can send the agent back to explore. The verifier also sits *between* the decision to answer and the answer itself. That placement is the structural expression of this thesis's contribution. There is no path from evaluator to answerer that does not pass through verification.

4.2 The Graph Tool API

The five operations are specified in full, with the Cypher each executes, in Appendix B.

4.2.1 Rationale: Constrained Tools Instead of Free-Form Cypher

An obvious alternative is to let the model write Cypher directly. That alternative was rejected. Generated-query approaches introduce a failure mode that is indistinguishable, in the results,

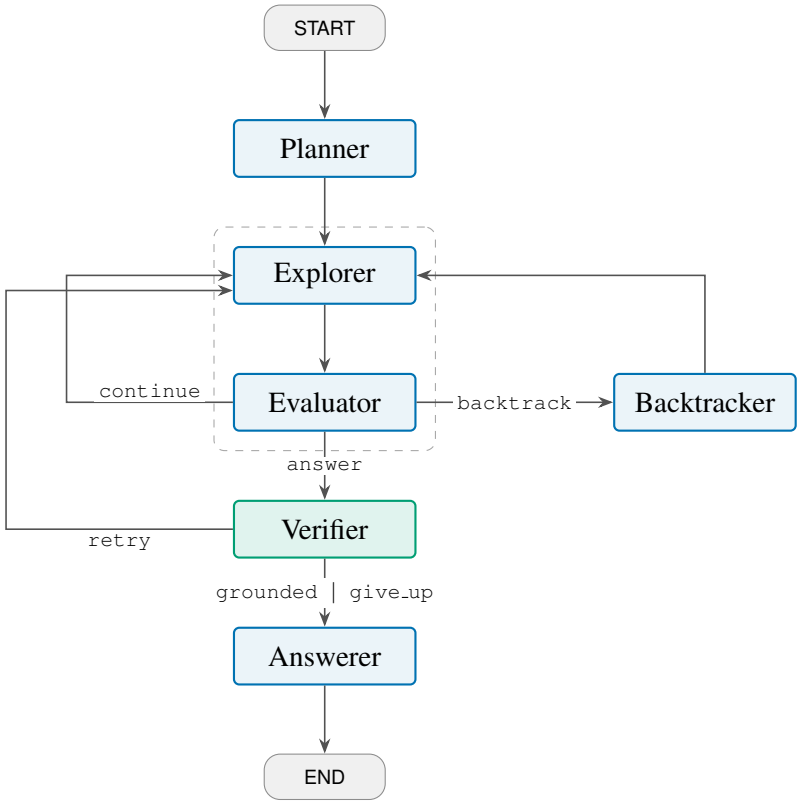


Figure 4.1: The AGR state machine. Three cycles exist. The dashed region marks the Explorer ↔ Evaluator search loop; Evaluator → Backtracker → Explorer restores an earlier frontier; and Verifier → Explorer is verification-driven re-exploration. All three are bounded by the budgets of Section 4.7 rather than by the graph structure. Every path from Evaluator to Answerer passes through the Verifier. That routing is the structural expression of this thesis's contribution.

from a reasoning failure. When a question is answered incorrectly, one cannot tell whether the agent reasoned badly or merely emitted invalid syntax. Since the thesis's central claims are about reasoning and verification, a confound that contaminates every error category is unacceptable.

The tools are therefore parameterised Cypher templates (Section 5.6.2), written once and unit-tested. They accept entity identifiers and return names alongside them. So the model always sees human-readable text while the program tracks identity. Every invocation is appended to a JSONL log with its arguments, result, and latency.

4.2.2 The Five Operations, of Which Four Are Live

Table 4.1 lists the five operations, not the four originally planned. Only four are reachable from a node in the final design. verify_triple, built first as the claim checker, did not survive contact with generated text. Claim decomposition produces relations phrased in English, while Freebase predicates are neither phrased that way nor reliably one edge long. So requiring the relation to match rejected claims that were true. verify_connection, added in response, drops the

Table 4.1: The graph tool API. All operations are deterministic parameterised Cypher; none consults an embedding except search_entity, which delegates to the resolver of Section 3.2.6.

Operation	Returns	Role
search_entity(*s*, k)	up to k candidate entities with id, name, score, and resolver stage	anchor seeding; the only entry point from text to graph
get_relations(*e*)	relation types incident to e in *both* directions, each with its fanout count	candidate generation for the scorer
get_neighbors(*e*, *r*, *d*)	entities reached from e along r in direction d, each with the via chain that reached it	frontier expansion
verify_triple(*h*, *r*, *t*)	{direct, via_cvt, supported}	*not called by any node*; built for claim checking and superseded by verify_connection (Section 4.2.2)
verify_connection(*a*, *b*)	{direct, via_cvt, connected}	relation-agnostic claim checking; the check the verifier actually uses (Section 4.2.3)

relation and asks only about adjacency. It superseded verify_triple rather than complementing it (Appendix E.28). It is left in the API as the natural primitive for the relation-aware check Section 6.3 proposes. But nothing in the running loop calls it.

Three design decisions inside these operations are load-bearing.

Relations are returned with fanout counts, in both directions. The count is free signal. A relation with a fanout of forty thousand is almost never the edge that answers a specific question. The scorer can use this signal without an extra query. Retrieving incoming as well as outgoing edges matters. Freebase's relation direction is a modelling artefact. A question about where someone was born may be answerable through people.person.place_of_birth outward or location.location.people_born_here inward.

Administrative predicates are filtered at the tool layer. Relations whose names begin with common., freebase., type., kg., user., dataworld., rdf-schema#, or owl# are dropped before the candidate list is built. These carry no factual content. Section 4.9 shows what happens when they are not filtered. They consume beam slots and drag the agent into the schema rather than the data.

The relation list is capped at 300 entries, sorted by fanout. An earlier cap of 80 was raised after it was found to hide precisely the low-fanout, high-precision relations that answer specific questions, as Section 4.9 also records.

4.2.3 Mediator Traversal and Relation-Agnostic Adjacency

Section 3.2 established that 63.7% of the environment's nodes are unnamed mediators. If the agent saw these as candidate entities, most of its frontier would consist of nodes with no surface form and no meaning to a language model.

Table 4.2: The agent state. Fields marked *append-only* accumulate across node executions through reducers; all others are overwritten by whichever node returns them.

Group	Fields
Question	qid, question, gold_q_entities
Plan	plan, a list of ordered sub-objectives, each with a status (pending/active/done) and its resolved entities
Search	anchors (current frontier), traversed (*append-only* triple log), backtrack_stack, banned, last_expanded
Answer	candidate_answers, draft, answer, answer_entities, supporting_triples
Verification	supported_claims, unsupported_claims, unsupported_claim_objs
Control	budget (meter, mutated in place), trace (*append-only*), and the router scratch keys _eval_decision, _last_n_new, _frontier_max_score

get_neighbors therefore hops through them transparently. When an expansion reaches a node flagged is_cvt, the tool immediately expands that node one further step, excluding the origin and excluding other mediators, and returns the entities on the far side. Each returned neighbour carries a via chain, with one relation for a direct edge and two for a mediator-mediated one. The model reasons about logical neighbours. The log keeps the full raw chain, so hop counts stay exact.

verify_triple applied the same principle to checking. It matched *undirected*, and it accepted a mediator in the middle. Being strict about direction would have produced false rejections. The claims it was given came from generated text, and generated text has no idea which way a Freebase edge points. That relaxation was necessary but not sufficient, as Section 4.2.2 records. Relaxing direction does not help when the relation itself has no Freebase counterpart.

verify_connection goes one step further and drops the relation entirely. It asks only whether two entities are adjacent, either directly or through one mediator. It exists because claim decomposition often produces a claim whose relation matches no single Freebase predicate. "X played for Y" may be stored through a roster mediator with no edge literally named for playing. Requiring an exact relation match in such cases rejects true claims. verify_connection is the adjacency test the verification layer uses. Section 4.12.1 gives the two routes it sits in. The cost of dropping the relation is stated there and not here. An adjacency test cannot tell a true claim from a false one that happens to join the same two entities. That gap is the boundary case Section 5.15.3 analyses.

4.2.4 Agent State

The state is a typed dictionary threaded through every node. Table 4.2 groups its fields by purpose.

Two disciplines govern this structure. Both are worth stating, since violating either produces bugs that are hard to localise.

Nodes write, routers read. A router is a pure function that inspects state, returns an edge label, and never mutates. Any signal a router needs must therefore be deposited in state by the node preceding it. That requirement is what the three underscore-prefixed scratch keys are for. The explorer records how many triples it added and the best score it saw. The evaluator records its decision, and the routers read those values and decide.

The budget meter is mutated in place and never returned as a delta. The meter is one mutable object that every node shares by reference. A counter that is merged rather than mutated in place can be silently lost when two nodes return overlapping state. Enforcement must not be able to drift like that.

4.3 The Planner

4.3.1 Decomposition into Ordered Sub-Objectives

The planner makes one model call. It turns the question into an ordered chain of sub-objectives, plus the entity mentions that seed the search. The chain is strictly sequential. Later objectives refer back to earlier results with a #N back-reference. A general dependency-graph planner was not built. A sequential chain covers the composition structure that dominates CWQ. A DAG planner is a research problem in its own right.

The prompt is reproduced in full as Appendix A.1. It is few-shot, with one example per question archetype. The archetypes are single hop, composition, conjunction, and superlative. The single-hop example is there to show that *not* decomposing is a valid output. Over-decomposing simple questions is a real failure mode. Section 5.13 shows it is not hypothetical. On the shallower benchmark, removing the planner improves accuracy.

The prompt also instructs the model to extract only specific named entities as topic mentions, and explicitly not generic type words such as “celebrity”, “university”, “country”, or “senator”. That instruction was added in response to evidence discussed in Section 4.9. Type words resolve successfully to type nodes in the graph, consuming anchor slots with entities that lead nowhere.

4.3.2 Plan Validation and Degenerate-Plan Handling

The planner’s output is validated by four checks, not trusted. The plan must contain a non-empty list of sub-objectives, and no more than four of them. Every #N reference must point strictly backwards, and topic mentions must be present. The back-reference check catches the planner’s most common structural error, a step that references itself or a future step.

If any check fails, or if the model call itself raises, the planner falls back to a single sub-objective consisting of the whole question, seeded with the dataset's topic entities. That fallback is a deliberate design principle. **Planning can degrade but can never abort a run.** A degraded plan is a data point that shows up in the results. A crash is a lost question that biases them.

Anchor seeding has two modes. In the default benchmark configuration, anchors are seeded from the dataset's annotated topic entities. That choice separates reasoning from entity linking. It is also the standard protocol in this literature [7, 9]. The alternative seeds from the mentions the planner itself extracted, which gives the end-to-end number. Section 3.2.6 showed that resolution succeeds on 99.86% of mentions. So the two modes differ very little in this environment.

Setting the planner ablation flag skips the model call entirely and installs the fallback plan directly. That substitution is what makes "no planner" a one-line configuration change rather than a separate code path.

4.4 The Explorer

The explorer performs one expansion of the frontier. It is the only node that touches the graph in bulk. Its structure is the heart of the navigation policy.

4.4.1 Frontier Construction and the Single Scoring Pass

The node works in five steps:

1. **Checkpoint.** The current frontier, the current depth, and the best score seen so far are pushed onto the backtrack stack.
2. **Gather.** For every anchor, get_relations is called. The results go into a *single* candidate list, with each row tagged with the anchor it came from.
3. **Score.** The whole list is scored in one pass (Section 4.4.2). Pooling candidates across anchors before scoring is what allows the hybrid scorer to consult the model *once* per expansion rather than once per anchor.
4. **Beam.** Candidates are taken in descending score order into per-anchor buckets of width three, skipping any (anchor, relation, direction) triple on the ban list (Section 4.5.3).
5. **Expand.** Each selected edge is expanded with get_neighbors; the resulting triples are appended to the traversal log, and the neighbours become frontier candidates.

The new frontier is deduplicated by entity identifier, sorted by the score of the edge that produced each entity, and truncated to the anchor cap. Sorting by score rather than by insertion order

matters. An insertion-ordered frontier keeps whichever entities happened to be produced first. That bias causes the agent to drift away from the anchor that mattered.

4.4.2 The Hybrid Scoring Function

Each candidate edge receives a hybrid score blending semantic similarity with a model judgement; the prompt carrying the second is Appendix A.2:

$$\text{score}(c) \;=\; \alpha \cdot \cos\big(\mathbf{v}_{\text{rel}(c)},\, \mathbf{v}_{\text{obj}}\big) \;+\; (1-\alpha)\cdot \text{LLM}(c),$$

where $\mathbf{v}_{\text{rel}(c)}$ is the precomputed embedding of the candidate's relation name (Section 3.2.5), $\mathbf{v}_{\text{obj}}$ is the embedding of the *rendered active sub-objective*, and $\text{LLM}(c) \in [0, 1]$ rates how likely following that edge from that anchor is to advance the sub-objective.

Three properties of this design deserve emphasis.

The objective, not the question, is embedded. Hop two of a two-hop question is scored against "find the director of Titanic", with the back-reference already substituted by the resolved entity name, rather than against the original question text. That substitution is the concrete mechanism by which decomposition is supposed to help. Isolating it is what makes the planner ablation interpretable.

The model scores only a shortlist. Candidates are pre-ranked by embedding similarity and only the top twenty are shown to the model, in one batched call. Without the pre-filter, an expansion from five anchors with hundreds of relations each would be unaffordable.

The equation above understates what that shortlist does, a gap worth closing here. A candidate outside the top twenty is not scored with the model term *omitted*. It is scored with $\text{LLM}(c) = 0$, the lowest rating the model could have given it. The embedding pre-filter is therefore a hard gate rather than a soft ranking aid. At $\alpha = 0.7$ an unrated candidate is penalised by up to 0.30 against an identically-similar rated one. So it can only re-enter the beam if its embedding similarity exceeds the rated candidate's by that margin. That penalty is defensible. The pre-filter is an embedding ranking, and a candidate below the cut is one the embedding already judged worse. But it means the twenty-candidate cut interacts with α rather than sitting orthogonally to it. A larger α softens the gate as a side effect of weighting the model less.

The blend is applied after the call, not inside it. The scoring prompt contains no configuration value, neither α nor τ. That omission is not cosmetic. The response cache (Section 4.8) keys on prompt content, so a prompt free of configuration means every condition in an α sweep shares the same cached model responses. So the sweep costs one set of calls rather than one set per condition. Putting a configuration value into that prompt would multiply the sweep's cost by the number of conditions and change no result.

If the model call fails or the budget runs out, the scorer treats every model score as zero rather

than abandoning the expansion. So each candidate scores $\alpha \cdot \mathrm{emb}$. That is the embedding ordering scaled by α, not the embedding score itself. The distinction is worth stating precisely. It does not matter here, though it could be assumed to. Since $\alpha > 0$ is a positive scaling, the ranking is identical. The backtracking trigger compares τ against σ. That threshold takes the unscaled embedding maximum as one of its terms (Section 4.5.3). No reported number changes on this path. Setting $\alpha = 1$ disables the model term entirely. That configuration is the embedding-only ablation.

4.5 The Evaluator and Routing Policy

4.5.1 Sub-Objective Completion Judgment

The evaluator makes one model call per expansion. It is shown the active sub-objective, a window of retrieved facts, and the remaining budget. It returns a decision (continue, backtrack, or answer), whether the objective is complete, and which entities satisfy it.

The fact window is *relevance-ranked, not recent*. Traversed triples are verbalised, embedded, and the thirty most similar to the current objective are shown, in traversal order. The distinction matters, because a large expansion can add hundreds of triples. A window that shows the most recent thirty would then push the answer out of view in exactly the cases where the expansion succeeded. Section 4.9 documents the case that forced this change.

4.5.2 The Groundedness Filter on Resolutions

The evaluator's resolved entity names are not taken on trust. Each is looked up in a table built from the tail entities of the *traversed* triples. Any name not in that table is discarded.

The groundedness filter is small in code and large in effect. Ask a language model which entities satisfy an objective, and it will happily list entities it knows from training but never retrieved. The filter refuses to let that parametric recall pass as the agent's evidence. Section 4.9 records a case where the evaluator confidently resolved nine correct countries that appeared in no retrieved triple. The filter rejected all nine, which cost a point on that question. But it preserved the property that every entity the system proposes came from the graph.

The table is built from tail names only. So an entity that appeared only as the head of a retrieved triple is not in it. A resolution naming such an entity is dropped even though the traversal did reach it. Appendix E.27 explains why that asymmetry is deliberate in the evaluator, and where it is made up for elsewhere.

Resolutions that survive the filter are added to candidate_answers on *every* evaluation, whether or not the objective was judged complete. An earlier design threw resolutions away when the objective was incomplete. That design lost correct answers found mid-run.

When an objective completes and further objectives remain, the plan advances, the decision is forced to continue, and the resolved entities become the next frontier, truncated to the anchor cap of Section 4.7. When the last objective completes, the decision is forced to answer.

4.5.3 Backtracking Triggers and the Backtrack Stack

Backtracking fires on the first of three conditions, evaluated in this order:

1. **Dead end.** The last expansion produced no new triples.
2. **Low score.** The frontier's *signal maximum* fell below the threshold τ.
3. **Evaluator veto.** The model judged the direction a dead end.

The order matters, because the first two are deterministic and cost nothing. Checking them before deferring to the model's judgement means a mechanically hopeless frontier is abandoned without an argument about it.

The second trigger deserves a precise statement, because it is easy to assume it tests the blended score and it does not. The quantity compared against τ is

$$\sigma \;=\; \max\Big(\; \underbrace{\max_{c\in E} \mathrm{score}(c)}_{\text{blended, expanded edges}} \;,\; \underbrace{\max_{c\in R} \cos\big(v(r_c), v(o)\big)}_{\text{embedding, all candidates}},\; \underbrace{\max_{c\in R_K} s_{\mathrm{LLM}}(c, o)}_{\text{model, rated candidates}} \Big),$$

the largest of three signals. They are the best blended score among the edges actually expanded, the best raw embedding similarity over every candidate relation the anchors offered, and the best model score over the top-K candidates the model was asked to rate. The trigger therefore fires only when *all three* signals are weak, which makes it deliberately conservative. It abandons a branch only on unanimous evidence of hopelessness. Section 5.7.1 shows the consequence, that τ barely fires at the values swept.

The backtracker then does three things. It pops the *highest-scoring* snapshot from the stack. It does not pop the most recent one, which would make backtracking a simple undo. It restores that snapshot's anchors and depth. And, critically, it adds (anchor, relation, direction) triples to a **ban list** that the explorer consults on its next pass. The ban list is what makes backtracking a search operation rather than a loop. Without it, a deterministic scorer facing an identical restored frontier selects identical edges, producing identical facts and an identical decision to backtrack until the budget is exhausted. Section 4.9 shows both behaviours in the logs.

The implementation departs from what this section specifies in three places. All three are recorded rather than repaired, under the standing rule of Section 4.9. Repairing them would change the frozen results this thesis reports. Appendix E.12 gives all three. Two concern the σ

formula and are masked in practice by the dead-end trigger being evaluated first. The third is the one with a measurable consequence.

That third is a misalignment between what the backtracker bans and what it restores. It bans the *most recent* explorer pass. It restores the *highest-scoring* snapshot, frequently older than that pass. So everything expanded in between stays unbanned. The explorer can be handed back a frontier with all of its previously selected edges still available. It then re-selects them, since the scorer is deterministic. Over the main matrix, 53 explorer passes (15 on WebQSP and 38 on CWQ) expanded an (anchor, relation) set identical to one the same question had already expanded. That is exactly the repetition the ban list exists to prevent. The misalignment that produces it is visible in at least 65 of the 262 backtracks, where the restored depth is not the depth the most recent snapshot carried.

4.6 The Answerer

The answerer is a finalizer rather than a generator. Which of its three paths runs is decided entirely by what the verification layer left in state. Two of the three are specified in Section 4.10. One is the *verified* path, where no claim was unsupported and the draft is emitted with no further model call. The other is the *repair* path, where unsupported claims remain and one rewrite call restates the answer around a deterministically filtered entity list (Section 4.13.4). On the development set the verified path handled 77 of 80 questions at zero additional cost.

The third path exists for the case where the verifier raised before producing a draft at all. It makes one model call over the traversed facts and the accumulated candidates. If the model budget is exhausted, it falls back without a call to the resolved entities of completed plan steps, or failing that to the accumulated candidates, producing a degraded but non-empty answer. If the call raises for any other reason, the error is recorded in the trace. The answer is an explicit failure string. That distinction matters, because an earlier version caught all exceptions as budget exhaustion and so misreported genuine errors as expected behaviour.

Every degraded path described here is specified and never exercised. Across all 6,960 committed runs no handler of this kind fired once. Every one of the 3,690 committed records that ran this graph carries a non-empty draft. So the third path is never entered either. That is also why its fact window is described here and nowhere in Section 5.7. Appendix E.21 records what the untested handler actually distinguishes.

4.7 Budgets and Termination Guarantees

Budgets do three jobs. They bound cost, they guarantee termination, and they produce the efficiency data of Section 5.12. Table 4.3 lists them.

Table 4.3: Budget configuration, and the two tool-layer caps that bound retrieval alongside it. The rows above the rule are the budget proper. They are hashed. The hash is recorded in every run record. So results can always be attributed to the configuration that produced them. The two rows below the rule are constructor arguments of KGTools and are *not* in that hash. They are listed here because they bound what the agent can see. That bound makes them budgets in effect if not in implementation. Four budgets carry two enforcement sites rather than one. The paragraph after this table says which and why.

Parameter	Value	Enforced in
max_depth	4	route_after_eval *and* route_after_verify
beam_width	3	explorer (slice)
max_anchors	5	explorer *and* evaluator (slices)
max_backtracks	3	route_after_eval
max_llm_calls	25	LLM client
max_verify_iters	2	route_after_verify *and* the verifier
max_seconds	300	route_after_eval *and* route_after_verify
max_fanout	200	get_neighbors (neighbours returned per expansion); sets truncated when it binds
max_relations	300	get_relations (candidate relations offered to the scorer)

The two tool-layer caps deserve a sentence of their own. They bound recall rather than cost. max_fanout caps neighbours returned from a single expansion at 200. A high-degree entity has more neighbours than that, so the frontier sees a truncated view. An answer sitting outside the first 200 is unreachable through that edge no matter how well the scorer ranks the relation. The hub-noise failures of Section 5.16.2 are where this would be expected to matter. The tool returns a truncated flag alongside the neighbours. Two defects in it are recorded rather than repaired, under the rule of Section 4.9. First, the flag does not reach the tool log. That log records the returned neighbour count instead, and cannot distinguish a capped expansion from one that returned exactly 200. Second, it is computed before mediator traversal, on the raw row count, while the final list is truncated after mediators have been expanded into it. So an expansion returning few raw rows but many entities through their mediators is cut without the flag being set. Both make it a lower bound on truncation rather than a record of it. No analysis in this thesis reads it. The cap's effect is measured from the returned counts instead, as in Section 5.4.3.

Enforcement is confined to routers and the client. That confinement is what keeps it auditable, though the mapping is not one site per budget. Appendix E.11 gives it, with three of the budgets checked in two places rather than one. In every case the second site exists because the first does not cover the verify–explore cycle. The retry edge re-enters the explorer without passing through the post-evaluation router. Without a second check, a repair would quietly buy an expansion past

the depth budget. Section 4.13 records the development trace that exposed exactly that.

Together these give a termination guarantee that does not depend on how the model behaves. Every cycle in Figure 4.1 passes through a router that decrements or checks a counter that only ever moves in one direction. So no sequence of model outputs, however adversarial, can produce a run that never ends. The mechanics tests of Section 5.6.2 check that guarantee directly. They script an evaluator that always says continue and assert that the depth budget halts it.

4.8 Instrumentation and Reproducibility

Two logs are written per run. The *tool log* records every graph operation with its arguments, result size, and latency. The *run log* records, per question, the answer and its extracted entities, the draft, the verification outcome and any unsupported claims, the count of supporting triples, every backtrack with its trigger reason and restored depth, the full decision trace, the budget snapshot, and wall-clock time. For provenance it also records the backbone description, the budget configuration hash, the run configuration, and the current git commit.

The response cache is what backs the thesis's reproducibility claims. Every model call is keyed by a hash over the model identifier, temperature, reasoning-effort setting, the full prompt text, and the schema hint. Hits are served from disk. The subtle part is that, inside the agent, a cache hit must be *indistinguishable* from a live call. The budget check still runs. The call counter still increments. And the original token counts are replayed into the meter from the stored record. So a fully cached rerun reproduces the original run's budget snapshots exactly rather than reporting zero tokens. One exception is a property of the technique rather than of this run. The replay restores prompt and completion tokens but not reasoning_tokens. The cache stores that field and does not add it back. That field is zero throughout this thesis, since the backbone runs at reasoning effort none (Section 5.3). So the snapshots do reproduce exactly here. Anyone reusing the design under a reasoning-effort setting would find they do not. A separate counter records cache hits so that live and replayed runs can be told apart, repeated experiments cost nothing, and a code change that is not supposed to alter behaviour can be verified by confirming that a rerun is served entirely from cache. The key includes the model string, and the backbone configured here is a dated snapshot rather than an alias. So a silent server-side change cannot poison results that were cached earlier.

4.9 Design Validation on the Smoke Set

Two tuning sets appear in this thesis, and they are not the same. This section uses the *smoke set*. It consists of twenty questions drawn from the training and validation splits, never from test. It was used once, to check that the assembled framework's mechanics worked. The 80-question

development set of Section 5.6.1, on which the hyperparameters were swept, is a separate and larger set. Only it carries any reported number.

The first end-to-end run answered roughly three to four questions correctly. The final configuration answers thirteen to fourteen. That interval is not incidental tuning. It is where several of the mechanisms described above were shown to be necessary. Appendix E.7 gives the record. It lists the five systematic root causes the first run's failures collapsed into, the mechanism each one motivated, and three episodes that bear on later chapters. Two of those episodes are the evidence Section 4.10.1 rests on. One is a confident answer no retrieved triple supported. The other is nine entities the evaluator asserted without retrieving. The third is where the standing rule against analysing an unchecked metric comes from.

This is also the one place in this thesis whose *evidence* is not fully archived. What survives is one record file, results/phase3/smoke20_a0.5_t0.2.jsonl, scoring thirteen correct, six hedged and one wrong at $\alpha = 0.5$ rather than the frozen 0.7. What does not survive is the before-state. It was read from a console log and never committed. So the three-to-four figure and the root causes rest on that reading alone. Neither number enters any result. That is why the omission was tolerable, and why the figures are given as ranges.

4.10 The Structural Verification Layer

Section 4.1 described a navigation loop. It ends with a set of traversed triples and a set of accumulated candidate entities. Nothing in that loop constrains what the final answer text *asserts*. The sections that follow specify the layer that does, together with the repair, filtering, and evidence-reporting policies built on it. Except where a sentence names the test sets explicitly, every quantity reported here is recomputed from the archived development-set run at the frozen configuration ($\alpha = 0.7$, $\tau = 0.20$, verify_claims = true, $n = 80$). Where a behaviour is established on a handful of questions rather than a distribution, the count is given in the sentence that makes the claim. The exceptions are in Section 4.13, where what the development set showed and what the 800 test questions show are not the same thing. Reporting only the first would leave this account contradicting Section 5.14.

4.10.1 Motivation: Verifying Before Emitting, Not After

Section 1.3 argued that agentic KGQA systems are typically evaluated on whether the correct answer is *among* what they assert, leaving the precision of the assertion unmeasured. The navigation loop makes that concrete. Its final act is a language-model call over a fact window and a candidate list. The model is free to name an entity that appeared in neither, with no downstream component to object. The first end-to-end run over the smoke set (Section 4.9) produced both

shapes of that failure. It gave an answer no retrieved triple supported, and a list of nine entities that appeared in no retrieved triple at all. Neither was caught by anything examining the answer.

The design response is to check the draft against the environment *before* it is emitted, rather than to fact-check an emitted answer afterwards. That ordering is where the layer parts company with the post-hoc verification literature. Chain-of-Verification [15] improves a response by drafting it, planning verification questions, answering them, and revising. FActScore [43] decomposes generated text into atomic facts and scores each against a corpus. Both are placed after generation. Both take the model's own answers to the verification questions as evidence. The layer specified here borrows their decomposition step (Section 4.11) and rejects their placement and their evidence source. The check runs before emission and adjudicates against traversed triples rather than against further generation. A model asked to check its own output is a weak detector of its own errors [41]. Three properties follow from placing the check inside the loop. The checker sees the same retrieved evidence the drafter saw. So when the two disagree, the disagreement means something. It is not an artifact of different context. The check happens while budget remains. So a rejected claim can trigger more exploration, not just a retraction. And the judge of whether a fact holds is the environment, not a second opinion from the model that wrote the draft.

What the layer promises needs to be stated precisely, because the terms are easy to inflate. It certifies that the atomic claims a draft asserts are *supported* by the environment in the sense fixed in Section 2.2.2. The traversed triples, or the graph in the retrieved neighbourhood, exhibit a connection consistent with the claim. It does not certify that the answer is correct. An answer every one of whose claims is supported can still be the wrong answer to the question. Section 4.15 shows that this is a structural gap rather than a hypothetical one.

4.11 Claim Decomposition of the Draft Answer

Verification begins with a single model call that produces three things at once. They are a prose draft, the list of entities that draft asserts as the answer, and a decomposition of the draft into atomic claims.

The call sees a fact window of at most sixty traversed triples, together with up to twenty-five accumulated candidate entities. The triples are ranked by embedding similarity between each triple's verbalisation and the *question*, with traversal order preserved among the survivors. Each claim is emitted as a triple of strings $\langle h, r, t \rangle$ over entity *names*, not identifiers, since the model has never seen an identifier.

Four provisions in the prompt are load-bearing. Each exists because its absence produced a specific defect.

Names are to be copied exactly as they appear in the facts. The graph's surface forms are

often not what a language model would write on its own. Freebase renders Yale University as University Yale. Every downstream check is keyed on the name string.

answer_entities names what the draft asserts as the answer, not every entity it mentions. Without this rule the list drifts toward a bag of every name mentioned, including the question's own subject.

A hedged draft has zero claims. This makes abstention explicit, not something inferred from a missing answer.

Answer entities must be specific entities of the kind the question asks for, never a restatement of the question or an entity type. This provision was added after development runs produced answers like "the senators from New Jersey are the United States Senate members associated with New Jersey". That is grammatically an answer and semantically empty. And, as Section 4.15 explains, a claim-level check cannot see it at all. It is a drafting-side mitigation of a failure the checking side cannot address.

The relation slot r is deliberately free text. The model is not asked to name a Freebase predicate, since it has no reliable way to do so. A wrong guess would be indistinguishable from a wrong claim. The consequence is that the structural checks below cannot match on the relation at all. The cost of that choice is analysed in Section 4.12.1 and again in Section 4.15.

Decomposition on the development set was sparse. Across 80 questions the drafter emitted 121 claims. That is a mean of 1.51 and a median of 1 per question, with a maximum of 5. Ten questions produced no claims at all, so for those ten the layer certified nothing. Section 4.15 returns to what that means for the outcome label they received.

4.12 Two-Stage Claim Checking

Each claim is routed through at most three tests. Two are structural and one is model-based. Algorithm 1 states the procedure. Figure 4.2 draws the same routing as a single claim's path through it. The subsections that follow explain each test and the reason for their order.

4.12.1 The Structural Check

The structural check works on entity identifiers. So it first needs a mapping from the names the model produced back to graph nodes. That mapping, μ, is built from the traversed triples alone. Every head and tail name met during navigation is registered against its identifier, and the first occurrence wins. The scope of that mapping is worth spelling out, since it bounds everything that follows. **A claim naming an entity the agent never traversed cannot reach either structural route**, regardless of whether that entity exists in the graph. Such claims fall directly to the entailment check.

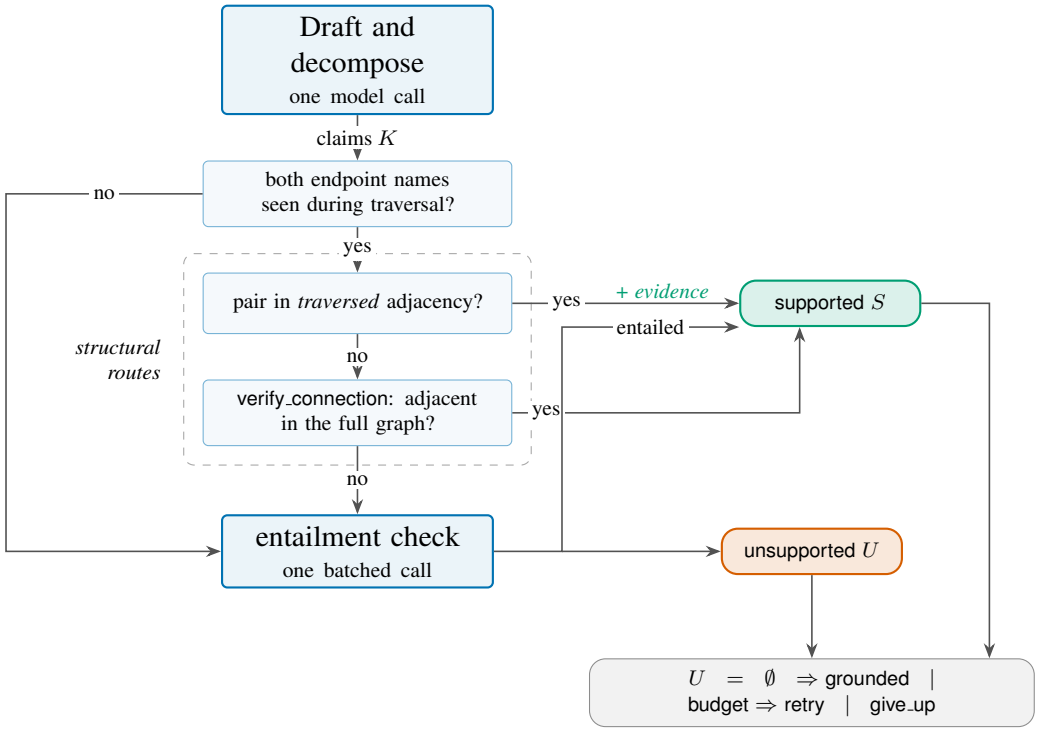


Figure 4.2: The path of a single atomic claim through the layer, mirroring Algorithm 1. Three tests are tried in a fixed order. A claim leaves at the first test that settles it. The two structural routes are boxed. Only the first of them adds to the supporting-triple list. That asymmetry is why Section 4.14 can report evidence for some grounded answers but not for others. A claim that names an entity the agent never traversed skips both structural routes entirely, because the name-to-identifier map is built from traversed triples alone.

Algorithm 1 Structural verification of a draft

Require: question q, traversed triples T, candidates C, tools, budget
Ensure: draft, answer entities, supported and unsupported claim sets, supporting triples

$W \leftarrow \textsc{TopFacts}(q, T, 60)$; $\mu \leftarrow \{\text{name} \mapsto \text{id}\}$ over T {first occurrence of a name wins}
$(\mathit{draft}, A, K) \leftarrow \textsc{DraftAndDecompose}(q, W, C)$ {a single model call}
if $\neg$ verify_claims **then**
  return $(\mathit{draft}, A, \emptyset, \emptyset, \emptyset)$ {ablation only; never chosen at run time}
end if
$\mathit{adj} \leftarrow \{(h,t),(t,h) : (h,r,t) \in T\}$; $S \leftarrow \emptyset$; $P \leftarrow \emptyset$; $E \leftarrow \langle\,\rangle$ {E is an ordered list, not a set}
for all $c = \langle h, r, t\rangle \in K$ **do**
  if $h \notin \mu$ **or** $t \notin \mu$ **then**
    $P \leftarrow P \cup \{c\}$ {an endpoint the agent never traversed}
  else if $(\mu[h], \mu[t]) \in \mathit{adj}$ **then**
    $S \leftarrow S \cup \{c\}$; $E \leftarrow E +\!\!+ \langle\, \theta \in T : \{\theta_h, \theta_t\} = \{\mu[h], \mu[t]\}\,\rangle$ {appended; duplicates are kept}
  else if verify_connection$(\mu[h], \mu[t])$ **then**
    $S \leftarrow S \cup \{c\}$ {accepted, but no evidence is recorded}
  else
    $P \leftarrow P \cup \{c\}$
  end if
end for
$U \leftarrow \emptyset$
if $P \neq \emptyset$ **then**
  $V \leftarrow \textsc{Entail}(W, P)$ {one batched model call; if it fails, every verdict is false}
  partition P into S and U by V
end if
if $U \neq \emptyset$ **and** verify budget remains **then**
  $\mathit{iters} \leftarrow \mathit{iters} + 1$; append repair objective for the first $c \in U$; re-anchor on $\mu[c_h]$
end if
route: $U = \emptyset \Rightarrow$ grounded; else verify, depth, and time budget all remain $\Rightarrow$ retry (**goto** explorer); else give_up
return $(\mathit{draft}, A, S, U, E)$

For claims whose endpoints both resolve, the first route is *traversed adjacency*. The traversed triples are collapsed into an undirected set of endpoint pairs. A claim is supported if its endpoint pair is in that set. Traversed adjacency is exact and costs nothing beyond a set lookup. Uniquely among the three, it also produces citable evidence. Every traversed triple joining the pair is appended to the supporting-triple list. They are appended, not unioned, and Section 4.14 records what that costs.

The second route is the verify_connection tool (Section 4.2.3), a single Cypher query asking whether the two entities are adjacent in the *full* graph, either directly or through one mediator node. It exists because the first route tests only the *traversal*, not the graph. The agent may have

reached both endpoints along different branches without ever walking the edge between them.

Both routes are relation-agnostic. r is free text with no reliable mapping onto a Freebase predicate. So neither route asks whether the edge it finds is the edge the claim names, and neither asks about direction. That is the price of allowing free-text relations. It is the layer's main acceptance risk, not an incidental looseness.

Appendix E.22 records how often each route fires, and two figures from it are quoted later. The second route is rare. It was consulted five times on the development set and on a small minority of test claims. So the first route does almost all of the checking. And the record cannot say how many of the $2{,}008$ accepted claims were certified by a relation-blind test rather than by entailment. The answer lies somewhere in $[39, 2{,}008]$. That interval is what Section 4.15 and Section 5.18 both read against. Section 6.3.2 is the change that would narrow it.

4.12.2 The Entailment Check

Claims that fail both structural routes collect in a pending set, together with those whose endpoints were never traversed. They are checked in one batched model call. The claims are numbered, rendered as h – r – t, and shown against a fact block. The model is asked for a boolean verdict on each. The prompt spells out what unsupported means, so that it covers the three cases a charitable reader would otherwise let through. These are claims that are plausible but absent, contradicted, or inferable only through outside knowledge.

Three properties of this check need stating, since each one bounds what its verdicts mean.

The fact block is the same window the drafter saw. It is the same sixty triples, ranked by relevance to the *question* rather than to the claim under test. So a claim about an entity peripheral to the question may be judged against facts that never mention it. The bias this introduces has a direction. It works against peripheral claims and in favour of claims about material the drafter had already seen.

Failure is conservative and lossy. If the call raises for any reason, such as budget exhaustion, a transport error, or malformed output, every pending claim is marked unsupported. That is the right default for a system whose purpose is to avoid unsupported assertions. But it means an infrastructure failure and a genuine rejection leave the same record, except in the trace.

The check can only add support, never remove it. Claims accepted structurally never reach it. A claim whose endpoints are adjacent for a reason unrelated to the asserted relation is accepted with no semantic examination at all.

4.12.3 Why the Structural Check Runs First

There are three reasons, in descending order of importance. The first is *authority*. The thesis's position, fixed in Appendix E.17, is that embeddings and language models rank while symbols

decide. So the model is consulted only where the graph is silent. It fills gaps but never overrides a finding of the environment. The second is *evidence*. Only traversed adjacency yields triples that can be attached to the answer. So the order does not change which claims are accepted. It does change how many of them can be justified. The third and least important is *cost*. A set lookup is free, a verify_connection probe is one indexed query, and the entailment call is the layer's only unavoidable expense beyond drafting. Running cheap before expensive means the expensive test sees only what is left over, not every claim.

4.13 Repair Policies for Unsupported Claims

The router after the verifier is pure Python with three outcomes. No unsupported claims routes to grounded. Unsupported claims with budget remaining route to retry, and anything else routes to give_up.

4.13.1 Targeted Re-Exploration Under Remaining Budget

On retry, the verifier has already re-targeted the agent before the router runs. It takes the *first* unsupported claim and builds a repair objective of the form verify whether $h\ r\ t$. It marks every active plan step done and appends the repair step as the new active objective. If the claim's head name resolves, it also resets the frontier to that single entity. Control then returns to the explorer, which now searches specifically for the missing evidence.

Two limitations follow. Only one claim is repaired per iteration. The one chosen is the first by position in the draft's decomposition, not the most important to the answer. Both choices keep the repair path cheap. Neither was revisited, since the path was rarely exercised.

The retry route needs verify, depth, *and* time budget all at once. That conjunction is why it almost never fires. By the time the verifier is reached, the depth budget is typically already spent. Appendix E.26 measures it. It gives the development-set condition under which the route could have run, why that correlation does not survive the move to test, and the instrumentation consequence that follows for reading the repair counts.

4.13.2 Answer Rewriting and Hedging

On give_up the answerer makes one rewrite call. It receives the draft and the unsupported claims. Crucially, it also receives the list of entities already decided to survive, with an instruction to keep them named exactly as written. It is asked to remove or hedge the unsupported material and to respond with prose only. That last phrase is the whole of the fix described in Section 4.13.4. In the original design this call also regenerated answer_entities. That regeneration turned out to be the single largest defect in the layer.

4.13.3 Iteration Cap and the Draft-Only Fallback

Repair is capped at two iterations. That cap, together with the depth and time conditions, guarantees the verify–explore cycle terminates. On the development set the cap was never reached, for the reason just given. The cycle never started. On test it binds on 15 questions, 4 on WebQSP and 11 on CWQ. That is the cap doing the job it was put there for, not sitting unused. It is the second place where the development set understated this route.

The grounded path costs nothing. When no claim is unsupported the answerer emits the draft verbatim with no further model call. That is the path 77 of 80 development questions took. The layer's aggregate cost is correspondingly small. Mean tokens per question rise from 4,858 without claim checking to 4,925 with it, about 67 tokens or 1.4%, and mean model calls rise from 7.2 to 7.3. Whatever the case for or against the layer, it is not a cost argument.

A distinct path exists when verify_claims is false. The verifier then returns the draft and its entity list untouched, with outcome draft_only. That outcome is the ablation condition of Section 5.6, not a runtime fallback. The system never selects it dynamically. A third path, the legacy answerer of Section 4.6, runs only when the verifier raised before producing a draft at all.

4.13.4 Entity Filtering of the Final Answer

The rule is one line, but its justification is the most instructive result in this chapter.

The rule. On the give-up path the final entity list is computed deterministically from the draft's own list. An entity is kept if it appears in at least one supported claim, or if it appears in no unsupported claim. Strings are copied exactly as they are. The function can only remove entities and never adds one. The second branch of the rule is permissive on purpose. An entity the decomposer never mentioned in any claim is kept. The principle is that the layer should punish unsupported assertions, not imperfect decomposition recall.

The scope. This filter applies to the give-up path only. On the grounded path the draft's entity list passes through unchanged, since there are no unsupported claims to filter against. On the development set the filter therefore ran on 3 of 80 questions.

The Attribution Census

The filter exists because of a paired comparison between the verified condition and its verify_claims = false twin at the frozen α. In aggregate, claim checking *cost* accuracy, with 64 hits against draft-only's 66. But aggregate counts cannot tell a mechanism working as intended from a mechanism misfiring. Working as intended here means turning an unsupported lucky hit into an honest hedge. So the two runs were diffed question by question. Exactly three questions differed, and Table E.2 gives all three.

The classification is clear-cut. *No entailment verdict was involved in any of the three*. Appendix E.15 goes through them one by one. The finding fits in one sentence. It is the conclusion this whole subsection exists to support:

> The verification logic was sound; its repair path frequently was not.

Replacing generation with deterministic filtering removed two of the three failures. It left the third, which is a different defect.

What the Census Does and Does Not Establish

Three qualifications belong here rather than in a limitations chapter. Without them, the numbers above would be read as more than they are.

The census is $n = 3$. It is a complete enumeration of the questions on which the two conditions differed. That is exactly why it could be classified by inspection. But three questions support a mechanism claim, not a rate.

Claim checking did not reduce wrong answers on this set. Post-fix verified scores 65 / 11 / 4 against draft-only's 66 / 10 / 4. The wrong count is identical, and the layer cost one correct answer and removed none. Its case therefore cannot be made on development-set accuracy. This thesis does not attempt to make it there. Neither of the two aggregate measurements that might seem to carry it survives Section 5.7 either. The groundedness result of Section 5.11 is reached by the agentic baseline as well. That result makes it a property of graph navigation rather than of this layer. Removing the layer, in turn, does not move per-question precision. So the precision margin over that baseline is not evidence for it either (Section 5.17.1). What remains is the output contract of Section 4.14, which is an auditability claim rather than an accuracy or a precision one. Beside it stands the case-level evidence of Section 5.17.3, where the layer is seen removing a false assertion on the questions its filter reaches. That is a narrower case than this section originally made. It is the one the measurements support.

The residual difference is a single question. Its interpretation involves a label changed after the fact. The sole remaining diff is Harbaugh, where draft-only's "hit" asserts Baltimore Ravens alongside an unverifiable founding-date filter. No traversed triple supports the Ravens, and they are not a team Jim Harbaugh played for. Set-intersection Hits@1 scores that as a clean hit, but per-question F1 in Section 5.7 does not. The question was additionally relabelled from cvt_heavy to unanswerable_env after this analysis, on the grounds that it carries a date constraint the environment cannot express (Section 3.3.3). The relabelling is defensible on its own terms and was recorded as a decision. But it was made after seeing the result it affects, and is flagged as such here and in Section 5.18. The aggregate figures reported above are not rescored on account of it.

4.14 Output Contract: Answer Paired with Supporting Triples

The system pairs each answer with the triples that support it. That pairing is the explainability deliverable named in Section 1.6. Its actual coverage is narrower than the phrase suggests in two independent ways.

Only one of the three routes records evidence. Supporting triples are collected in the traversed-adjacency branch alone. A claim certified by verify_connection is accepted with nothing attached, and so is a claim certified by entailment. The contract is therefore "answer paired with whatever traversed triples happened to justify it", not "answer paired with justification for every claim".

What survives in the log is the count, not the evidence. The pairing holds inside the run. The answerer receives the verifier's triple list and emits against it. But RunLogger writes n_supporting_triples, an integer, and drops the list. No committed artifact in this work contains a single supporting triple. Every statistic in this section is computed from that counter. An examiner can confirm that these answers came from a system which tracked its evidence. They cannot inspect the evidence, which puts a limit on the auditability this section claims. It is the same gap that Section 6.3.2 approaches from the other side. One logging change would repair both. It is recorded rather than repaired this late, under the standing rule of Section 4.9. Repairing it now would separate the code from the results already frozen against it.

On the development set, the measured consequence is that 13 of 80 answers carry no supporting triples at all. Ten of those asserted no claims. They are hedges, for which zero is the honest number. Of the remaining three, one had every claim rejected and two had their single claim certified by verify_connection, which records no evidence. Across all 80 records the median is 3 and the mean 4.1, with a maximum of 16, read from the persisted counter since the triples themselves are exactly what the log does not keep.

Three instrumentation defects in this area are recorded rather than quietly fixed, in keeping with the same standing rule. Appendix E.10 gives them. The first is a falsy-empty-list trap in the answerer, since repaired. The second is an unrepaired counter whose name misdescribes what it counts, and which is therefore fenced off from every reported result. The third is a mismatch between Algorithm 1 and the code, the algorithm accumulating E by set union where the implementation appends. The third bears on the figures above. Two claims sharing an endpoint pair each append the same matching triples. So n_supporting_triples counts matches rather than distinct triples. The exposure is bounded at 378 of the 908 verifier firings, itself an over-count. That bound makes that counter an *upper* bound on distinct supporting triples. The median of 3 and the mean of 4.1 above are upper bounds too.

4.14.1 A Worked Example

Appendix E.20 walks one question end to end through drafting, claim decomposition, both structural routes, and the entity filter. The question is *which university, founded in 1701, did actor James Franco attend?* Its instructive part is the counterfactual. Before deterministic filtering replaced a second generation call, the same trajectory emitted an entity the traversal had never supported. Replacing generation with filtering is what removed it.

4.15 What the Verification Layer Establishes, and What It Does Not

The ways this layer can be wrong follow from its design, independently of how often they occur. Appendix E.8 enumerates them. There are five mechanisms of wrongful rejection, led by the composite claim whose unverifiable component drags down its verifiable one. There are four of wrongful acceptance, led by relation-agnostic adjacency. Section 5.15.3 reports which of them actually fired.

One of those mechanisms bounds the layer's principal risk and belongs here rather than behind the body. Both structural routes ignore the relation and ignore direction. So any edge between the endpoints certifies any claimed relation between them. A claim that X is Y's mother is accepted on an edge recording that X is Y's child. The population exposed is therefore not the rare second route but every claim the two of them accept between them. That population is, on test, somewhere in $[39, 2{,}008]$ of the $2{,}008$ accepted claims. The lower end is what the log pins down, and the upper end is what the mechanism permits. Nothing measured in this thesis narrows that interval. Section 6.3.2 is the change that would.

Stated at the level of precision that catalogue requires, the layer establishes that the entity pairs a draft's claims relate are connected in the environment. On the repair path, it also makes the emitted entity list a deterministic function of those verdicts rather than a second generation. It does not establish that the claimed relation is the one the edge records, that an entity absent from every claim was grounded at all, that the answer is responsive to the question, or that a claim it could not check is false. Those boundaries are properties of the design rather than defects discovered in it. Section 5.14 reports how much of the residual error they account for.

Chapter 5

Evaluation: Setup, Results, and Error Analysis

The evaluation is reported in three parts. They are the experimental design fixed before any test question was answered (Section 5.1), the measurements it produced (Section 5.7), and a hand reading of every failure that remained (Section 5.14).

5.1 Experimental Setup

This part of the evaluation fixes everything that had to be decided before any test question was answered. It settles which questions, which systems, which numbers, and which rules for reading them. It records the decisions that went the other way as well as the ones that stood, each where it belongs rather than collected into a confessional at the end.

The aggregate figures quoted in this chapter are regenerated by scripts/build_thesis_numbers.py into results/phase4/thesis_numbers.json from the scoring, groundedness, judge, and census artifacts. Anything read instead from a run record or a census sheet is cited to it there. Two *reported* figures cannot be recomputed from this repository and say so where they appear. One is the API spend of Section 5.2, a billing record. The other is the second through fourth determinism rounds of Section 5.3, read from console tables that were never committed. Most tables are hand copies checked against that JSON rather than emitted from it. That is the weaker guarantee.

5.1.1 Mapping Experiments to Research Questions

Each research question of Section 1.5 is answered by a distinct experiment rather than by one table read three ways. **RQ1** is answered by the main matrix. It runs five systems on two datasets, scored with Hits@1 and F1, broken down by hop stratum, with paired significance tests (Sections 5.7.2, 5.7.3 and 5.10). The claim about *multi-hop* accuracy rests on the per-stratum

breakdown, not on the aggregate. **RQ2** is answered by three measurements that disagree with one another. That disagreement is itself the finding. The three are the two-tier groundedness metric (Section 5.11), the precision and F1 columns of the main matrix, and the verifier ablation (Section 5.13). **RQ3** is answered by four single-deviation ablations on stratified half-samples, each compared against the unmodified system restricted to the same questions (Sections 5.6 and 5.13).

5.1.2 Pre-Specification and the No-Tuning Policy

One rule governed everything from the moment the test files were built. *Nothing gets tuned.* No prompt was edited, no configuration value changed, and no threshold adjusted after test data was touched. Defects discovered mid-matrix were documented rather than fixed. Four decisions were fixed in writing before the measurements they govern. The first is the frozen configuration $\alpha = 0.7$, $\tau = 0.20$ with claim verification enabled, tuned on the development set (Section 5.6.1) and never revisited. The second is the sample size of 400 questions per dataset, named as the fallback before the API budget was known (Section 5.2). The third is the baseline certification threshold, under which a retrieval baseline may hedge with the gold answer visible in its own retrieved facts on fewer than 15% of its hedges (Section 5.4). The fourth is the judge validation threshold of Cohen's $\kappa \geq 0.7$ against blind human annotation (Section 5.5.5). The last was missed, by 0.0005, and is reported as missed. A threshold that is only pre-specified when it is met is not pre-specified at all. Nothing was filed with a public registry, so the claim throughout is pre-specification rather than pre-registration.

Phase 4 required a single code change, a use_planner flag to disable decomposition. It was certified as a no-op on the frozen path by re-running an already-answered development question and confirming that every language-model call was a cache hit. The reference condition in the ablation table is therefore provably the same system that produced the main matrix.

Path fidelity was the third criterion named in the approved proposal. It measures agreement between the relation chain the system traversed and the chain the benchmark's gold SPARQL query specifies. It is not reported, because it requires the gold relation chains. The RoG distribution (Section 3.1.3) publishes name-keyed triples without the originating SPARQL. So the chains would have to be reconstructed from the upstream releases and re-aligned to this environment's relation vocabulary. That was outside the project's remaining budget, so Section 6.3.5 treats it as future work.

5.2 Evaluation Datasets and Test Sets

5.2.1 WebQSP and ComplexWebQuestions

Two benchmarks are used, in the RoG distribution [9] whose per-question subgraphs also form the knowledge environment (Section 3.1.3). **WebQSP** [19] is the semantic parses of WebQuestions. Its natural questions come from search logs and are mostly one or two hops, with multi-valued answers common. **ComplexWebQuestions** (CWQ) [20] is built by composing WebQSP questions with additional constraints through SPARQL templates, giving deeper chains, conjunctions, superlatives, and comparisons. As Section 5.2.4 documents, it also gives a higher rate of machine-generated label defects.

The distinction that matters operationally is visible in the gold answer sets. The metrics count distinct answer strings, since Section 2.1.4 defines precision and recall over sets. Counted that way, WebQSP averages 7.15 gold entities per question, but its median is 1.5. Exactly 200 of its 400 questions carry a single gold answer. The mean is pulled up by a long tail whose largest question lists 237. CWQ averages 1.75 with a median of 1.[1] So the two benchmarks differ less in the typical question than the means suggest, and more in the tail. A metric that rewards one correct entity behaves very differently on a question with hundreds of gold answers than on one with a single answer. That tail is one reason this thesis reports F1 alongside Hits@1 (Section 5.5).

5.2.2 Stratified Sampling, Seeds, and Published Question IDs

The evaluation uses only the *test* splits. The development set of Section 5.6.1 came from the train and validation splits, and the test-set builder asserts that the two pools do not overlap.

Sample size was a budget decision, as the full test splits hold 1,628 and 3,531 questions. At the measured token rates, running five systems over both projected to roughly $45–55, against a stated ceiling of $15–20. So the evaluation fell back to the pre-specified alternative, a stratified sample of 400 questions per dataset (Table 5.1). At that size the bootstrap confidence intervals on Hits@1 are roughly ± 5 points. The arithmetic was written down before any run fired.

In the end the whole benchmark, ablations included, cost $11.13 in API spend. That figure comes from the provider's billing record, not from the run artifacts. The reproducible proxy is the token and call accounting of Section 5.12, which every run record carries per question.

Sampling is proportional within hop strata, at seed 42. The resulting question sets are committed as immutable artifacts (results/phase4/test_webqsp.json, results/phase4/test_cwq.json), and

[1] The distinction matters. The raw answer lists repeat a string on 4 WebQSP and 6 CWQ questions, all of them in the tail. Counting the repeats takes the WebQSP mean to 7.98 and its largest question to 382. But the scorer deduplicates before matching, so nothing scored in this thesis ever sees them. Quoting the raw counts would describe a population that no reported number uses.

Table 5.1: The two certified test samples. The unreachable stratum is kept on purpose. The ceiling is the share of the 400 questions with at least one gold answer reachable within four hops. No result in Section 5.7 can score above it on Hits@1.

Dataset	n	h1	h2	h3plus	unreach.	Ceiling
WebQSP	400	256	128	4	12	97.0%
CWQ	400	137	211	49	3	99.2%

Appendix C lists every question identifier. Once the first system ran, the files never changed again.

A near miss worth recording. The first build read the training shards rather than the test shards. The development-overlap guard caught it. The contamination guard, built for hygiene, is what found the contamination. No system had run against those files. They were deleted and rebuilt, and the rebuilt pools report 1,628 and 3,531 with skipped={}. Appendix E.25 records the diagnosis. The cheap lesson is that an overlap assertion belongs in every benchmark builder.

5.2.3 Hop-Count Stratification and the Accuracy Ceiling

The stratification variable is the length of the shortest path, in *this* environment, from any topic entity of the question to any gold answer entity, capped at four raw hops. The same machinery certified the development set. So mediator nodes count the same way throughout, as the two edges they are (Section 4.2.3). Each question falls into one of four strata, h1, h2, h3plus, and unreachable.

Two facts in Table 5.1 come back again and again in the results. WebQSP is 64% one-hop, and its h3plus tail holds four questions. Four is too few to interpret, and the tail is labelled as such wherever it appears. CWQ is mostly two-hop, with a real deep tail of 49 questions. Because the samples keep the stratum proportions of their parent pools, per-stratum numbers describe the benchmark and not just the draw.

The ceilings in the table differ from the ones the validation gate reported. The difference is explained here rather than left for a reader to reconcile. Table 3.3 measured the full test splits and found 97.3% and 99.7% of questions reachable. The table above measures the same quantity over the 400 questions actually evaluated and finds 97.0% and 99.2%. Neither figure corrects the other. The first says what the environment could support across the whole benchmark. The second says what any system in Section 5.7 could have scored, and every accuracy in this thesis is read against it. The gap between the two is sampling. It stays under a point on both datasets because the draw kept the stratum proportions (Appendix E.24).

The unreachable stratum stays in. Dropping it would raise every system's score by the same amount and misdescribe the environment. Keeping it preserves an honest ceiling. It also turns

Table 5.2: Gold-label adjudication. Rows and distinct questions are separate units because the pass emits one row per consensus answer. Exclusions are the union of gold_wrong and ambiguous_question questions.

Dataset	Rows	Questions	gold_wrong	ambiguous	Excluded
WebQSP	89	58	12	10	22 (5.5%)
CWQ	59	47	17	2	19 (4.8%)

these questions into the sharpest available probe of parametric leakage, as Section 5.11 shows. Any "hit" on a question with no graph path to gold is an assertion the environment cannot support.

5.2.4 Gold-Answer Quality Control

Both benchmarks carry label defects, CWQ more. Its questions and answers are generated by SPARQL templates over the knowledge base rather than written by hand. A question whose gold answer is wrong is not a system failure. Counting it as one corrupts both the accuracy tables and the error taxonomy. Some detection procedure was therefore necessary. A naive one, though, is worse than none, as the results below show.

The detection principle is **independent multi-system consensus against gold**. When at least three of the five systems of the main matrix assert the same non-gold answer, the label is worth examining. The voters are those five systems, spanning two knowledge sources and four retrieval paradigms, rather than the four ablation conditions, whose agreement would describe an architecture rather than a label. Appendix D.4 gives the procedure, the three verdicts, the automatic triage that cleared 15 of 89 rows on WebQSP and 2 of 59 on CWQ, and the two *opposite* evidence signatures that make the adjudication reproducible. A world-fact error requires mixed consensus, while an annotation-pipeline error shows graph-only consensus. One row is emitted per (question, consensus answer) pair, so rows and questions are separate units throughout and question counts are always deduplicated.

Census-Based Exclusions and the Dual-Reporting Policy Two numbers must not be conflated, and Table 5.2 separates them. The consensus pass *flagged* 58 and 47 questions, while 22 and 19 were *adjudicated* as defective. The gap is not noise. Most flagged questions are all five systems converging on the same wrong answer. That convergence is a finding about the systems rather than the labels, and it is treated as one in Section 5.16.2. A policy of rescoring on consensus alone would have corrupted the results with exactly those cases. The excluded questions are removed from the failure census of Section 5.14 as benchmark artifacts rather than system failures, ambiguous_question on the same logic, an underdetermined question being no more a system failure than a broken label. Of those excluded, 12 of 22 (WebQSP) and 15 of 19

(CWQ) were failures of the full system. Those are the ones that shrink the census pool.

The accuracy tables are not rescored. They are reported exactly as measured, with this section as the footnote, for two reasons. Rescoring on self-adjudicated labels would substitute one contestable ground truth for another. And the errors cut both ways. The remaining 10 and 4 excluded questions are ones the full system scored as *hits* against labels now judged broken or ambiguous. That symmetry is what makes the footnote credible rather than defensive. It also bounds the net bias. Recomputing every cell of Table 5.4 with the excluded questions removed moves each system by between $+0.001$ and $+0.020$ Hits@1 and changes no ranking on either dataset, an order of magnitude below the defect rates, because removing a question a system got right subtracts from the numerator as well as the denominator.

Two scope limits belong on the record. The rule that assigned gold_wrong automatically to questions with implausibly long gold lists is a heuristic, and its outputs were spot-checked rather than verified one by one. And the manual adjudication covered every mixed-evidence flag and every graph-only flag that was not a topic echo. That is broad coverage, but it is not the complete flag population.

5.3 Backbone Model Selection and Qualification

Every system runs on the same frozen backbone, gpt-5.4-mini-2026-03-17, at temperature 0.0, with reasoning_effort set explicitly to "none" and the response cache enabled. Every one of the $4{,}000$ test-matrix records and every ablation record carries the model string, both sampling parameters, and the cache mode. So any record found anywhere states what produced it. A non-reasoning model was chosen so that sampling parameters remain available and no hidden reasoning tokens contaminate the cost metric. The second was verified rather than assumed, by a check that logs the reasoning-token count on every call and read zero across every run in this thesis.

Selection between gpt-4.1-mini-2025-04-14 and gpt-5.4-mini-2026-03-17 was settled by four rounds of a determinism probe. The probe counts how often a *decision signature* repeats exactly across reruns of the same questions. A decision signature is a hash of the plan, the expanded relations, the evaluator verdicts, and the answer. The rounds disagreed with one another, because temperature-zero decoding on a hosted API is greedy rather than deterministic and a sequential agent amplifies a single divergent token. Appendix E.9 gives the four rounds, that mechanism, and the tiebreaker pre-specified before rounds three and four. That tiebreaker decided the choice on longevity grounds.

The stability figure that belongs to this thesis is the frozen model's, which is the lower of the two. gpt-5.4-mini repeated its decision signature on 8 of 12 probes, or $\approx 67\%$. Roughly one trajectory in three diverged on a rerun. The $9/12$, or $\approx 75\%$, belongs to the runner-up. At

$n = 12$ neither rate is measured tightly enough to separate from the other, and the thesis claims determinism at neither. Only the first round survives as per-question artifacts. The qualification script opens its detail file in truncating mode, so each later round overwrote the one before.

Two observations from those four runs matter more than the choice they settled. Each candidate carried a *persistent* ungrounded answer. The 4.1-mini model asserted "United States Dollar" for a Dominican-peso question in two of the four runs, and 5.4-mini named the wrong Beckham child in all four. Each also carried *intermittent* ones. The same question, on the same graph, under the same budget, moved between grounded-correct and ungrounded-wrong as the sampling weather changed. Choosing a backbone does not remove this class of error but only moves it around. That is the premise of this thesis, observed over four runs and two frontier-family models, and it is the reason Section 4.10 exists.

So reproducibility rests on the cache, not on the model. Section 4.8 specifies the mechanism, and Appendix E.3 gives the identity check it supports. A cache hit is indistinguishable from a live call. A fully cached rerun therefore reproduces the original run's budget snapshot exactly, not approximately. One defect qualifies that. The replay restores prompt and completion tokens but not the reasoning_tokens counter. The defect does not bite here, because that field is zero in every one of the 6,960 agent runs committed for this work. It is recorded rather than repaired, for the reason Section 5.1.2 gives.

Every table in Section 5.7 comes from one recorded run per condition. That run replays byte for byte for as long as the cache survives. Fresh-run variance is a different problem. This thesis does *not* measure it, the bootstrap intervals cannot bound it, and Section 5.18 carries it as a limitation. Latency, finally, is computed over cold-cache records only. That restriction keeps the latency column honest where a run was interrupted and restarted.

5.4 Baseline Systems

Four baselines span the space between no retrieval and agentic navigation. All four run against the same tools, the same graph, and the same backbone.

5.4.1 The Two Non-Graph Baselines

The **no-retrieval LLM** makes a single call. It receives the question, no facts, and an instruction to return an empty entity list when it does not know the answer. WebQSP and CWQ predate current models and may well sit in their training data. So this system measures the floor that every retrieval result has to be read against. Its entity strings come from parametric memory rather than from the graph, which means its surface forms mismatch gold more often than a graph-grounded system's do. Its score is therefore a *lower bound* on parametric knowledge.

The asymmetry is reported and not corrected, because grading one system leniently and another strictly would corrupt the comparison.

Vector-RAG turns every triple into a sentence, embeds it with MiniLM [29], and indexes it. The question retrieves the top 30 sentences by exact inner product, and one call answers from them. That is dense retrieval [28] applied to a graph. Its limitation is structural rather than incidental. A fact stored through a mediator, such as a marriage or a film role, has no single triple that expresses it. So no single retrieved sentence can express it either. The index drops triples with a mediator endpoint at build time, because a verbalised half-edge means nothing and leaks raw machine identifiers into answers. Filtering that noise out does not repair the paradigm, and both facts are reported.

5.4.2 Static GraphRAG

This is the structure-without-navigation comparison [4]. The baseline resolves the question's topic entities and expands a *one-logical-hop* neighbourhood under the same relation blocklist and mediator pass-through contract that every other graph-touching system uses. It then reranks the result by embedding similarity to the question and answers from the top 30 facts in one call. It shares Vector-RAG's answering prompt and cutoff exactly, so the difference between the two isolates retrieval strategy and nothing else.

The radius deserves a precise statement. An earlier draft and the implementation's own docstring both called it two hops, but it is one. Expansion runs once from each topic entity. When it meets a mediator node it hops through and concatenates the two relations, so a mediator path costs two graph edges but stays one logical hop. An ordinary neighbour is emitted and never expanded again. The run records settle the point, since every logged fact reads topic relation endpoint. A genuine second hop would be anchored at an intermediate entity. All 7,431 logged facts begin with a topic entity.

The radius is not the only thing that bounds this baseline. The fanout is, and it binds harder. Expansion retrieves at most 100 edges per topic entity, under a Cypher query with no ORDER BY. So on a higher-degree entity the 100 retrieved are an arbitrary sample of the neighbourhood rather than its best or its first. Measured over the 711 distinct entities the 800 test questions resolve to, 55.1% carry more edges than the cap retains. At least one topic entity is truncated on 72.5% of questions (Appendix E.14). So for a hub topic the baseline answers from a small fraction of the neighbourhood, chosen without a stated criterion. The hub-noise diagnosis of Section 5.10 has to allow that the answer may never have entered the sample the reranker saw. The full system's own get_neighbors shares the property (Appendix B.3) but not the exposure. Its cap is 200 and applies per relation rather than per entity, binding on 3.3% of its 16,301 neighbour calls.

Two implementation details fixed on development data were comparability repairs rather than

improvements. Neighbourhood expansion initially passed *through* ordinary entities and stopped at mediator nodes, exactly inverted for Freebase and the reason machine identifiers were reaching answers. And retrieved facts are deduplicated by endpoint pair before reranking. Cosine similarity favours redundant phrasings of one connection and was crowding the informative row out of the top 30. The rule keeps the shortest rendering. That choice resolves systematically towards the direct edge over the mediator-expanded path whenever both connect the same pair. That bias runs against this baseline on exactly the questions mediator expansion was added to serve. So it is disclosed rather than left in the code.

What this baseline licenses, and what it does not. A one-hop retriever cannot contain a two-hop chain. So its decay across hop strata (Section 5.10) is partly a property of the radius and not only of the paradigm. That distinction has to be drawn. The alternative is to read an implementation limit as a result. The evidence that it is a radius effect is visible in the strata. This baseline scores 0.44 on WebQSP's h2, where two-hop questions are largely mediator paths a one-hop expansion *does* reach, against 0.16 on CWQ's h2, where the chains are genuine compositions it cannot reach at all. A retriever failing uniformly on depth would not show that gap. The claim this thesis therefore makes on the strength of this baseline is the narrow one. *Retrieval fixed before reasoning must commit to a radius in advance, and any fixed radius is wrong for some question in the set.* It does *not* claim that a two-hop static retriever would score as this one does. That experiment was not run. The honest expectation is that it would recover much of the CWQ h2 gap while losing precision to a larger and noisier window. Re-running it is the first item in Section 6.3.5. Where the per-stratum argument needs a static comparator that is definitely not radius-limited, it uses Vector-RAG, whose one-triple context cannot represent a chain at any radius.

5.4.3 Think-on-Graph (Agentic Baseline)

The credibility-critical comparison. Think-on-Graph [7] is reimplemented on this environment's tools. It uses beam width 3, depth 3, language-model relation pruning, language-model entity pruning, and a sufficiency check at each depth. It is a reimplementation rather than the authors' code, running on this graph with this backbone. That makes it a *stronger* comparison than citing published numbers. Graph, model, budget, and scoring rule are held constant, and the algorithm is what varies. It correspondingly means its numbers are not comparable to the published ones and are not presented as such.

Holding the budget constant is not the same as making it irrelevant. A fixed cap bites harder on a method whose cost multiplies with every step. So the comparison measures search quality and affordability together. Section 5.8 separates them by splitting the results on whether the cap actually truncated the baseline. The answer changes what the aggregate margin should be taken to mean. Under the same 25-call cap the full system operates under, Think-on-Graph is clipped

on 117/400 WebQSP questions (29.2%) and 176/400 CWQ questions (44.0%). A clipped run still answers, from whatever it gathered, by the same graceful degradation the full system uses. One asymmetry there favours the baseline and is stated so the clip rates are not mistaken for a count of runs that returned nothing. The baseline stops its search one call *before* the shared limit, reserving that call for the answer it then has to compose, where AGR makes no such reservation.

A second asymmetry is not in the baseline's favour, and it bears on how Section 5.8 should be read. Both systems call the same tools but truncate the returned candidate sets at different widths. Think-on-Graph keeps the first 40 relations per entity and 20 neighbours per relation, the pruning widths of the reimplemented algorithm, against AGR's 300 and 200 (Table 4.3). Since get_relations returns rows by descending fanout, a binding cut discards the low-fanout tail. That is the material Section 4.2 reports raising AGR's own cap from 80 to 300 to stop discarding. It binds on 31.6% of the entities Think-on-Graph expanded against 1 of AGR's 3,097 (Appendix E.19). The consequence is a bound on one specific claim. Section 5.8 concludes that the margin over Think-on-Graph is affordability rather than search quality. That conclusion is safe in the direction it is stated, since a narrower candidate set cannot explain why the baseline runs *out of calls*. But the residual gap on unclipped questions is measured against a baseline searching a thinner pool. So it is a lower bound on that baseline's attainable quality rather than an estimate of it. Equalising the widths is the first item in Section 6.3.5.

5.4.4 Variables Held Constant, and Certification

Every system shares the backbone and its sampling parameters, the knowledge environment and its relation blocklist, the entity resolver, the question files, the budget meter and its 25-call cap, the output schema so that scoring reads one field for all systems, and the record schema so that any record self-describes with backbone, budget hash, run configuration, and source commit. Both graph-navigating systems seed from the dataset's annotated topic entities. That choice removes entity linking as a confound, but Section 5.18 lists it as a limitation.

Two things are *not* held constant, and both are named here rather than left buried. The first, which is deliberate, is that the baselines get a plain answering prompt. The full system's drafting prompt carries the anti-vacuity and grounding provisions of Section 4.11, and those provisions belong to the system under test, not to the shared harness. The second is a property of the algorithms and is where a reader should look first for a confound. It is the width of the candidate set each system considers per step. Section 5.4.3 gives the measured binding rates and the bound they place on Section 5.8, and Section 5.4.2 gives the static baseline's own fanout cap.

Baselines were debugged exclusively on development data and certified before any test question ran. The pre-specified diagnostic targets the one failure mode that would be an implementation defect rather than a paradigm limit. In that mode, *retrieval found the gold answer and the system hedged anyway*. That failure mode is checkable, because both retrieval baselines log a sample of

what they retrieved. The rule was written before the numbers. Under 15% of hedges certifies the baseline as-is, and over 30% means an over-abstinent prompt, to be adjusted identically for both baselines with both files re-run. Measured, Vector-RAG scored 0% on both datasets and Static GraphRAG 10% and 14%. Both certified, with two caveats recorded rather than smoothed over. The diagnostic samples the top ten of thirty retrieved facts, so it is a lower bound. And Think-on-Graph's 0% is *vacuous* rather than clean, since it logs no retrieved-fact sample and the diagnostic compared against an empty string. As a multi-step navigator it was not the diagnostic's target. But the reading is a null, not a pass. All ten test files completed at exactly 400 records with zero failures and zero duplicate question identifiers.

5.5 Metrics and the Groundedness Judge

5.5.1 Answer Accuracy: Hits@1 and F1

Both metrics are computed over *entity sets* after Unicode NFKC normalisation, case folding, and whitespace stripping. There is no fuzzy matching and no substring containment. *Hits@1* is 1 if any predicted entity matches any gold entity. That is the convention in this literature, and it is what makes published numbers comparable at all. It does not fix how strings are matched, and RoG's scorer is looser (Section 5.9). *F1* is computed per question between the predicted and gold sets as defined in Section 2.1.4, and averaged over questions.

The two metrics are not redundant. Hits@1 rewards a system that names one correct entity among several wrong ones. On WebQSP that loophole is narrow on the typical question and wide in the tail. The median question has 1.5 gold answers, but the mean is 7.15 and the largest question lists 237. The precision component of F1 closes it. Section 5.7.2 shows how far apart the two metrics can sit for a single system. That gap is a property of the answer sets the system emits. It does not measure any component inside it. Removing the verification layer moves F1 by $+0.004$ on WebQSP and -0.009 on CWQ, and neither figure is distinguishable from zero (Section 5.13).

Strict matching raises an obvious worry, that "St. Petersburg" versus "Saint Petersburg" is manufacturing failures. The worry was measured rather than assumed. Of the full system's wrong answers, 1 of 65 (2%) on WebQSP and 6 of 99 (6%) on CWQ are surface-form near-misses under aggressive normalisation. The failure census of Section 5.14 is reading real errors.

5.5.2 Accounting for Hedges and Abstentions

A *hedge* is an empty predicted entity set, meaning the system declined to assert. Hedges score zero on both accuracy metrics, but the hedge rate gets its own column, so the trade between abstention and accuracy stays visible. A system can buy precision with abstention. The reader

has both numbers and can watch it happen. Hedges are also not wrong answers, and Section 5.14 keeps the two apart throughout. A wrong answer is a reasoning error. A hedge is usually a coverage gap that never produced a committal claim.

One counting convention follows from this, and it governs every hits/hedges/wrong triple in the thesis. *The three counts always partition the whole set*, and nothing leaves the denominator. In particular, questions the environment provably cannot answer (Section 3.3.3) count as hedges rather than being excluded, even though the system had no route to a correct answer. Excluding them would flatter every condition equally. It would also make the counts incomparable with the accuracy figures, which include them.

5.5.3 Two-Tier Groundedness and Hallucination Rate

Groundedness is measured at two levels. The word *tier* in this thesis refers to this metric and nothing else.

Tier 1, structural, is deterministic, free, applies to every asserted entity in all $4{,}000$ records, and operationalises Section 2.2.2 directly. *An asserted entity is graph-grounded if and only if it exists as a node in the knowledge environment and is connected to at least one of the question's topic entities within four hops.* The entity-level ungrounded rate is the fraction of asserted entities failing this test. The question-level rate is the fraction of answered questions containing at least one such entity. The test is deliberately strict about surface form. A parametric spelling that does not exist as a node is ungrounded *by definition of the environment*. That strictness is the point of defining hallucination relative to an environment at all.

Tier 2, semantic, asks the harder question Tier 1 cannot. Is the entity supported *as the answer*? For a stratified sample of 60 asserted answers per system per dataset (600 judgements), the graph's own shortest connecting paths between topic and asserted entity are retrieved. A language model judges whether any one of them expresses the relationship the question asks about. The governing instruction, that *mere connectedness is not support*, is what separates the two tiers. Two design choices are pre-specified and consequential. Judging is against **the environment** rather than each system's own retrieved context. The logs do not preserve per-system evidence uniformly, and environment-judging asks every system the identical question. The metric therefore applies even to the no-retrieval control. And path retrieval takes the three shortest paths. That means a supporting chain can lose to shorter irrelevant ones. Tier 2 rates are accordingly *conservative lower bounds*, uniformly depressed across all systems, and are read as a between-system comparison rather than as absolute support rates.

5.5.4 Efficiency: Tokens, LLM Calls, and Wall-Clock Latency

Tokens and calls are counted by the shared budget meter, identically for every system, and are exact. The cache replays original usage, so they are unaffected by which runs were warm. Wall-clock latency is not, and is computed over **cold-cache records only**. That restriction is repeated wherever latency appears, beginning with the seconds column of Table 5.10. Where a condition's records are entirely cache replays there is no cold-cache mean at all. That is why Table 5.11 drops its seconds column rather than carry one measured on four conditions of five.

5.5.5 The Groundedness Judge and Its Validation

The Tier-2 judge sees a question, one asserted entity, and up to three connecting paths rendered as relation chains with mediator nodes masked. It does not see which system produced the assertion. It does not see whether the assertion was scored a hit, what the gold answer is, or what the system's own reasoning was. The prompt was frozen before any per-system result was inspected. The judge runs on the same backbone as the systems it judges. Section 5.18 states that overlap as a limitation; this chapter does not claim independence from the model family.

One hundred judged items were drawn and written to a sheet with the judge's verdicts stripped out. The author then labelled them *blind*. Neither the judge's output file nor the answer key was opened until the sheet was saved. Blinding is what makes the resulting statistic a validity check rather than a measure of anchoring. The written annotation guideline fixed during calibration is reproduced in Appendix D.1. Appendix D.10 records how the sample was drawn and labelled. Every one of the guideline's eight rules was forced by a case. The last of them says to treat a path expressing only a question's identifying descriptor, rather than its actual ask, as unsupported. That is the *composition-echo* case that the failure census later finds at scale (Section 5.16.2).

On $n = 100$ items, observed agreement is 85%, with near-symmetric marginals (54 human-supported against 51 judge-supported) and near-symmetric disagreement (9 human-yes/judge-no against 6 human-no/judge-yes). So neither rater is systematically the more lenient. Cohen's κ [38] is

$$\kappa = \frac{0.8500 - 0.5008}{1 - 0.5008} = 0.6995.$$

The pre-specified threshold was $\kappa \geq 0.7$. Rounded to three decimals this reads 0.700 and appears to clear, but it does not. It falls short by 0.0005, and it is reported as short. What follows is a matter of weight rather than of discarding. By the conventional bands [39] a κ of 0.70 is substantial agreement. The task is genuinely hard. Whether a relation chain really expresses an asked relationship has real grey area for a careful human, as the guideline's list of decided edge cases attests. The Tier-2 numbers are therefore reported with the κ beside them and the miss stated. No conclusion rests on Tier 2 alone. Where it is load-bearing, in the between-system semantic-support comparison of Section 5.11, the finding is also supported by the precision

column of the main matrix, which is measured without a judge. Tightening the judge prompt and re-judging a fresh sample until the bar was cleared was available and pre-committed. It was not taken, because the honest number was already in hand, and re-running until a pre-specified threshold is met is exactly what pre-specification exists to prevent.

5.6 Ablation Conditions and the Development Set

Four conditions, each a single deviation from the frozen configuration, each isolating one mechanism:

- **no-planner.** Decomposition is disabled, and the whole question becomes one objective. Anchor seeding is untouched, so this varies decomposition and nothing else.
- **no-backtrack.** It sets $\mathsf{max_backtracks} = 0$, a configuration change which correctly alters the budget hash stamped on the records.
- **no-verifier.** It sets $\mathsf{verify_claims} = \text{false}$. The drafting call is unchanged, claims included, but no claim is checked and the draft is emitted verbatim (Section 4.13). So the condition varies the checking and not the drafting.
- **embedding-only.** It sets $\alpha = 1.0$, removing the language-model term from the hybrid scorer while leaving τ at its frozen value.

The reference row is the *unmodified* system restricted to the same questions, obtained by filtering the main-matrix records to the half-set question identifiers rather than by re-running them. That filtering also supplies the matched pairs McNemar's test requires. Comparing against the full 400-question score would be invalid, accuracy on 400 questions and on 198 being different numbers for the same system. Ablations run on stratified halves of 200 WebQSP and 198 CWQ questions. That was the pre-specified response to the benchmark's burn coming in above projection, and it cut the ablation *sample* rather than any system or condition. The odd 198 is floor division within each stratum, CWQ's $137/211/49/3$ halving to $68/105/24/1$. The consequence, reduced statistical power, is confronted in Section 5.13.

5.6.1 The Development Set and the α–τ Sweep

The development set is 80 questions, disjoint from both test samples by an assertion in the test-set builder. Seventy-seven were mined from the train and validation splits to span hop counts and to include mediator-heavy and constraint-bearing questions, and certified for answer reachability by the procedure applied to the test samples. The other three are hand-written and, having no gold answer to reach, fall outside both. That is why the coverage report carries 77 rows. The set

contains the twenty-question smoke set of Section 4.9. That is why identifiers such as smoke-20 appear in the dev-set listing of Appendix C. Seven questions are unanswerable, in two buckets that are not interchangeable. Three are hand-written ones naming a fabricated entity, so that a correct refusal can be distinguished from a coverage failure. Four are ordinary benchmark questions with real gold answers this environment cannot express (Section 3.3.3).

The two free parameters of the navigation loop are the blend weight α in the hybrid scorer (Section 4.4.2) and the low-signal backtracking threshold τ (Section 4.5.3). They were swept jointly over $\alpha \in \{0.3, 0.5, 0.7, 1.0\}$ and $\tau \in \{0.0, 0.20\}$. That gives eight conditions, each a complete run of all 80 questions, with two additional draft-only conditions to price the verification layer. Each condition is a constructor argument whose values are stamped into every record it writes. So no condition can be mislabelled after the fact. The chosen values, $\alpha = 0.7$ and $\tau = 0.20$, were frozen on the basis of that sweep and never revisited. Section 5.7.1 reports it.

5.6.2 Implementation, Environment, and Reproducibility

The system is implemented in Python against LangGraph for the state machine, the Neo4j Python driver for graph access [21], and sentence-transformers for embeddings [29], with the graph in a local Neo4j instance. All Cypher [22] is templated inside the tool layer and never composed by a language model (Section 4.2). Randomised operations use seed 42 throughout.

Three practices carry more reproducibility weight than the dependency list. Appendix E records them as techniques. The first is that *records self-describe*. Every record carries the backbone description, a budget-configuration hash, the full run configuration, and the source commit. That self-description is what made the cache-replay incident of Section 5.3 diagnosable. The second is that *frozen artifacts are committed and regenerable ones are not*, the question sets and result files embodying spent money and non-repeatable runs where the graph import and the indexes are deterministic products of committed scripts. The third is that *code changes are proved harmless by cache identity* rather than by a passing test suite. Re-running an answered question and confirming every call is a cache hit proves byte-identical prompts and hence an untouched frozen path. That proof is the certificate behind the ablation reference row.

Two limits belong here rather than in a footnote. The archived results carry neither per-folder documentation nor generated manifests. So file provenance rests on the self-describing records and on this chapter. And one generated summary in the development-set archive predates a fix applied to the runs it summarises. So rescoring that directory yields different numbers than the committed summary. The discrepancy stands. Section 5.7.1 states it where those numbers are used, because editing a generated file is not a fix.

Table 5.3: The development-set sweep, $n = 80$. Hits, hedges, and wrong answers add up to the whole set. Verify means claim verification was enabled. The two draft-only rows disable it. All rows come from the sweep as it was run, before the correction to the answer-entity filter of Section 4.13.4. The text gives the frozen condition's post-correction figures.

α	τ	Verify	Hits	Hedge	Wrong	F1	Tokens	Calls	Bktrk
0.3	0.00	yes	61	12	7	0.728	4,999	7.4	16
0.3	0.20	yes	61	12	7	0.728	5,030	7.5	18
0.5	0.00	yes	62	10	8	0.743	4,874	7.2	16
0.5	0.20	yes	62	10	8	0.743	4,874	7.2	16
0.7	0.00	yes	64	11	5	0.769	4,925	7.3	17
0.7	**0.20**	**yes**	**64**	11	5	0.769	4,925	7.3	17
1.0	0.00	yes	59	15	6	0.713	3,408	4.9	22
1.0	0.20	yes	59	16	5	0.713	3,400	4.9	26
0.5	0.20	no	64	9	7	0.770	4,807	7.1	16
0.7	0.20	no	66	10	4	0.797	4,858	7.2	17

5.7 Main Results

This part reports the measurements that Section 5.1 specified, arranged so that the weakest claims sit next to the strongest. The headline accuracy result is large and significant on every comparison. The structural groundedness result is emphatic but points to a different cause than the one expected. The semantic groundedness result is underpowered. Three of the four ablations detect nothing, yet all four outcomes get the same volume.

Every test-set number comes from results/phase4/thesis_numbers.json. The development sweep of Section 5.7.1 is the one set of figures that does not. It reads the archived phase-3 runs directly, as Chapter 4 does. Confidence intervals are 95% bootstrap intervals over questions, from 10,000 resamples at seed 42 [36]. Paired comparisons use the exact-binomial form of McNemar's test [37] on per-question correctness.

5.7.1 Development-Set Tuning Outcomes

The sweep varied the blend weight α and the low-signal backtracking threshold τ jointly over the 80-question development set (Section 5.6.1). Table 5.3 reports all eight conditions, plus the two draft-only conditions that put a price on the verification layer.

The sweep supports three readings. The first, and the only one it was run to obtain, is that $\alpha = 0.7$ is the maximum. Appendix E.13 gives the other two. The curve is not clean. The wrong-answer column peaks at $\alpha = 0.5$, where the system commits more often, and τ changes almost nothing at the values swept. That flatness is what the signal-maximum formulation of Section 4.5.3 predicts. Both facts are reported rather than smoothed away. In particular, $\tau = 0.20$ was frozen on the strength of a sweep in which it barely mattered. That is stated plainly rather than dressed up as a tuned choice.

Table 5.4: Main results, $n = 400$ per dataset. Brackets are 95% bootstrap intervals. **P** and **R** are mean per-question precision and recall. **Hedge** is the share of questions on which the system asserted nothing. The Hits@1 ceilings are 97.0% and 99.2% (Table 5.1). Every figure here is measured inside the environment of Chapter 3. That environment's per-question construction is what puts the ceilings so high. So the levels are not comparable with published results over full Freebase. Section 5.18 states why.

Data	System	Hits@1	F1	P	R	Hedge
WebQSP	No-retrieval	0.453 [0.403, 0.500]	0.305 [0.266, 0.345]	0.380	0.303	12.2%
WebQSP	Vector-RAG	0.568 [0.517, 0.618]	0.432 [0.389, 0.475]	0.469	0.460	27.0%
WebQSP	GraphRAG	0.340 [0.292, 0.388]	0.262 [0.224, 0.302]	0.279	0.284	55.8%
WebQSP	Think-on-Graph	0.660 [0.613, 0.705]	0.477 [0.436, 0.518]	0.571	0.480	18.8%
WebQSP	**AGR**	**0.755** [0.713, 0.797]	**0.642** [0.600, 0.684]	**0.697**	**0.653**	**8.2%**
CWQ	No-retrieval	0.307 [0.263, 0.352]	0.279 [0.236, 0.323]	0.285	0.282	38.0%
CWQ	Vector-RAG	0.203 [0.163, 0.242]	0.168 [0.133, 0.204]	0.171	0.189	48.2%
CWQ	GraphRAG	0.205 [0.168, 0.245]	0.183 [0.147, 0.222]	0.187	0.194	60.8%
CWQ	Think-on-Graph	0.435 [0.388, 0.482]	0.378 [0.332, 0.423]	0.403	0.384	22.5%
CWQ	**AGR**	**0.522** [0.475, 0.570]	**0.469** [0.423, 0.514]	**0.484**	**0.481**	23.0%

Verification costs accuracy on this set. Against the frozen condition, the draft-only row scores 66 hits to 64 and F1 0.797 to 0.769. The answer-entity filter of Section 4.13.4 was later corrected, on development data and before any test question ran. After that correction the frozen condition scores 65 hits, 11 hedges, 4 wrong, F1 0.785. Across those two rows the layer costs one correct answer and removes no wrong ones on this set of 80. That comparison spans the correction, since the draft-only condition was never re-run after it. It is therefore the closest available reading rather than a matched pair. Section 5.13 finds the same thing at $n \approx 200$ on test data. Section 5.17.1 explains why that is the expected result rather than a disappointing one. Only the frozen condition was re-run after the correction. So Table 5.3 is the pre-correction sweep throughout. It is internally consistent, since all eight conditions ran under the same code. Where the committed summary and the archived per-question records disagree, the records are authoritative.

5.7.2 Main Results on WebQSP

On WebQSP, AGR answers 302 of 400 questions correctly. That is 75.5% against a reachability ceiling of 97.0%, +9.5 points over Think-on-Graph, and +30.2 over the parametric floor. The floor matters. WebQSP is a decade old and quite possibly in the backbone's pretraining data. The no-retrieval control confirms as much. The backbone answers 45.3% of these questions from memory alone. Any claim about what retrieval contributes therefore starts from 0.453, not from zero.

F1 widens the margin that Hits@1 understates. Against Think-on-Graph the Hits@1 gap is +9.5 points; the F1 gap is +16.5 (0.642 against 0.477). The precision column, at 0.697 against

0.571, shows where the extra margin comes from. WebQSP questions carry seven gold answers on average, so a system that asserts a long list collects Hits@1 credit for the one entity that matches and pays nothing for the rest. Precision charges for the rest. Together the two columns support the sentence this thesis leads with. AGR answers more questions and asserts fewer falsehoods. It does so at an 8.2% hedge rate, the lowest of the five systems, so the precision is not bought with abstention.

Raw hits mislead on the retrieval baselines. The correction is a finding. GraphRAG scores 0.340, below the no-retrieval control's 0.453. That gap invites the conclusion that the implementation is broken. Put the wrong-answer counts beside the hits and the picture inverts. No-retrieval asserts on 351 of 400 questions and is wrong on 170 of them. That is 51.6% *assertion precision*, the share of the questions a system commits on that it answers correctly. It is a per-question measure. It is also distinct from the per-entity precision of Table 5.4, which is averaged over all 400 questions whether the system committed on them or not. GraphRAG asserts on 177 and is wrong on 41, for 76.8%. The mechanism is the prompt. The retrieval baselines are instructed to answer only from the facts given and to return nothing otherwise. When retrieval misses, they therefore abstain, while the no-retrieval control has no such constraint and guesses from memory. On a memorised benchmark, guessing pays. *A grounded system scoring below an ungrounded one on raw hits while committing four times fewer falsehoods* is the contamination story the control was built to expose. It is reported as a result rather than as an anomaly.

5.7.3 Main Results on ComplexWebQuestions

On CWQ, AGR answers 209 of 400 questions. That is 52.2% against a 99.2% ceiling, +8.7 points over Think-on-Graph and +21.5 over the floor. Every system scores lower here than on WebQSP. But the ordering changes in one important way. *Both* static retrieval baselines now fall below the parametric control. Vector-RAG drops from 0.568 to 0.203 and GraphRAG from 0.340 to 0.205, while no-retrieval falls only from 0.453 to 0.307.

The reason is structural rather than incidental, though the two baselines earn that description to different degrees. Vector-RAG's context is a single verbalised triple. One triple cannot hold a chain, at any retrieval quality or any radius. Section 5.4 predicted that paradigm limit at implementation time, and Section 5.10 shows it stratum by stratum. GraphRAG's context is a one-logical-hop neighbourhood (Section 5.4.2). Its CWQ number is the weaker evidence of the two, because it confounds the paradigm with the radius. Part of its fall belongs to a boundary chosen in advance, not to static retrieval as such. The claim therefore rests on Vector-RAG.

The number worth leading with on this dataset is assertion precision. AGR and Think-on-Graph commit at effectively the same rate, answering 308 and 310 of 400 questions rather than hedging. From those near-identical commitments, AGR is right 209 times and wrong 99, against 174 and

Table 5.5: Paired McNemar tests of AGR against each baseline, on per-question correctness over the same 400 questions. **AGR only** counts the questions that AGR alone answers correctly; **Baseline only** counts the reverse. Every comparison rejects.

Dataset	Baseline	AGR only	Baseline only	p
WebQSP	No-retrieval	152	31	2.3×10^{-20}
WebQSP	Vector-RAG	117	42	2.2×10^{-9}
WebQSP	GraphRAG	186	20	6.5×10^{-35}
WebQSP	Think-on-Graph	76	38	4.8×10^{-4}
CWQ	No-retrieval	121	35	2.7×10^{-12}
CWQ	Vector-RAG	151	23	2.8×10^{-24}
CWQ	GraphRAG	148	21	1.0×10^{-24}
CWQ	Think-on-Graph	86	51	3.5×10^{-3}

136 for Think-on-Graph. That is 67.9% assertion precision against 56.1%. On the harder, deeper benchmark, the system with an explicit plan, a guided scorer, and a claim check finds 35 more answers and asserts 37 fewer falsehoods than beam search does on the same graph, with the same model, from the same number of commitments. Section 5.8 attributes that margin to the budget rather than to resolution quality.

Table 5.5 settles the comparisons that matter, and all eight reject. That includes the two against the agentic baseline, the only comparison in the table where both systems navigate the same graph, with the same model, under the same call budget. The discordant counts make the margin concrete rather than distributional. On CWQ, AGR uniquely solves 86 questions Think-on-Graph misses against 51 the other way. What that budget does to the comparison is the subject of the next section. It changes what the margin means.

5.8 Where the Margin Over the Agentic Baseline Comes From

The shared budget is a control, and like any control it is doing something. The call cap binds on Think-on-Graph far more often than on AGR, because beam search costs width times depth. Section 5.12 records the clip rates. So "same budget" does not mean "same freedom to search". Splitting the 400 questions of each dataset on whether the cap actually cut Think-on-Graph off separates the two things the aggregate confounds. One is how good its search is. The other is whether it can afford to run that search to completion.

On the questions it is allowed to finish, Think-on-Graph is ahead on both datasets. It scores 0.852 against AGR's 0.788 on WebQSP and 0.629 against 0.607 on CWQ. The entire aggregate margin comes from the 29.2% and 44.0% of questions where the cap truncates it. There its accuracy falls to 0.197 and 0.188 while AGR's holds at 0.675 and 0.415. This is stated plainly for two reasons. It is the first thing an unsympathetic reading of Table 5.5 should look for. And

Table 5.6: The comparison against the agentic baseline, split on whether the shared 25-call cap cut it short. Clipping is read from the per-record budget_exhausted trace flag, not from a call count, because a run may spend its last permitted call on the step that finishes it. The *combined* rows reproduce the Hits@1 column of Table 5.4, recomputed independently from the raw records. Generated from results/phase4/thesis_numbers.json.

Dataset	**Subset**	n	**ToG**	**AGR**	Δ
WebQSP	ToG ran to completion	283	0.852	0.788	−0.064
	ToG clipped by the cap	117	0.197	0.675	+0.478
	combined	400	*0.660*	*0.755*	**+0.095**
CWQ	ToG ran to completion	224	0.629	0.607	−0.022
	ToG clipped by the cap	176	0.188	0.415	+0.227
	combined	400	*0.435*	*0.522*	**+0.087**

burying it would misrepresent what was measured.

This does not weaken the result but changes what the result says. The finding is not that guided best-first search resolves entities better than language-model-pruned beam search. On the evidence here it does not, and the unclipped subset is the fairest available estimate of that. The finding is that *beam search cannot afford to finish under a budget a deployed system would actually impose*, and *guided search can*. Width times depth multiplies, and every question pays it. A plan plus a scored frontier does not multiply. A comparison run without a cap would measure search quality in the abstract. That would be a legitimate experiment, and a different one. This thesis fixed the budget in advance (Section 5.4) because cost is a property of the method, not a nuisance to control away. On that reading the clip rate is not an artefact of the setup but the result. Section 5.12 prices the same finding in tokens.

Three qualifications keep the claim honest. The first is that the cap of 25 calls is itself a choice. A larger one would move the split, and the clip rates are reported so a reader can see how far. The reimplementation's beam width and depth follow the original paper. So its cost profile is the published algorithm's rather than a weakened variant. That fidelity is what makes the cost finding meaningful instead of circular. But the candidate widths *beneath* the beam do not come from the paper. The baseline prunes from 40 relations and 20 neighbours where AGR prunes from 300 and 200. Those cuts bind on about a third of its expansions against almost none of AGR's (Section 5.4.3). That asymmetry cannot explain the clipping, because a thinner candidate set makes a step cheaper, not dearer. So the affordability finding stands, though the asymmetry does bear on the sentence before it. The 0.852 and 0.629 on the unclipped subset are a *lower bound* on what this baseline's search could resolve given AGR's candidate widths, not an estimate of it. The claim that guided search does not out-resolve beam search is correspondingly the weaker one.

The breadth of AGR's answers is checked rather than assumed. On answered WebQSP questions AGR names 3.37 entities against Think-on-Graph's 2.56. A metric scoring a hit on

Table 5.7: AGR against RoG's published figures, in points. RoG's row is Table 1 of [9], measured over the full test splits (1,628 and 3,531 questions). AGR's row is Table 5.4, measured over the certified 400-question samples of Table 5.1. Those samples are stratified draws from those same splits. So the two rows are computed over nested populations, but under different protocols. The paragraphs below say what that permits and what it forbids.

	WebQSP		CWQ	
System	**Hits@1**	**F1**	**Hits@1**	**F1**
RoG (published)	85.7	70.8	62.6	56.2
AGR (this work)	75.5	64.2	52.2	46.9

any match rewards naming more. The check is entity precision over those same answered questions, where AGR leads 0.760 to 0.703. That is Table 5.4's measure restricted to the answered subset, not the per-question assertion precision of Section 5.7.3. On CWQ the figures are 1.54 against 1.40 entities and 0.629 against 0.520 precision. If AGR bought breadth without accuracy, precision would fall as the entity count rose. It does not fall.

5.9 Positioning Against Published RoG Results

Two facts in this thesis invite a direct comparison. The knowledge environment of Section 3.1.3 is built from the per-question subgraphs that the RoG distribution publishes, and Section 2.3.3 surveys RoG [9] as the closest published system in the path-retrieval family. A reader who notices both will ask how the two systems compare. The aggregate answer does not favour this work.

RoG leads on all four figures. The margin is not sampling noise. AGR's 95% confidence intervals on Hits@1 are $[71.3, 79.7]$ on WebQSP and $[47.5, 57.0]$ on CWQ. RoG's 85.7 and 62.6 fall outside both, a fact stated before the qualifications. The qualifications do not reverse it, and a comparison offered only with its excuses attached is not a comparison.

Five differences bound what Table 5.7 establishes. The first is decisive, and the second favours RoG by an amount nothing here measures. The third and fourth are ordinary, and the fifth is measured and small.

RoG is trained on these benchmarks. AGR is trained on nothing. Section 5.1 of [9] states that its backbone is "instruction finetuned on the training split of WebQSP and CWQ as well as Freebase for 3 epochs". In the distribution this thesis also uses, those training splits hold 2,826 and 27,639 questions. AGR performs no training of any kind. The backbone is frozen and prompted at temperature 0 (Section 5.3), and the system has seen no question from either benchmark before it is evaluated. The comparison is therefore between a supervised system and a zero-shot one. That is a difference in kind rather than in degree. It is the same objection Table 2.2 already raises against every fine-tuned entry in the surveyed literature.

RoG searches a smaller graph. RoG instantiates its relation paths inside each question's own subgraph, which its released code builds from that question's record alone. AGR navigates the union of them all, which Section 3.2 builds because a per-question subgraph on its own is close to an oracle. The difference favours RoG, and nothing here measures by how much.

The backbones differ. RoG fine-tunes LLaMA2-Chat-7B, whereas AGR prompts the model named in Section 5.3. Section 2.5 argues that backbone accounts for much of the spread in cross-paper tables. That is precisely why Section 5.1 holds it constant across the five systems it measures. It cannot be held constant against a system whose contribution *is* its fine-tuned weights. So no protocol available here would make this row comparable in the sense Table 5.4 uses the word.

The evaluation populations differ. RoG reports over the full test splits. The figures here are over stratified 400-question samples of the same splits (Section 5.2). The samples preserve the hop-stratum proportions of their populations. So they estimate the same quantity. The intervals quoted above are the estimate's uncertainty. RoG publishes no interval, so the gap can be bounded from one side only.

The scorers differ, by less than a point. RoG's released scorer lower-cases both strings and strips punctuation and articles. It then counts a gold answer as found anywhere inside the predicted text, where Section 5.5 matches whole entities. RoG's paper defines Hits@1 by the top-1 answer, but its scorer reads every answer emitted, as Section 5.5 does. Scored with RoG's functions, copied unchanged into scripts/rog_scorer.py, AGR's figures become 76.0 and 64.9 on WebQSP and 52.2 and 47.2 on CWQ. Hits@1 changes on two WebQSP questions, such as "Kingdom of Denmark" against the gold "Denmark", and on no CWQ question. No figure moves by as much as a point.

What the comparison does settle is worth stating positively. It fixes AGR's place in the published landscape. A training-free system reaches 75.5 and 52.2 against a supervised system's 85.7 and 62.6, on the same data, with no in-domain examples. And it locates this thesis's contribution somewhere other than the accuracy column. RoG's answers are read off retrieved paths, which makes them faithful by construction. The pre-generation verification question of Section 4.10 never comes up for them (Section 2.3.3). The price is that the plan is generated once, before retrieval, with no mechanism to revise it when it fails to instantiate. AGR's claims are the opposite shape. They are generated freely and then checked against the graph the agent actually walked. That check is what makes an unsupported assertion detectable at all. Neither the component-level ablation of Section 5.13, with the stratum-dependent decomposition finding it produced, nor the failure census of Section 5.14 has a counterpart in RoG's evaluation. None of them would be settled by closing the gap in Table 5.7.

Table 5.8: Hits@1 and F1 per hop stratum. Stratum sizes are those of Table 5.1. WebQSP's h3plus has $n = 4$ and is not interpreted.

Data	System	h1	h2	h3plus	unreach.
WebQSP	No-retrieval	0.43 / 0.29	0.52 / 0.36	0.25 / 0.00	0.17 / 0.17
WebQSP	Vector-RAG	0.82 / 0.63	0.12 / 0.07	0.00 / 0.00	0.08 / 0.08
WebQSP	GraphRAG	0.30 / 0.23	0.44 / 0.35	0.25 / 0.07	0.08 / 0.08
WebQSP	Think-on-Graph	0.63 / 0.45	0.78 / 0.58	0.25 / 0.07	0.17 / 0.17
WebQSP	**AGR**	0.75 / 0.64	**0.84 / 0.73**	0.00 / 0.00	0.08 / 0.08
CWQ	No-retrieval	0.38 / 0.35	0.30 / 0.27	0.14 / 0.14	0.00 / 0.00
CWQ	Vector-RAG	0.33 / 0.27	0.14 / 0.11	0.10 / 0.09	0.67 / 0.67
CWQ	GraphRAG	0.34 / 0.30	0.16 / 0.14	0.02 / 0.02	0.33 / 0.33
CWQ	Think-on-Graph	**0.53** / 0.48	0.37 / 0.32	0.45 / 0.32	0.33 / 0.33
CWQ	**AGR**	0.46 / 0.42	**0.55 / 0.49**	**0.57 / 0.50**	0.33 / 0.33

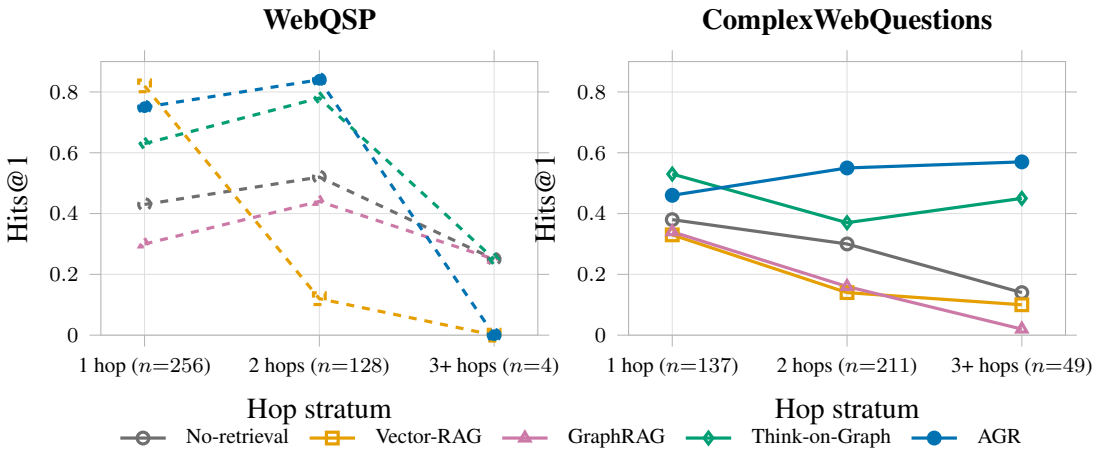


Figure 5.1: Hits@1 across hop strata. The shape is the finding. On ComplexWebQuestions AGR is the only system whose accuracy rises with hop count. Every other profile ends below where it started, three of them monotonically and Think-on-Graph by falling and partly recovering. WebQSP is drawn dashed because its h3plus stratum holds four questions and cannot carry a trend. The stratum sizes are printed on the axis for that reason. The unreachable stratum is omitted here and discussed below, since by construction it measures assertion rather than accuracy. Generated from results/phase4/thesis_numbers.json.

5.10 Accuracy Broken Down by Hop Count

Table 5.8 is the direct answer to RQ1. It answers it with a shape rather than with a difference of means, and Figure 5.1 is that shape.

On CWQ, AGR's accuracy rises with hop count. Hits@1 goes $0.46 \rightarrow 0.55 \rightarrow 0.57$ across h1, h2, h3plus, and F1 $0.42 \rightarrow 0.49 \rightarrow 0.50$. It is the only system on that dataset that ends above where it started. The other four all end below, and three of them decay monotonically. No-retrieval goes $0.38 \rightarrow 0.30 \rightarrow 0.14$, Vector-RAG $0.33 \rightarrow 0.14 \rightarrow 0.10$, and GraphRAG $0.34 \rightarrow 0.16 \rightarrow 0.02$, though that last row is radius-limited and is read with the qualification

given below. Think-on-Graph is the exception to the monotonicity but not to the direction. It falls from 0.53 to 0.37 and partially recovers to 0.45, still 0.08 below its own one-hop score. The distinction is worth keeping. "Every other system decays" would be the tidier sentence, but the table does not say it. A system that gets *better* as the required chain gets longer is not merely ahead on average. Its profile is the only one in the table that fits a system which actually traverses the chain. That rising profile is a stronger form of the multi-hop claim than any aggregate could carry.

Think-on-Graph beats AGR on CWQ's one-hop stratum (0.53 against 0.46, $n = 137$). This is reported rather than buried because it makes the rest of the story more credible, not less. Section 5.13 identifies the mechanism. Decomposition costs accuracy on questions that do not need decomposing. CWQ's `h1` stratum is full of them.

Vector-RAG's WebQSP profile is the paradigm in two cells. It scores 0.82 at one hop and 0.12 at two. The one-hop figure is the best single-hop number anywhere in the table, better than AGR's 0.75. When one triple suffices, dense retrieval over verbalised triples is excellent. When a chain is required, it has nothing to offer. This is the cell the multi-hop argument rests on. A one-triple context is chain-incapable by construction rather than by configuration.

GraphRAG's WebQSP inversion has two causes. Only one of them was originally named. It scores 0.30 at `h1` against 0.44 at `h2`. Part of that is hub noise, since popular one-hop entities have large neighbourhoods. The answer is drowned in the reranked window. More exactly, it is drowned *or absent*, since the retrieval step samples at most 100 edges per topic entity without an ordering, and 55.1% of its topic entities carry more edges than that (Section 5.4.2). Which of the two applies to any given question is not recoverable from the run records. But the inversion also marks the edge of the retrieval radius. The same section scopes that as a bound on this implementation rather than on static retrieval as such. WebQSP's `h2` stratum is largely mediator paths, which a one-hop expansion reaches by hopping through the mediator. CWQ's `h2`, by contrast, is genuine composition, which it cannot reach at all. The scores are 0.44 against 0.16 on nominally the same stratum depth. Read GraphRAG's per-stratum row as measuring what its radius covers, not how far static retrieval can see.

The unreachable stratum is a hallucination probe, not an accuracy measurement. For these questions, by construction, no path to gold exists within the hop bound. So every "hit" is an assertion the environment cannot support, one that just happened to match the label. Vector-RAG scores 0.67 on CWQ's three unreachable questions and no-retrieval 0.17 on WebQSP's twelve. The counts are tiny, and no weight is placed on them. But the direction is exactly what Section 5.11 measures at scale. Hits@1 will happily reward what RQ2 is about the absence of.

Table 5.9: Two-tier groundedness. Tier 1 is deterministic over every asserted entity in all 4,000 records. Tier 2 is a language-model entailment judgement over a stratified sample of 60 assertions per system per dataset. The judge itself was validated separately, at $\kappa = 0.6995$ over 100 paired annotations (Section 5.5.5).

		Tier 1: structural			Tier 2
Data	**System**	**Entities**	**Ungrounded**	**Questions**	**Supported**
WebQSP	No-retrieval	661	179 (27.1%)	37.6%	63.3%
WebQSP	Vector-RAG	1040	38 (3.7%)	6.2%	61.7%
WebQSP	GraphRAG	800	6 (0.8%)	2.8%	55.0%
WebQSP	Think-on-Graph	832	**0 (0.0%)**	**0.0%**	61.7%
WebQSP	AGR	1235	**0 (0.0%)**	**0.0%**	**66.7%**
CWQ	No-retrieval	340	42 (12.4%)	13.7%	36.7%
CWQ	Vector-RAG	289	7 (2.4%)	3.4%	**50.0%**
CWQ	GraphRAG	224	2 (0.9%)	1.3%	36.7%
CWQ	Think-on-Graph	433	**0 (0.0%)**	**0.0%**	43.3%
CWQ	AGR	474	**0 (0.0%)**	**0.0%**	48.3%

5.11 Groundedness and Hallucination Rate Across Systems

5.11.1 Tier 1: Structural Hallucination Is a Property of the Paradigm

The differential is emphatic. The parametric control asserts 661 entities on WebQSP, of which 179 (27.1%) have no basis in the knowledge environment. And 37.6% of the questions it answers contain at least one such entity. Both navigating systems sit at exactly zero, over 1,235 and 832 asserted entities. Parametric answering hallucinates structurally at double-digit rates. Graph-navigated answering removes structural hallucination outright.

But Think-on-Graph reaches zero as well. That changes the attribution. Structural groundedness is a property of the *navigation paradigm*. Any system that copies its answer entities out of triples it actually traversed inherits it. It is not a property of this thesis's verification layer specifically. So the honest claim is narrower, and more testable, than the one the work set out to make. *Both navigating systems achieve zero structural hallucination; they differ at the level of semantic support*. Think-on-Graph's precision of 0.571 and 0.403, against AGR's 0.697 and 0.484 (Table 5.4), shows it asserting far more entities that exist, connect to the topic, and were copied from real triples, yet are not the answer. Section 1.3 builds on exactly this gap.

Two readings of the middle band belong on the record. The no-retrieval control's CWQ rate (12.4%) is less than half its WebQSP rate (27.1%). That is not because it fabricates less on harder questions. It is because it hedges more, at 38.0% against 12.2%. Abstention pushes the measured hallucination rate down. That is the abstention trade appearing in a third metric. And Vector-RAG's 38 ungrounded entities are mostly not fabrications, because its index is global. So retrieval can surface facts about a similarly-named subject that exists in the graph but has no connection to the question's topics. The model then copies faithfully from them. The metric

correctly flags those as unanchored to the question. But the mechanism is retrieval error rather than generation error. Structural ungroundedness conflates the two. The distinction is recoverable from whether the entity appears in the system's own retrieved facts.

5.11.2 Tier 2: A Narrow Band and an Underpowered Comparison

Tier 2 asks whether the asserted entity is supported *as the answer*. Under the instruction that mere connectedness is not support, every system lands in a 36.7–66.7% band. AGR is highest on WebQSP at 66.7% and second on CWQ. It is 5.0 points ahead of Think-on-Graph, which is the comparison the rest of this chapter makes, but 1.7 behind Vector-RAG's 50.0%. Its margin over the agentic baseline is 5.0 points on both datasets. Its lead over *every* system is a WebQSP result only.

Neither gap is resolvable at this sample size. With $n = 60$ per cell the 95% binomial interval has a half-width of roughly 12–13 points. So a 5-point difference is inside the noise, and so is Vector-RAG's 1.7. No conclusion in this thesis rests on either, but what the tier does establish is the level. Semantic support is *hard* for every system, including the two that are structurally perfect. Zero structural hallucination coexists with roughly a third to a half of assertions failing a strict semantic-support test. That coexistence is the single most useful thing the metric says. It says it about the paradigm, not about any one system.

Two properties of the measurement limit how it can be read, and both were pre-specified. The rates are conservative lower bounds, depressed by the same amount across systems. That uniformity keeps the between-system comparison intact and makes the absolute levels untrustworthy. And the judge was validated at $\kappa = 0.6995$ against a pre-specified bar of 0.7 (Section 5.5.5). That is substantial agreement, just short of the threshold. Both are reasons to treat this tier as corroborating rather than decisive.

5.12 Efficiency and Cost

5.12.1 Token and Call Budgets per System

Agency costs roughly ten times what one-shot retrieval costs. It takes $4{,}511$ and $6{,}818$ tokens per question against ~ 500, and 11–17 seconds against one or two. That is the price of RQ1's accuracy, and it is stated as a price.

The informative comparison is with the other agentic system, because the two run under the same 25-call cap. AGR uses *half* the calls, 6.2 against 12.8 on WebQSP and 8.9 against 18.2 on CWQ. It spends more tokens per call, because each call carries a richer prompt with a plan, a scored candidate set, and a fact window. Fewer, better-informed decisions rather than more, blinder ones.

Table 5.10: Cost per question. The seconds column uses cold-cache records only. **Clipped** is the share of questions on which the shared 25-call budget ran out mid-search.

Data	System	Tokens	Calls	Seconds	Clipped
WebQSP	No-retrieval	113	1.0	1.2	—
WebQSP	Vector-RAG	527	1.0	1.4	—
WebQSP	GraphRAG	531	1.0	2.1	—
WebQSP	Think-on-Graph	3,615	12.8	12.9	29.2%
WebQSP	AGR	4,511	6.2	11.2	0.0%
CWQ	No-retrieval	118	1.0	1.0	—
CWQ	Vector-RAG	535	1.0	1.4	—
CWQ	GraphRAG	470	1.0	2.2	—
CWQ	Think-on-Graph	5,572	18.2	15.2	44.0%
CWQ	AGR	6,818	8.9	17.2	0.0%

The clip column is the sharpest form of that. Beam search costs are multiplicative in width and depth. So under the shared budget Think-on-Graph exhausts its calls before finishing on 29.2% of WebQSP questions and 44.0% of CWQ questions, while AGR never does. On nearly half the multi-hop benchmark the baseline is answering from a truncated search. This is a fair constraint (Section 5.4.3), since both systems live under the same cap. It is the efficiency finding rather than a caveat on it. Under an identical budget, AGR converts its calls into correct answers at roughly twice the rate.

Never reaching the call cap is not the same as running unbounded. The distinction matters, because the rest of this thesis restates this result. AGR's own budgets bind, the depth cap of Table 4.3 hardest. The meter refuses a deeper expansion on 25.6% of test questions, against 6.2% for backtracks and 1.9% for repair iterations. Section 4.13 and Section 4.14.1 each narrate a run that ends that way. What the clip column establishes is the narrower and more useful thing. Of the two systems run under one shared call budget, that budget truncates only one.

5.12.2 The Accuracy–Cost Frontier

Reading Table 5.4 against Table 5.10, and plotted as Figure 5.2, the frontier depends on which cost is charged. The two costs this thesis records do not agree.

On tokens, the axis of Figure 5.2, Think-on-Graph is not dominated. It buys its lower accuracy at a genuinely lower token price on both datasets, 3,615 against AGR's 4,511 on WebQSP and 5,572 against 6,818 on CWQ. It sits on the Pareto set of both, alongside the no-retrieval control at 113 tokens and, on WebQSP, Vector-RAG at 527. The interior points are the static graph retrievers, which the parametric control beats on both axes at once. Any claim that Think-on-Graph is dominated has to name an axis on which that is true.

On calls it is dominated. Calls are what the budget meters (Section 4.7). Think-on-Graph spends 12.8 and 18.2 calls against AGR's 6.2 and 8.9. That is lower Hits@1 at roughly double

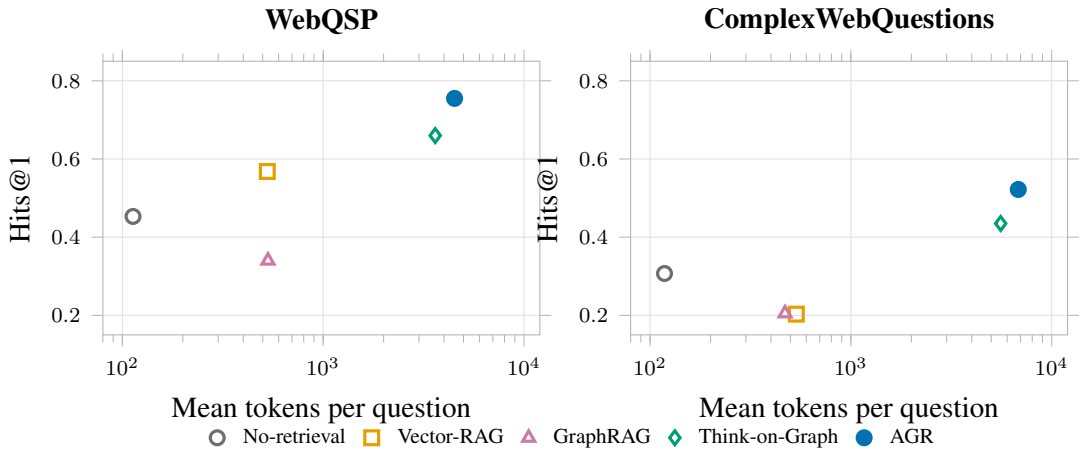


Figure 5.2: Accuracy against token cost, one point per system. The horizontal axis is logarithmic. The agentic systems cost roughly an order of magnitude more than the one-shot ones, and a linear axis would compress every non-agentic system into a single column. Read this figure for the token frontier only. Think-on-Graph sits below *and* to the left of AGR, so it is a frontier point on this axis, not a dominated one. The domination claim holds on calls, which are not plotted and are given in Table 5.10. Generated from results/phase4/thesis_numbers.json.

the call cost, on both datasets, with no interior ambiguity. The two costs diverge because AGR spends more than twice as many tokens per call, converting fewer, larger calls into a comparable token total. Which cost binds is a deployment question. Token billing separates the two systems very little, whereas a per-call rate or latency budget separates them by a factor of two. It is the call budget that clips Think-on-Graph on 29% and 44% of questions (Section 5.8).

Around that, the corners are unchanged. Vector-RAG on WebQSP is the cheap corner, 0.568 Hits@1 for 527 tokens and 0.82 on the one-hop stratum, which makes it the right choice for shallow questions and useless for deep ones. AGR is the accurate corner on both datasets. And the no-retrieval control holds the cheap end of the token frontier while being off any accuracy-worthy frontier once precision is counted. It remains the cheapest way to be right about half the time on WebQSP, which is the contamination point restated as an economic one.

5.13 Ablation Study

Four single-deviation conditions on stratified halves ($n = 200$ WebQSP, $n = 198$ CWQ), each against the unmodified system restricted to the same questions. Table 5.11 reports the conditions, and Table 5.12 summarises the component attribution. Throughout, Δ is *ablated minus reference*. So a positive Δ means removing the component *improved* the metric.

Table 5.11: Ablation conditions on stratified halves. The reference row is the frozen system restricted to the same questions, at no additional cost. P and R are the per-question means of Section 5.5, reported for the full systems in Table 5.4 and carried here so that each condition's precision effect can be read off directly. Cost is given in tokens and calls only. There is no seconds column, because every record in the no-verifier condition is a full cache replay and latency would not be comparable across conditions.

Data	Condition	Hits@1	F1	P	R	Hedge	Tok.	Calls
WebQSP	reference	0.755 [0.695, 0.810]	0.659 [0.599, 0.717]	0.710	0.664	8.5%	4,562	6.2
WebQSP	no-planner	**0.830** [0.775, 0.880]	**0.742** [0.687, 0.794]	0.788	0.749	5.5%	3,159	4.0
WebQSP	no-backtrack	0.745 [0.685, 0.805]	0.646 [0.586, 0.705]	0.704	0.649	9.5%	4,261	5.8
WebQSP	no-verifier	0.760 [0.700, 0.820]	0.663 [0.604, 0.721]	0.711	0.673	8.0%	4,460	6.1
WebQSP	embedding-only	0.740 [0.680, 0.800]	0.643 [0.583, 0.702]	0.701	0.644	13.5%	3,413	4.4
CWQ	reference	0.495 [0.424, 0.566]	0.445 [0.379, 0.511]	0.461	0.451	23.2%	7,105	9.1
CWQ	no-planner	0.434 [0.369, 0.505]	0.398 [0.333, 0.464]	0.407	0.405	26.8%	5,617	6.7
CWQ	no-backtrack	0.490 [0.419, 0.561]	0.445 [0.379, 0.511]	0.456	0.453	28.3%	5,792	7.4
CWQ	no-verifier	0.490 [0.419, 0.561]	0.436 [0.371, 0.503]	0.451	0.446	20.2%	6,704	8.7
CWQ	embedding-only	0.475 [0.409, 0.545]	0.437 [0.370, 0.502]	0.452	0.444	28.3%	4,925	5.9

5.13.1 Without the Planner: A Stratum-Dependent Effect

This is the one condition that reaches significance. And it points the opposite way from what the design intended.

On WebQSP, *removing* the planner improves every axis at once. Hits@1 goes $0.755 \rightarrow 0.830$, F1 $0.659 \rightarrow 0.742$, hedge rate $8.5\% \rightarrow 5.5\%$, tokens $4{,}562 \rightarrow 3{,}159$, and calls $6.2 \rightarrow 4.0$. McNemar gives $p = 5.9 \times 10^{-3}$ on 6 against 21 discordant pairs. The per-stratum breakdown locates the damage precisely. Without the planner, h1 rises $0.75 \rightarrow 0.85$ and h2 rises $0.84 \rightarrow 0.89$. So on this dataset decomposition hurts simple *and* two-hop questions. The planner prompt's first worked example tells the model not to decompose single-hop questions. The ablation proves that instruction fires too rarely, and it proves it with a p-value.

On CWQ the sign reverses, but significance is not reached. Hits@1 goes $0.495 \rightarrow 0.434$, on 27 against 15 discordant pairs, with $p = 0.088$. The stratum table shows a coherent mechanism rather than noise. h1 *improves* ($0.49 \rightarrow 0.54$), h2 collapses ($0.48 \rightarrow 0.34$), and h3plus moves little ($0.62 \rightarrow 0.54$). That is exactly the pattern "decomposition helps genuine composition and hurts questions that do not need it". It does not clear $\alpha = 0.05$ at $n = 198$, so the honest statement carries both halves. *The planner's benefit depends on the stratum and is significant on WebQSP ($p = 0.006$); the CWQ pattern points the same way but does not reach significance at this sample size ($p = 0.088$).*

So the finding is neither "the planner helps" nor "the planner hurts". The planner's value depends on question structure. Applying decomposition to every question regardless is the defect. Section 5.15.1 reads the 21 and 15 questions where removing the planner flipped a failure into a success and names the mechanisms. Section 6.3 turns them into an adaptive gating proposal that this ablation earned rather than motivated.

5.13.2 Without Backtracking

Hits@1 moves -0.010 on WebQSP and -0.005 on CWQ. F1 moves -0.013 and 0.000. Discordant pairs are 5 against 3 and 6 against 5, giving $p = 0.73$ and $p = 1.00$. There is no detectable effect at this sample size.

Two readings are available, and the data does not separate them. Backtracking's contribution may be small because the scorer and the verifier already prevent most of the excursions it exists to correct. Or $n \approx 200$ may simply be too few questions to resolve a small effect. This is reported as "no detectable effect at $n \approx 200$", never as a confirmed null. The mechanism is not idle. The full runs record 63 backtracks on WebQSP and 199 on CWQ, dominated by evaluator-triggered ones (47 and 170). So the condition removes real activity whose accuracy consequence this experiment cannot measure.

5.13.3 Without the Verification Layer

The cleanest null in the table, and the most informative one. Discordant pairs are 0 against 1 on WebQSP and 1 against 0 on CWQ. Across 398 paired questions, removing claim verification changed the correctness of exactly two. Hits@1 moves $+0.005$ and -0.005, and F1 moves $+0.004$ and -0.009. Both $p = 1.00$.

This reproduces the development-set result of Section 5.7.1 exactly. Section 4.10 predicted as much, by construction. On 77 of 80 development questions the layer certified the draft and left it alone; the repair path never ran. A mechanism that mostly passes cannot move an accuracy metric.

The conclusion this licenses is narrower and sharper than "the verifier helps". It is the one Section 5.17.1 adopts. *Claim verification does not measurably change how often AGR is right; it changes how often AGR is right for a reason it can exhibit*. Hits@1 cannot see that distinction. What makes it checkable is the supporting-triple output contract of Section 4.14. The evidence for the layer is that contract and the specimen analyses of Section 5.15.3. It is neither of the two aggregate results it would be natural to reach for instead. Table 5.9 is reached by the agentic baseline as well. That is why Section 5.17.1 attributes it to graph navigation. And the precision column of Table 5.4 does not move when the layer is removed, as the P column of Table 5.11 shows directly. It is not this row either, though this row is reported at full length rather than omitted.

5.13.4 Embedding-Only Scoring

Removing the language-model term from the hybrid scorer costs -0.015 and -0.020 Hits@1 and -0.016 and -0.008 F1. The effect is consistently negative on both datasets and both metrics, and consistently short of significance ($p = 0.66$ and $p = 0.48$).

Table 5.12: Component attribution, RQ3. ΔF1 is *ablated minus reference*, so positive means removing the component improved F1. Only the planner condition reaches significance.

Component	WebQSP ΔF1 (p)	CWQ ΔF1 (p)	Verdict
Planner	$+0.083$ (0.006)	-0.047 (0.088)	Stratum-dependent
Backtracking	-0.013 (0.73)	0.000 (1.00)	No detectable effect
Verification	$+0.004$ (1.00)	-0.009 (1.00)	No accuracy effect
α-blend	-0.016 (0.66)	-0.008 (0.48)	Directional, underpowered

Two corroborating details make the direction credible without over-claiming from it. The condition is cheaper than the reference on both datasets. It uses $3{,}413$ against $4{,}562$ and $4{,}925$ against $7{,}105$ tokens, and 4.4 against 6.2 and 5.9 against 9.1 calls. So the language-model scoring term is buying accuracy with tokens as designed. And it carries by far the most backtracks, 57 and 143 against the reference's 38 and 108, with the low_score trigger that τ governs roughly tripling in both datasets and accounting for about three quarters of the WebQSP increase and about half of CWQ's.

That second detail was originally read as the τ mechanism of Section 4.5.3 visible in the logs. The reading survives only partly. The signal τ thresholds is not computed identically in the two conditions, for the reason Section 4.5.3 records. At $\alpha = 1$ the two auxiliary maxima are never taken. So σ is a maximum over beam-selected, non-banned edges rather than over every candidate offered. After a backtrack has banned the best relation, σ falls in this condition, where it holds steady in the reference. That divergence adds low_score backtracks through an implementation asymmetry, not through the scoring change under test. The two effects push the same way. Separating them would mean re-running the condition with the auxiliary terms restored, and that would no longer be the frozen system.

So the honest statement is narrower than the original one. Removing the model term raises the backtrack count, but an unknown and non-trivial share of that rise is an artefact of how σ is computed when the term is disabled. The backtrack count is corroborating, not load-bearing.

Note which way the defect cuts. Extra backtracks make the embedding-only condition look busier than the scoring change alone would, so the defect flatters the component under test rather than the null. Nothing in Table 5.12 depends on it, since the accuracy and F1 deltas come from answers, not from backtracks. The main evidence for $\alpha = 0.7$ remains the development sweep of Table 5.3, where the $\alpha = 1.0$ condition lost five hits on 80 questions. This ablation corroborates it at test scale rather than independently establishing it.

5.13.5 Paired Significance Testing and Statistical Power

One of four conditions reaches significance. That is the correct outcome to report, not a disappointing one. A correction for multiple comparisons was not among the pre-specified

decisions of Section 5.1.2. Applying one now would be a choice made after seeing the results. So the uncorrected p-values are reported as computed and read as descriptive rather than as a family-wise claim. Appendix E.16 gives the arithmetic behind the second qualification. At $n \approx 200$ McNemar's test needs a discordance pattern this experiment could not have produced for the effects it was looking for. So three conditions are reported as "no detectable effect at this sample size" rather than as no effect. The distinction is the whole content of Section 5.18's entry on underpowered ablations.

5.14 Error Analysis

Section 5.7 measured how often each system fails. This analysis reads the failures and names the mechanism of each one. It is the part of the work no script produced. Its 256 failure packets were read and labelled by hand, one category and one sentence each. Each packet holds the question, the gold answer, the answer given, the plan, the explored relations with their scores, the backtrack reasons, and the verifier outcome.

The reading protocol was fixed before it began. For each packet, find where the trajectory *first* left the correct path, and label that point rather than the last visible symptom. Where two causes compete, take the earlier plausible one and record the other. Without that rule, a run that mis-decomposes, then explores the wrong relation, then hedges would be counted three times under three categories.

A note on the identifiers used throughout this analysis, because they are meant to be looked up. A ComplexWebQuestions identifier is the WebQSP identifier it was derived from followed by a thirty-two character hash. The stem alone does not identify a question. Within these samples one stem covers up to twelve distinct CWQ questions. Thirteen stems are themselves complete WebQSP identifiers naming something else entirely. Cases are therefore cited as stem plus the first eight hash characters. That combination is unique within both samples. Appendix C lists every identifier in full.

5.14.1 Annotation Protocol and the Failure Taxonomy

Ten categories carry the census, each naming a distinct architectural locus rather than a symptom, so that a count against a category points at the component that produced it. Appendix D.11 gives the schema in full, together with the open subtype vocabulary beneath it and the one naming decision that was got wrong and corrected. The subtype vocabulary drives nothing. It exists so that a mechanism can be named before it is understood.

Population, Not Sample The census is a *population*, not a sample. Every question the full system did not answer correctly was read and labelled. Three sets are set aside before it is taken,

Table 5.13: The merged failure histogram, $n = 256$. Wrong and hedge are kept separate. Counts come from scripts/synthesize_census.py over the three label sources of Table D.4. The table body is generated from results/phase4/thesis_numbers.json.

	WebQSP		CWQ		
Category	**wrong**	**hedge**	**wrong**	**hedge**	**Total**
relation_selection	19	6	20	19	**64**
composite_claim	1	—	23	23	**47**
kg_gap	8	4	7	24	**43**
decomposition_error	15	11	10	2	**38**
answer_selection	1	1	8	5	**15**
echo	5	2	4	2	**13**
ambiguous_question	1	1	3	5	**10**
premature_termination	1	4	1	2	**8**
gold_noise	1	—	2	3	**6**
other	—	1	—	5	**6**
verifier_fn	—	3	1	1	**5**
verifier_fp	—	—	—	1	**1**
Total	**52**	**33**	**79**	**92**	**256**

of which only the adjudicated benchmark defects of Section 5.2.4 remove questions from the analysis altogether. The other two are of questions that were never failures. Appendix D.8 reconciles the three, which overlap, and shows how the 256 read failures follow. That the census is complete rather than sampled is what licenses the category counts below being read as counts rather than as estimates.

5.14.2 Distribution of Failure Categories Across Datasets

The Shape Flip, and What It Is Not The two datasets fail differently, but not in the way a first reading of Table 5.13 suggests. relation_selection sits at or next to the top on *both*. It has 25 of WebQSP's 85, one behind decomposition_error, and 39 of CWQ's 171, behind composite_claim alone. So it is not what distinguishes them. Picking the wrong edge is the base rate of graph navigation, independent of the dataset.

What distinguishes them is a pair of categories that are nearly absent from one and dominant in the other. composite_claim is 1 of 85 on WebQSP and 46 of 171 on CWQ, and kg_gap is 12 and 31. In the other direction, decomposition_error is 26 on WebQSP against 12 on CWQ. On WebQSP that is the near-tie noted above, the two together accounting for 60% of its failures.

That asymmetry is demonstrable rather than assertable, from two facts already established. The first is the stratum distribution of Table 5.1. WebQSP is 64% one-hop with a four-question deep tail. So most of its questions have no composition to get wrong and no constraint to drop, while presenting the planner with exactly the questions that do not need decomposing. That mismatch is also the significant ablation result of Section 5.13, appearing here as a category

count. The second is the expressiveness limits certified in Section 3.3.3. The environment carries no date literals, no numeric literals, and no ordinals. CWQ's SPARQL templates invoke all three constantly, while WebQSP's naturally-occurring questions rarely do. The failure profile of a system on a dataset is, to this extent, a property of the dataset's construction.

Hedging Where It Should A second reading of Table 5.13 is worth stating as a positive result rather than only as a deficit. On CWQ, kg_gap skews three-to-one toward hedges, 24 against 8, while on WebQSP it leans the other way, 4 against 8. When CWQ's numeric identifiers, date literals, and ordinal constraints are unreachable, the system much more often declines than asserts something wrong. That is the behaviour the anti-vacuity and grounding provisions of Section 4.11 were written to produce. It is visible in the one place where the environment guarantees the system cannot succeed.

5.15 Failures in Planning, Navigation, and Verification

The categories in this section each sit inside one component of the agent. The components are the planner and the drafter, the relation scorer, and the verifier. The three subsections read them in that order, and a count against any of them points at the code that produced it.

5.15.1 Decomposition, Drafting, and Answer-Selection Errors

Three families share this section because they share a locus. That locus is everything downstream of retrieval and upstream of the answer. By the time these failures happen, the reasoning is often already complete and correct. That timing makes them diagnostic, and it makes them cheap to fix.

The Entity-Extraction Bug Nine of the 38 decomposition_error cases carry the subtype extraction_bug, and they are the most actionable finding in the census. On profession-shaped and "what did X do" questions, the evaluator resolves every correct gold value and the drafted text states them word for word. Then answer_entities collapses to the grammatical *subject* of the sentence. Table 5.14 gives three instances read from the raw traces.

Table 5.14 understates how complete the drafts are. For WebQTest-1215, "who was stephen r covey", gold lists Author, Manager, Writer, Consultant, Professor, and Motivational speaker. The drafted answer reads *"Stephen Covey was a Writer, Motivational speaker, Author, Consultant, Professor, and Manager"*. It has all six, correct, in a different order. answer_entities comes back as ['Stephen Covey']. Navigation found the answers, the verifier certified them, and the prose reports them. The step that fails is the one that turns answer text into an entity list, and it reaches for the subject rather than the predicate values every time.

Table 5.14: The entity-extraction bug. In each case the drafted answer text names every gold value correctly, and the scored entity list holds only the sentence's grammatical subject.

Question ID	Gold	Named in text	Scored entities
WebQTest-1215	6 professions	all 6	[Stephen Covey]
WebQTest-704	6 professions	all 6	[Thor Heyerdahl]
WebQTrn-124_0782789f	4 films	all 4	[Angelina Jolie]

The consequence is a limitation on Section 5.7, not only a bug. Scoring reads answer_entities, so Table 5.4 records every one of these cases as a wrong answer or a hedge. On "list the professions" and "list what X did" question shapes, an extraction defect that has nothing to do with reasoning quality pulls the reported accuracy down. Nine cases in 256 cannot move the headline numbers much. But the measured figure for that question shape is a floor, and it should be read as one. Section 6.3 lists the fix among the repairs the census earned.

Right Candidate Retrieved, Wrong One Drafted answer_selection (15 cases, 13 of them on CWQ) is the extraction bug's more serious sibling. Here the exact gold value demonstrably entered the resolved candidate set, and the drafter chose something else. WebQTest-1555 asks what the parliament of Nepal is called. The evaluator resolves Parliament of Nepal itself; the draft names its two component chambers instead. WebQTrn-2784_abedc8ac asks which Tupac Shakur film Malcolm Campbell edited. The trace records resolved=['Nothing but Trouble'], the literal gold value, which satisfies both constraints. The draft substitutes Gang Related and then hedges on it.

One case in this family was first filed as a verifier error, and the correction is worth giving in full because it is the kind of mistake this taxonomy exists to prevent. WebQTrn-1294_a4b2006a was labelled verifier_fn on the theory that the verifier had checked a claim about a different candidate than the one the evaluator resolved. The run record refutes that theory. The draft itself reads "Marlon Brando also played in *Joy*", and the verifier rejected exactly what the drafter asserted. Brando was not in *Joy*, but De Niro was, and the evaluator had resolved him. **The verifier was correct.** The error sits upstream, in the answerer, and the label now reads answer_selection. The case is also the clearest example in the census of the verification layer doing its job. Section 5.15.3 opens by pointing back to it.

Failing to Commit: Six Cases of Hedging on a Verified Fact The other category was left open for mechanisms the schema did not anticipate. All six of its cases turned out to share one. Nothing predicted it, which is why the schema lacks it. *The evaluator resolves the exact gold value, the verifier returns* grounded, *and the drafted text hedges anyway*. No rejection is involved, so this is not a verifier error at all.

WebQTest-689 states "Spanish Language was spoken in Spain" and then, in the same answer, that

the first language spoken in Spain could not be determined. WebQTrn-125_003c6a04 resolves Memphis, matching gold exactly, and the draft says the location could not be determined. WebQTrn-1392_d372995c names both of Eleanor Roosevelt's schools correctly and then declares the school undeterminable. WebQTest-989_4b6636a0 (Battle of Dunkirk), WebQTest-1923_a64ef0f5 (*The Last Song*), and WebQTrn-2721_2d207ed9 (Alois Hitler) have the same shape. In every case the entity list comes back empty and the question scores as a hedge.

The contrast with the extraction bug is exact and worth holding on to. There, the text is right and the entity list is wrong. Here, the text contradicts a fact it has just stated. Six instances out of six share the mechanism, and that uniformity makes it a defect rather than a distribution.

Context-Stripping: When Decomposition Discards the Question Subtyping is thinner in this category than elsewhere, and the shortfall belongs on the record before the cases are read. decomposition_error holds 38 failures. The four named subtypes, extraction_bug (9), paraphrase_drift (9), context_stripping (3), and over_decomposition (1), cover 22 of them. The other 16 carry a category label only. So the mechanisms below are the mechanisms of the 22, not of all 38. The subtype vocabulary stayed open rather than being stretched to cover the residue.

paraphrase_drift and over_decomposition are the expected decomposition failures. In the first, a rewritten sub-objective scores worse against the target relation than the raw question text. In the second, a redundant step resolves an already-unambiguous entity. WebQTest-1829 is the clean over-decomposition case. The plan splits "what type of religion did massachusetts have" into ['find Massachusetts', 'find the religion type associated with #1'], burns 14 of 16 calls on three evaluator backtracks, and hedges, while the undecomposed run finds the same fact in three calls with no backtracks.

context_stripping (3 cases) is the mechanism this analysis would feature if it could feature only one. It is a failure of decomposition that decomposition's own design cannot see. Splitting a question into steps *removes* a qualifier that appears only in a later clause. So an early sub-objective resolves the wrong sense of an ambiguous term before the disambiguating information ever arrives. In WebQTest-1367, which asks where Glastonbury, England is, the plan's first step anchors on 'find Glastonbury' alone. The explorer resolves toward the identically-named Connecticut town. In WebQTrn-2615_48a6ac56, which asks what instruments the author of *Fela!* plays, the first sub-objective is scored with no mention of instruments and resolves the wrong sense of an ambiguous authorship relation, where the undecomposed run answers all five instruments correctly in fewer calls.

WebQTrn-2570_d63877a4 is the strongest specimen in the census. Its trace shows every link of the mechanism. The question, "This 33rd president was the nation's leader during WW2", is a single entity under two descriptions. The planner emits ['find the 33rd president', "find the nation's leader during WW2"], with no #1 reference between them, modelling two descriptors of one entity as though they were a chain. Because sub-objective one is the active scoring context,

the anchor World War II scores 0.128–0.133, which is the noise floor, while presidency-hub relations score 0.46–0.54. Decomposition actively *suppressed* the one anchor that could have discriminated. Objective one then never completes (objective_done: false at every round). So objective two never activates. The WW2 constraint is never used at any point in the search. The agent spends its depth budget wandering around the presidency hub. It then drafts a plausible-sounding president. The draft names Woodrow Wilson, the 28th president and nowhere near the war. The undecomposed run answers Harry S. Truman correctly in three calls. "WW2" stayed in the scoring context. Two further observations belong to that trace. No ordinal relation appears anywhere in it. So the "33rd" constraint played no part either, consistent with the environment's lack of ordinal encoding (Section 5.16.1). And the verifier returned grounded on the Wilson claim, which Section 5.15.3 takes up as a boundary case rather than a defect.

Three instances of one mechanism, two of them surfaced by the ablation contrast and one by ordinary reading, is what makes context-stripping a named finding rather than an anecdote. It is the concrete evidence behind the adaptive-gating proposal of Section 6.3. A fourth case belongs beside them and is filed as plain decomposition_error, since no subtype fits it. It is WebQTest-869, "where is ancient phoenician", whose planner rewrote the sole sub-objective as ['find ancient Phoenician']. So the explorer re-identified the topic instead of locating it. The run answered "Phoenicia", the question's own subject, grounded, and useless. An undecomposed run answered "Ancient Phoenicia was in Lebanon" in three calls. What separates it from the three above is *what* the rewrite discarded. They drop a constraint arriving in a later clause. This one drops the interrogative, the word that says what is being asked for. The result lands in the graph as an echo (Section 5.16.2). With the ask removed, the nearest well-grounded entity to a topic is the topic. Both losses are visible to a planner that compares the rewrite against the original question, which is why the gating proposal checks the rewrite for the interrogative as well as for redundancy.

5.15.2 Relation-Selection and Navigation Errors

relation_selection is the largest category at 64 and carries no subtypes, because the mechanism is uniform. Either a wrong relation was explored, or the right one was explored and abandoned. Its interest is not in the individual cases but in the fact that several of them are the *same* gap seen from different angles. One relation confusion is reproduced across three unrelated questions. Most sharply, a CWQ triplet reaches the same entity by three different routes, none of which ever tries the single relation that answers all three. Three independent entry points into the same resolver hitting the identical dead end is the cleanest evidence in the corpus that a relation gap is *systemic* rather than an artefact of one question's phrasing. It is a property of the scorer's ranking over that entity's relation set, not of the question.

premature_termination is small (8 cases) and splits into two mechanisms that must not be merged.

Five carry the evaluator subtype and are the *same* systemic pattern. The correct relation is explored with a solid score, as high as 0.731, and the evaluator backtracks away from it and never returns, across entirely unrelated domains. A relation scoring 0.73 being explored and discarded is not a reasoning gap. It reads as a threshold or backtracking-policy defect. It is one of the most directly fixable findings in the census. The other three carry budget and are genuine exhaustion. Appendix D.7 gives all thirteen with their scores.

5.15.3 Verifier Rejection and Acceptance Errors

Before the errors come the correct rejections, and Section 5.15.1 narrates one in full. The verifier rejected a claim the drafter had fabricated about the wrong actor. The layer worked exactly as designed. The 52 questions on which the verifier fired unsupported are not a population of mistakes.

The two error polarities are, however, very unevenly evidenced, with five cases against one. That imbalance is not a fact about the verifier but about the instrumentation. It is the finding of this section.

Defining the Error Polarities The verifier's positive class is "the claim is supported". A *false positive* is therefore a claim wrongly **accepted**, and a *false negative* a claim wrongly **rejected**. Because the verifier's own outputs are labelled supported and unsupported, "positive" has no stable referent for a reader. The rest of this thesis uses the plain-English terms.

Wrongly Rejected, and Wrongly Accepted Five cases are wrongly rejected, all sharing one mechanism and all carrying the subtype structural_not_semantic. In each, the claim is true and the answer correct. But no single triple states it in the phrasing the drafter used. So a transitively-true or differently-anchored fact is rejected, and the retry loses the answer. Three of the five had already resolved the exact gold answer before the rejection forced the retry. This is the limitation Section 4.15 describes by construction, now seen at census scale. Section 6.3 names the semantic-level check that would address it.

Exactly one clean specimen of wrongful acceptance exists in the whole census. WebQTrn-1597_8adc06ba asserts a Seth MacFarlane connection to *ThunderCats*. The verifier returns grounded. A direct Cypher query confirms that no such edge exists anywhere in the environment. Its instructive companion looks like a second defect and is not one. WebQTest-1620 asks when President Wilson was in office. The environment exposes inauguration events but no date literal. So the system answers with two inauguration mentions whose entities exist and whose edges are real. The structural check passed *correctly* while the answer is wrong. That contrast is the most useful point in this section. Structural grounding is a claim about edge existence, not about answer adequacy. So a verifier that checks whether a claim is *in* the graph cannot, by

construction, check whether it is *the answer*. Table 5.9's Tier-2 column measures that gap. Appendix D.6 gives all seven cases with their traces.

A Rate for One Polarity, and a Blank for the Other Three quantities are easy to conflate, and only one is the denominator this polarity needs. The verifier is invoked 828 times over the 800 questions, since a question that repairs is verified again afterwards. Its *first* verdict is unsupported on 52 questions, 16 of 400 on WebQSP and 36 of 400 on CWQ. Of those, 13 end grounded after a repair, leaving 761 questions answered under a grounded verdict and 39 under an unsupported one. The five wrongly-rejected cases are drawn from the 52. So that polarity has a denominator in the question unit. Roughly one rejected question in ten cost a correct answer. Even that is a lower bound, since a wrongful rejection that did not change the final answer never entered a census that reads only failures. The units matter because the run record stores the *first* verifier entry. So a question repaired into groundedness is filed under unsupported in the committed verifier_outcome field. Reading that field as the outcome would put 13 questions on the wrong side of the ledger. Treating the 52 as firings would credit the verifier with 52 of 828 rather than the 52 of 800 the rate is computed against.

Wrongful acceptance has *no logged population at all*. Accepted claims are never persisted (Section 4.8). So the wrongly-accepted cases sit undifferentiated inside the 761 questions answered under a grounded verdict, with nothing marking them. The single specimen surfaced only because the ablation-discordance reading happened to walk that trajectory by hand. What can be given is the size of the space they hide in, since a blank invites the reading that the space is small. The verifier accepted 2,008 of the 2,110 claims it decomposed on test. Neither structural route matches on the relation or the direction. So every acceptance that did not come from the entailment check was made by a relation-blind test. Section 4.12.1 shows the record cannot say how many that is beyond the interval $[39, 2{,}008]$. The two polarities are therefore reported asymmetrically and deliberately. Rejection gets a rate and acceptance an explicit blank, and the two are not averaged, since a combined "verifier error rate" computed from these numbers would be an artefact of what the logs happen to record. Section 5.18 treats this as the instrumentation gap it is, and Section 6.3 gives the one-line fix.

5.16 Graph Gaps, the Echo Attractor, and Benchmark Defects

The categories in this section belong to no single component. In the first, the environment cannot express what the question asks. In the second, the graph offers a true fact one node away from the answer, and the system takes it. In the third, the benchmark's own label is wrong or its question is underdetermined.

5.16.1 Knowledge-Graph Gaps: Literals, Temporal Qualifiers, and Ordinals

kg_gap is the third-largest category at 43 and CWQ's single largest hedge category at 24. It invokes the unanswerable-in-environment class defined in Section 3.3.3. Its five subtypes cluster by mechanism. Those five account for 39 of the 43; Appendix D.9 gives them with their specimens. The remaining 4 carry no subtype, because the environment gap in each was specific enough that grouping it with anything else would have been an invention. They stay in the category total throughout rather than being dropped silently.

One subtype differs in kind from the other four, and the difference matters for what follows from them. data_error is a gap in this particular import. ordinal, numeric_literal, date_literal, and temporal_qualifier are gaps in what a triple-and-traversal architecture can express at all. No larger graph fixes them; they need an operator. That is why Section 6.3 lists literal and ordinal support with the architectural items rather than with the repairs.

5.16.2 The Echo Attractor: Answering with the Topic or an Intermediate Entity

echo names a mechanism that recurs independently of the two mechanical categories above. The system resolves a real, well-grounded entity that sits *near* the correct answer in the graph. The verifier passes it, because it genuinely is grounded. The fact is true but not what was asked.

Three shapes appear, and the boundary between them is not always crisp. topic (6 cases) is confusion between entities that share a name or a tight cluster. WebQTest-1707 asks who plays Lex Luthor on *Smallville* and answers "John Glover plays Lionel Luthor", a different character in the same fictional cluster. Lionel, Alexander, and Lucas Luthor variants all surface in the trace before the run gives up. granularity (7 cases) is the right entity family at the wrong level. WebQTest-86 answers "Kingdom of Denmark" where gold is "Denmark". The two are genuinely distinct nodes, the formal sovereign-state entity and the common-name one. WebQTrn-1405_ced20d84 asks which school founded in 1636 President Kennedy attended, has Harvard College in its very first candidate list, and narrows to Harvard University instead. The verifier then correctly rejects its founding-date claim. intermediate, which stops at a hop along the way rather than continuing to the target, has no rows of its own in this census. The reading pass filed those cases under granularity, the two being best treated as one family.

The throughline is what makes this a named mechanism rather than a residue category. None of these are cases where the system knew nothing. Every one resolves a real, verifiably-true fact through a real edge. The failure is entirely in *which* true fact gets surfaced. That is architecturally different from picking a wrong edge and from there being no edge at all. The right edge exists, is used, and lands one node away.

That is also why the echo attractor is invisible to any evaluation treating systems as independent. Section 5.16.3 shows all five systems of the main matrix falling into it together, on the same questions.

5.16.3 Benchmark Defects: Gold Noise and Ambiguous Questions

Two numbers must be kept apart. The consensus pass of Section 5.2.4 *flagged* 58 WebQSP and 47 CWQ questions where at least three of the five systems agreed on the same non-gold answer. Adjudication confirmed 22 and 19 as genuine defects. The gap between those figures is not measurement noise. It is the echo attractor of Section 5.16.2, operating across systems.

Read what the flagged consensus answers have in common. Five systems answer "rick scott" where gold lists his professions, "thor heyerdahl" where gold lists his occupations, and "amelia earhart" where gold is Pilot and Writer. Each returns the topic entity as the answer. Five answer "belgium" where gold is the containing region, "dominican republic" where gold is Greater Antilles, and "san francisco giants" where gold is the 2014 World Series. Each is the composition's intermediate hop rather than its final one. Five answer "maryland" for counties, "nevada" for Las Vegas, and "united arab emirates" for Dubai. Each is one level too coarse.

When systems agree with one another against gold, the usual cause is a shared attractor, not a label error. That is a finding about evaluation methodology as much as it is about these systems. A policy of rescoring whenever three systems agree is a natural thing to want. It would have silently converted this system's most characteristic failure mode into apparent correctness. The manual adjudication step is what stands between the two. It is the reason the pass is reported with both numbers rather than one.

The Confirmed Defects Appendix D.5 reads the confirmed defects case by case. They are the twelve WebQSP and seventeen CWQ questions whose gold is wrong and the ten and two whose questions are underdetermined, and the appendix says where each came from. Two patterns account for most of the CWQ set, both properties of template generation rather than of annotation care. One is a gold list flattened from a multi-valued relation. The other is an annotation query that traversed one relation too far.

5.17 Discussion

5.17.1 Findings Against RQ1, RQ2, and RQ3

RQ1: Does agentic navigation improve multi-hop factual accuracy?

Yes, and the multi-hop qualifier is doing real work. AGR leads every accuracy cell of Table 5.4. All eight paired comparisons reject, including both against the agentic baseline running on the

same graph with the same model under the same budget. The stratum profile is the substance of the answer. On CWQ, AGR's accuracy *rises* with hop count ($0.46 \rightarrow 0.55 \rightarrow 0.57$), the only system that ends above where it started, while Vector-RAG collapses from 0.33 to 0.10 on a context that cannot hold a chain at any radius.

Three boundaries are reported with it. Think-on-Graph hedges marginally less on CWQ, 22.5% against 23.0%. That is the one cell of Table 5.4 AGR does not lead, and the reason the claim is about the accuracy columns rather than the whole table. It is also better on CWQ's one-hop stratum, as Vector-RAG is on WebQSP's. The margin over it is located in the budget. Where the shared cap lets it finish, it is marginally ahead on both datasets. So the claim defended here is that guided search completes under a realistic budget where beam search does not (Section 5.8). And that comparison is against a systematically narrower baseline, since Think-on-Graph prunes to the published algorithm's 40 relations and 20 neighbours against AGR's 300 and 200, binding on roughly a third of its expansions and almost none of AGR's (Section 5.4.3). The margin is a lower bound on what an equal-width reimplementation would show. The static graph baseline's per-stratum decay is not offered as evidence here. Its retrieval radius confounds it (Section 5.4.2).

RQ2: What does pre-generation verification contribute beyond navigation?

The answer has three parts that a yes-or-no question would have merged, which is why it was not asked as one.

First, navigation eliminates structural hallucination. The verifier is not what does it. Both navigating systems assert zero ungrounded entities on WebQSP, over $1{,}235$ assertions for AGR and 832 for Think-on-Graph. Parametric answering on the same set leaves 27.1% ungrounded. Any system that copies its answer entities out of triples it traversed inherits this. So attributing it to the verification layer would be wrong. Table 5.9 is the evidence that it would be wrong.

Second, the precision advantage belongs to the architecture. This experiment cannot award any of it to the layer. AGR's assertion precision exceeds Think-on-Graph's by 11.8 points on CWQ (67.9% against 56.1%) and its per-question precision by 0.13 and 0.08, at a lower hedge rate on WebQSP. So it is not bought with abstention. That margin is real, but it is not evidence for the verification layer. The one condition that isolates the layer does not reproduce it. Removing the verifier moves per-question precision by $+0.001$ on WebQSP and -0.010 on CWQ (Table 5.11), a sign flip an order of magnitude below the gap it would have to account for. The mechanism cannot carry the attribution either. Entity filtering runs only on the give-up path (Section 4.13.4), which 10 of 400 WebQSP and 29 of 400 CWQ questions entered. A filter touching 2.5% of a test set cannot produce a 0.13 precision margin on it. Two differences the ablation holds fixed remain live as explanations, and neither is the claim check. One is the groundedness filter on evaluator resolutions (Section 4.5.2), which is navigation rather than verification and runs in every ablation condition. The other is the drafting prompt's anti-vacuity and grounding provisions, which the baselines' plain prompt lacks (Section 5.4.4). Some share of the precision margin over the agentic baseline may be prompt rather than structure. This work cannot say how large that share

is.

Third, verification shows no measurable Hits@1 effect. Removing it changes two questions out of 398. That is not a failure of the layer but a statement about what Hits@1 measures. A metric satisfied by one correct entity among several cannot see a mechanism that works on the others. What the layer offers is an answer that arrives with the triples supporting it (Section 4.14). That is an auditability claim, not an accuracy claim. The honest formulation is that *claim verification does not change how often the system is right; it changes how often the system is right for a reason it can show*.

RQ3: Which components contribute what, at what token cost?

One condition of four reaches significance. The planner's effect depends on the stratum. Removing it improves WebQSP by 0.083 F1 at $p = 0.006$, saving 31% of tokens and 35% of calls, and costs CWQ 0.047 F1 at $p = 0.088$. That asymmetry, not a pooled average, is the result. It says how decomposition should be *applied*, not whether it works. Backtracking, verification, and the α-blend show no detectable effect at $n \approx 200$, each for a different reason, and Section 5.13 separates the reasons out. On cost, the whole apparatus runs at half the language-model calls of the agentic baseline and never exhausts the call budget that clips the baseline on 44% of CWQ questions.

The three answers do not point the same way, and the chapter does not force them to. Navigation is where the accuracy comes from; verification is where the auditability comes from, the precision margin belonging to the architecture as a whole rather than to any one component this ablation could isolate; and decomposition, the component the literature treats as straightforwardly beneficial, is the one component this work can prove is sometimes harmful.

5.17.2 What Agency Buys, and What It Costs

Agency buys the deep tail. On CWQ the accuracy of the agentic system *rises* across hop strata while every static system decays. It buys that at roughly an order of magnitude in tokens over one-shot retrieval. Against the other agentic system, it buys the same accuracy for half the language-model calls, never exhausting the call budget that clips the baseline on 44% of multi-hop questions. That is the difference between a guided search and a blind one, priced. What it costs, beyond tokens, is a longer failure surface. Every mechanism in this census except kg_gap and the benchmark defects is a failure the static baselines cannot have. They have no plan to mis-decompose, no frontier to abandon, and no draft to check. decomposition_error at 38 cases is the clearest instance. A component added to help contributed a failure family of its own. Section 5.13 shows it is a net negative on the shallower benchmark. Agency converts a retrieval problem into a control problem. Control problems fail in more ways.

5.17.3 When Verification Helps and When It Over-Hedges

The verification layer's contribution is auditability, and its cost is recall. Both directions show up in this census at case level, and case level is where they are established, because the aggregate effect of removing the layer is indistinguishable from zero (Section 5.17.1). The layer helps when a draft asserts something the traversal never supported. Two cases show it. One is the De Niro case of Section 5.15.1, where the verifier rejected a fabricated actor before it could be emitted. The other is the Vicksburg-president case of Section 5.16.3, where declining to assert was the honest thing to do and the metric scored it as a loss. The layer over-hedges when a true claim has no single edge stating it in the drafted phrasing. The five wrongly-rejected cases of Section 5.15.3 show that. Three of them had already resolved the exact gold answer before the rejection forced a retry that lost it. It is the same trade the development set measured in Section 5.7.1, and the same one the ablation could not detect at $n \approx 200$. So the claim is narrow. Verification does not make this system more accurate. It makes the answers it does give attributable, at a small and measurable cost in recall. It provides no protection at all against the failure mode this census finds most often. That is an answer that is grounded, connected, true, and not what was asked.

5.18 Threats to Validity

Eleven threats follow. Each is stated against the measurement it bears on, with the mitigation where one exists.

The environment is friendlier than the knowledge base it stands in for. This bounds every absolute number. It is the union of per-question subgraphs the RoG distribution pre-extracted *so as to contain their own question's answer* (Section 3.1.3). Taking the union restores distractor mass but cannot restore what was never extracted. That gap is why answer reachability is certified at 97.3% and 99.7% (Section 3.3) where a BFS extraction from full Freebase under the same hop bound would not reach those rates. Accuracies measured here are therefore *not* comparable to evaluations over full Freebase. The ceilings should be read as part of the result. The mitigation, real but narrow, is that every system navigates the identical environment. So the *relative* ordering is sound even where the absolute level is optimistic. Every claim in Section 5.17.1 is relative for this reason.

An identical environment is not identical access to it. One baseline sees less of it. The agentic baseline truncates candidate sets at 40 relations and 20 neighbours against this system's 300 and 200. Those are the reimplemented algorithm's own pruning widths. But the narrower cut binds on about a third of its expansions against effectively none of this system's (Section 5.4.3). The comparison holds the graph constant and does not hold the view of it constant.

Every reported number comes from one run per condition. Run-to-run variance is

unmeasured. The confidence intervals and paired tests resample questions. So they bound the sampling variance of the question draw and nothing else. Each system's answer set enters them as fixed. The backbone is not deterministic at the trajectory level even at temperature 0.0. Section 5.3 measures the frozen model at 8 of 12 probes, about one trajectory in three diverging on a rerun. A divergent trajectory can change an answer, a hedge, a token count and a call count at once. That figure is itself only partly auditable. Only the first of four rounds survives as a committed artifact. A reader who declines to take the other three on trust should therefore read the stability figure as unquantified rather than as 67%. That substitution makes this threat larger, not smaller. Two things make a rank reversal from decoding noise alone unlikely. The margins defended here are large relative to their intervals, and the paired tests are computed per question. But unlikely is the strongest statement available. The experiment that would bound the variance was not run, and at $11.13 against a $15–20 ceiling it was affordable. So this is a gap in the design, not a budget constraint. A repeated matrix reporting across-run spread is the cheapest correction (Section 6.3).

Entities are keyed by name, so homonyms are merged. Distinct real-world entities that share a surface form collapse into one node. Two consequences follow, in opposite directions, and neither is measured here. A merged node can manufacture a path between things that are not connected, which inflates reachability and lets a wrong answer verify as grounded. The same merge can destroy a distinction the question depends on. Name-keying carries this inherently; it is not a defect of the construction. But it means structural groundedness is a claim about *this* graph's node identity, not about the world.

The instrumentation gap is the most serious of the remaining items. The verifier logs only the claims it rejects, never the ones it accepts or the triples that matched them. So wrongful acceptance does not diagnose itself. The single specimen in this census surfaced only through manual trace reconstruction, and nothing says how many others sit inside the 761 grounded questions. One polarity is reported at census scale and the other anecdotally. That asymmetry comes from the logging decision, not from the verifier. Section 6.3 gives the fix.

The judge fell short of its pre-specified bar. Cohen's κ came to 0.6995 against a threshold of 0.7 (Section 5.5.5). By the conventional reading that is substantial agreement, and the miss is 0.0005. The Tier-2 numbers are still reported as corroborating rather than decisive. The judge also runs on the same backbone as the systems it judges.

Topic entities are given, not linked. Both graph-navigating systems seed from the dataset's annotated topic entities. That isolates navigation from entity linking, as intended. But the reported accuracies assume a linking step that a deployed system would have to perform itself, with errors of its own.

Ablations are underpowered. Three of four conditions detect nothing at $n \approx 200$. "No detectable effect" is not "no effect" (Section 5.13).

One relabelling happened after the result it affects was known. A development question first

classified as mediator-heavy was reclassified as unanswerable-in-environment once its outcome was in. Section 4.10 flags it in place, and the aggregates were not rescored.

The reported accuracy has an unmeasured floor. The extraction bug of Section 5.15.1 turns correct reasoning into a wrong entity list on one specific question shape, so the measured accuracy on that shape is a lower bound rather than an estimate.

The author adjudicated the benchmark defects. The exclusion set of Section 5.2.4 rests on single-annotator judgements, made with the consensus signal visible, and that section records the scope limits. The accuracy tables are not rescored, since the defects run in both directions.

Chapter 6

Conclusion

6.1 Summary of Contributions

This thesis asked what an agent gains by navigating a knowledge graph rather than retrieving from it, and what a claim-level check against the traversed subgraph adds on top. The answers are not uniform, and the value of the work lies partly in where they diverge.

Section 5.7 reported the measurements. AGR leads every accuracy cell of the main matrix. Every one of the eight paired comparisons rejects, including both against a reimplementation of the agentic baseline on the same graph, with the same model and budget. Where the shared call cap lets that baseline finish, the baseline is slightly ahead. The whole margin comes from the 29% and 44% of questions where the cap truncates it (Section 5.8). The finding is therefore about affordability rather than resolution quality. AGR runs at half the language-model calls of the agentic baseline and never exhausts the cap. On ComplexWebQuestions it is the only system whose accuracy *rises* with hop count, and every other system ends below where it started (Section 5.10). Structural hallucination is eliminated. Over both datasets, none of $1{,}709$ asserted entities is ungrounded, against 22.1% of the $1{,}001$ the parametric control asserts.

Section 5.14 read all 256 remaining failures and named each mechanism. Three findings generalise beyond this system. The first is *context-stripping*. Decomposing a question can discard a qualifier that appears only in a later clause. An early sub-objective then resolves the wrong sense of an ambiguous term before the disambiguating information ever arrives.

The second is *the echo attractor*. The characteristic failure of a graph-navigating system is not to invent an entity. It is to surface a real, well-grounded, verifiably true entity that sits one node away from the answer. The verifier passes it, because it genuinely is grounded.

The third is that *shared attractors break consensus-based evaluation*. Five systems sharing one graph and one backbone landed on the same wrong answer often enough that rescoring on majority agreement would have turned this failure mode into apparent correctness.

Section 1.6 claimed six contributions. The echo attractor is the second finding above, and the

pre-specified evaluation protocol is reported in Section 5.1.2, including the threshold it missed. The other four follow.

The framework and its verification layer. An agentic KGQA system that returns its answer paired with the triples supporting it, built so that the supporting evidence is a byproduct of the control flow rather than a post-hoc reconstruction.

A component-level ablation. Four single-deviation conditions with paired significance testing, closing the attribution gap of Section 2.5.1 for this architecture. It finds that only one of the four components has a detectable accuracy effect.

Stratum-dependent decomposition. The literature treats decomposition as straightforwardly beneficial, but it is not. Removing the planner *improves* WebQSP accuracy by 0.083 F1 at $p = 0.006$ while cutting tokens by 31%, and trends toward harming CWQ. The asymmetry, not the pooled average, is the result. It says that decomposition should be gated on question structure rather than applied unconditionally.

Quantified benchmark defect rates. Fifty-seven questions across the two test samples where the benchmark, not the system, needed correcting. They come with the two evidence signatures that distinguish a world-fact error from an annotation-pipeline error, and the provenance argument that explains where one class of them comes from.

6.2 Limitations

Section 5.18 states the threats to validity in full, against the measurements they bear on. Four of them constrain what may be concluded here more than the rest, and are repeated for that reason.

Wrongful acceptance is unmeasured. The verifier logs the claims it rejects and not the ones it accepts. So one polarity of its error is reported at census scale. The other is reported from a single manually reconstructed specimen. The exposed population is bounded only as $[39, 2{,}008]$ of $2{,}008$ accepted claims (Section 4.15). Nothing measured here narrows it.

Structural grounding is not answer adequacy. Both navigating systems assert zero ungrounded entities on WebQSP. That says the answers are made of the graph's own material. It does not say they answer the question. The echo attractor (Section 5.16.2) is the counter-example. It is not the most frequent failure in this census but the one no grounding check can catch.

The evaluation is single-environment, single-backbone, and single-run. Absolute levels are bounded by an environment built from answer-containing subgraphs. Run-to-run variance is unmeasured against a backbone that repeats a trajectory on about two probes in three.

The baseline comparisons are bounded in the baselines' favour, and against them. The agentic baseline prunes from a narrower candidate set than AGR. The static one retrieves a single logical hop with an unordered fanout cap. So the reported margins are lower bounds on

what equal-width, equal-radius reimplementations would show (Section 6.3.5). Entity linking is assumed throughout, topic entities being given by the benchmarks.

6.3 Future Work

The census makes it possible to separate proposals backed by a counted, repeated pattern from proposals backed by an argument. Both are listed, but they are not the same kind of claim.

6.3.1 Repairs the Census Earned

Three defects each recur with a uniform mechanism across unrelated questions. Each is small. The first is *entity extraction*, with nine instances. The step converting drafted answer text into the scored entity list defaults to the sentence's grammatical subject rather than its predicate values. The fix is to tell the claim decomposer which argument of the drafted sentence is the answer. That is a prompt instruction, not an architectural change.

The second is *evaluator abandonment*, with five instances. The explorer visits a relation scoring as high as 0.73, backtracks away from it, and never returns, across four unrelated domains. That pattern looks like a threshold or backtracking-policy defect. Logging the evaluator's decision beside the score of the relation it discards would diagnose it.

The third is *drafting commitment*, with six instances, all six of the other category. When the evaluator has resolved the exact gold value and the verifier has returned grounded, the drafter should commit to it rather than stating the fact and declaring it undeterminable in the same sentence.

6.3.2 Logging Accepted Claims to Make Wrongful Acceptance Measurable

Persist each accepted claim together with the triples that matched it. This converts the wrongly-accepted class from an anecdote into a rate with a denominator. It also makes grounded-but-wrong answers self-diagnosing from the logs rather than by manual trace reconstruction. The same change closes a second gap. The run record currently stores the *number* of supporting triples rather than the triples (Section 4.14). So the output contract holds inside the run. It cannot be checked afterwards from any committed file. As a logging change, it is the highest-value change in this list relative to its cost.

6.3.3 Operators the Failure Categories Specify

Three additions are specified by categories rather than motivated by them. The first is *adaptive decomposition gating*. A decomposition step should be gated on whether the rewritten sub-objective preserves the question's actual interrogative. The Phoenician case of Section 5.15.1 lost the word "where" entirely. It should also be gated on whether the step resolves an entity that was already unambiguous, and on whether the sub-objectives reference each other at all, since ones that do not are descriptors of a single entity rather than a chain. The second is *a set-intersection operator*. The category composite_claim is 47 cases and CWQ's largest category at 46 of them. Its no_set_intersection subtype names the gap precisely. The architecture has no AND-join primitive. So a "find the overlap" sub-objective degrades into another relation-scoring pass over an incomplete first-hop set. The third is *ordinal and literal-value support*. The category kg_gap is 43 cases and CWQ's largest hedge category at 24. Two of its subtypes need a minimum-or-maximum-by-value operation and a literal-extraction path rather than a bigger graph.

6.3.4 Semantic-Level Verification

All five wrongly-rejected cases share one mechanism. The structural check requires a single edge asserting the drafted claim. A transitively-true multi-hop fact does not present one. A check that accepts a bounded relation chain expressing the claim would recover those answers. The relation-path formulation of RoG [9] is the natural source for what such a chain may look like. Using it as an acceptance criterion rather than as a retrieval plan is the change proposed. The Tier-2 judge of Section 5.5.3 is a prototype of exactly this judgement. That resemblance suggests folding a bounded form of it into the loop. Its κ of 0.6995 is also a caution about how reliable such a check would be.

6.3.5 Measurements This Thesis Could Not Make

Path fidelity is the third evaluation criterion of the approved proposal, deferred for the reason given in Section 5.1.2. It needs the benchmarks' gold SPARQL relation chains. The distribution used here does not carry them. Reconstructing them would measure whether the system reaches the right answer *by the right path*. The groundedness metrics can only approximate that question, and it bears directly on the echo attractor.

Two baseline bounds are cheap to lift, since both are re-runs rather than redesigns. The first is to *equalise the candidate widths*. That means re-running the agentic baseline at AGR's 300 relations and 200 neighbours, leaving beam width, depth, budget, prompts, and scoring untouched. That decides whether the unclipped-subset comparison of Section 5.8 measured search strategy or candidate supply. Both outcomes are usable, though it will also raise the clip rate. Wider

candidate sets make each pruning call dearer. So the split must be reported rather than the aggregate. The second is to *widen the static retriever's radius, and order its fanout*. That means re-running it with neighbours expanded a second time, everything else frozen, and reporting the same strata, so that its collapse on deep CWQ questions can be attributed to the paradigm rather than to the particular radius chosen. The same pass should impose an ordering on the fanout cap, so that the 100 edges kept per topic entity are a stated selection rather than an arbitrary one (Section 5.4.2). Until then those two limits are confounded.

6.3.6 Beyond This Environment

Four directions extend the work rather than repairing it. *LLM-constructed knowledge graphs* as a controlled comparison would separate what the framework contributes from what a curated environment contributes, by holding the agent fixed while varying the substrate's provenance and noise profile.

Rollout-based value estimation would supply what Section 2.2.4 explicitly disclaims. The current scorer evaluates each candidate once and never revises it in light of what is found later. Genuine tree search over this same tool API would be a well-defined next system, not a rebranding of this one.

Temporal and multi-source environments address the failure family that recurs throughout Section 5.14, questions with a time qualifier the graph cannot express.

And *transfer to high-stakes domains* is where the auditability property matters more than the accuracy.

6.4 Closing Remark

The result this work would most want carried forward is not the accuracy margin. It is the shape of the gap that remains. A system that traverses a real graph and copies its answers out of real triples can be made to assert nothing ungrounded. That problem is solved, and the navigation paradigm solves it, not any checking layer built on top. What remains is an answer that is grounded, connected, and true, and is still not the answer to the question that was asked. The census in this thesis found that failure more often than any other mechanism. Fact verification against a knowledge graph turns out to be the easier half of the problem. *Relevance* verification is the half still open.

References

[1] L. Huang, W. Yu, W. Ma, W. Zhong, Z. Feng, H. Wang, Q. Chen, W. Peng, X. Feng, B. Qin, and T. Liu, "A survey on hallucination in large language models: Principles, taxonomy, challenges, and open questions," *ACM Transactions on Information Systems*, vol. 43, no. 2, pp. 1–55, 2025.

[2] P. Lewis, E. Perez, A. Piktus, F. Petroni, V. Karpukhin, N. Goyal, H. Küttler, M. Lewis, W.-t. Yih, T. Rocktäschel, S. Riedel, and D. Kiela, "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 9459–9474, 2020.

[3] Y. Li, Z. Li, P. Wang, J. Li, X. Sun, H. Cheng, and J. X. Yu, "A survey of graph meets large language model: Progress and future directions," *arXiv preprint arXiv:2311.12399*, 2023.

[4] D. Edge, H. Trinh, N. Cheng, J. Bradley, A. Chao, A. Mody, S. Truitt, D. Metropolitansky, R. O. Ness, and J. Larson, "From local to global: A graph RAG approach to query-focused summarization," *arXiv preprint arXiv:2404.16130*, 2024.

[5] B. J. Gutiérrez, Y. Shu, Y. Gu, M. Yasunaga, and Y. Su, "HippoRAG: Neurobiologically inspired long-term memory for large language models," in *Advances in Neural Information Processing Systems*, vol. 37, pp. 59532–59569, 2024.

[6] D. Li, Y. Niu, Y. Ai, X. Zou, B. Qi, and J. Liu, "T-GRAG: A dynamic GraphRAG framework for resolving temporal conflicts and redundancy in knowledge retrieval," in *Proceedings of the 33rd ACM International Conference on Multimedia*, pp. 11880–11889, 2025.

[7] J. Sun, C. Xu, L. Tang, S. Wang, C. Lin, Y. Gong, L. M. Ni, H.-Y. Shum, and J. Guo, "Think-on-Graph: Deep and responsible reasoning of large language model on knowledge graph," in *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2307.07697.

[8] L. Chen, P. Tong, Z. Jin, Y. Sun, J. Ye, and H. Xiong, "Plan-on-Graph: Self-correcting adaptive planning of large language model on knowledge graphs," in *Advances in Neural Information Processing Systems*, vol. 37, 2024. arXiv:2410.23875.

[9] L. Luo, Y.-F. Li, G. Haffari, and S. Pan, "Reasoning on graphs: Faithful and interpretable large language model reasoning," in *International Conference on Learning Representations (ICLR)*, 2024. arXiv:2310.01061.

[10] J. Jiang, K. Zhou, W. X. Zhao, Y. Song, C. Zhu, H. Zhu, and J.-R. Wen, "KG-Agent: An efficient autonomous agent framework for complex reasoning over knowledge graph," *arXiv preprint arXiv:2402.11163*, 2024.

[11] Y. Xu, S. He, J. Chen, Z. Wang, Y. Song, H. Tong, K. Liu, and J. Zhao, "Generate-on-Graph: Treat LLM as both agent and KG for incomplete knowledge graph question answering," in *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, 2024. arXiv:2404.14741.

[12] J. Jiang, K. Zhou, Z. Dong, K. Ye, W. X. Zhao, and J.-R. Wen, "StructGPT: A general framework for large language models to reason over structured data," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, 2023. arXiv:2305.09645.

[13] S. Pan, L. Luo, Y. Wang, C. Chen, J. Wang, and X. Wu, "Unifying large language models and knowledge graphs: A roadmap," *IEEE Transactions on Knowledge and Data Engineering*, vol. 36, no. 7, pp. 3580–3599, 2024.

[14] C. Ma, Y. Chen, T. Wu, A. Khan, and H. Wang, "Unifying large language models and knowledge graphs for question answering: Recent advances and opportunities," in *Proceedings of the 28th International Conference on Extending Database Technology (EDBT)*, pp. 1174–1177, 2025.

[15] S. Dhuliawala, M. Komeili, J. Xu, R. Raileanu, X. Li, A. Celikyilmaz, and J. Weston, "Chain-of-verification reduces hallucination in large language models," in *Findings of the Association for Computational Linguistics: ACL 2024*, pp. 3563–3578, 2024.

[16] Y. Du, S. Li, A. Torralba, J. B. Tenenbaum, and I. Mordatch, "Improving factuality and reasoning in language models through multiagent debate," in *Proceedings of the 41st International Conference on Machine Learning*, 2024.

[17] Y. Hu, W. Xuan, Q. Zhou, Z. Li, Y. Li, J. Hu, and F. Fang, "A self-correcting agentic graph RAG for clinical decision support in hepatology," *Frontiers in Medicine*, vol. 12, p. 1716327, 2025.

[18] K. Bollacker, C. Evans, P. Paritosh, T. Sturge, and J. Taylor, "Freebase: A collaboratively created graph database for structuring human knowledge," in *Proceedings of the 2008 ACM SIGMOD International Conference on Management of Data*, pp. 1247–1250, 2008.

[19] W.-t. Yih, M. Richardson, C. Meek, M.-W. Chang, and J. Suh, "The value of semantic parse labeling for knowledge base question answering," in *Proceedings of the 54th Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pp. 201–206, 2016.

[20] A. Talmor and J. Berant, "The web as a knowledge-base for answering complex questions," in *Proceedings of the 2018 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*, pp. 641–651, 2018.

[21] I. Robinson, J. Webber, and E. Eifrem, *Graph Databases: New Opportunities for Connected Data*. O'Reilly Media, 2nd ed., 2015.

[22] N. Francis, A. Green, P. Guagliardo, L. Libkin, T. Lindaaker, V. Marsault, S. Plantikow, M. Rydberg, P. Selmer, and A. Taylor, "Cypher: An evolving query language for property graphs," in *Proceedings of the 2018 International Conference on Management of Data*, pp. 1433–1445, 2018.

[23] J. Berant, A. Chou, R. Frostig, and P. Liang, "Semantic parsing on Freebase from question-answer pairs," in *Proceedings of the 2013 Conference on Empirical Methods in Natural Language Processing*, pp. 1533–1544, 2013.

[24] T. B. Brown, B. Mann, N. Ryder, M. Subbiah, J. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, *et al.*, "Language models are few-shot learners," in *Advances in Neural Information Processing Systems*, vol. 33, pp. 1877–1901, 2020.

[25] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Advances in Neural Information Processing Systems*, vol. 35, pp. 24824–24837, 2022.

[26] T. Schick, J. Dwivedi-Yu, R. Dessì, R. Raileanu, M. Lomeli, E. Hambro, L. Zettlemoyer, N. Cancedda, and T. Scialom, "Toolformer: Language models can teach themselves to use tools," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[27] S. Yao, J. Zhao, D. Yu, N. Du, I. Shafran, K. Narasimhan, and Y. Cao, "ReAct: Synergizing reasoning and acting in language models," in *International Conference on Learning Representations (ICLR)*, 2023.

[28] V. Karpukhin, B. Oğuz, S. Min, P. Lewis, L. Wu, S. Edunov, D. Chen, and W.-t. Yih, "Dense passage retrieval for open-domain question answering," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing*, pp. 6769–6781, 2020.

[29] N. Reimers and I. Gurevych, "Sentence-BERT: Sentence embeddings using Siamese BERT-networks," in *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*, pp. 3982–3992, 2019.

[30] P. E. Hart, N. J. Nilsson, and B. Raphael, "A formal basis for the heuristic determination of minimum cost paths," *IEEE Transactions on Systems Science and Cybernetics*, vol. 4, no. 2, pp. 100–107, 1968.

[31] J. Pearl, *Heuristics: Intelligent Search Strategies for Computer Problem Solving*. Addison-Wesley, 1984.

[32] L. Kocsis and C. Szepesvári, "Bandit based Monte-Carlo planning," in *Proceedings of the 17th European Conference on Machine Learning*, pp. 282–293, 2006.

[33] C. B. Browne, E. Powley, D. Whitehouse, S. M. Lucas, P. I. Cowling, P. Rohlfshagen, S. Tavener, D. Perez, S. Samothrakis, and S. Colton, "A survey of Monte Carlo tree search methods," *IEEE Transactions on Computational Intelligence and AI in Games*, vol. 4, no. 1, pp. 1–43, 2012.

[34] H. Luo, H. E, Y. Guo, Q. Lin, X. Wu, X. Mu, W. Liu, M. Song, Y. Zhu, and L. A. Tuan, "KBQA-o1: Agentic knowledge base question answering with Monte Carlo tree search," in *Proceedings of the 42nd International Conference on Machine Learning (ICML)*, 2025. arXiv:2501.18922.

[35] T. Shen, J. Wang, X. Zhang, and E. Cambria, "Reasoning with trees: Faithful question answering over knowledge graph," in *Proceedings of the 31st International Conference on Computational Linguistics*, pp. 3138–3157, 2025.

[36] B. Efron, "Bootstrap methods: Another look at the jackknife," *The Annals of Statistics*, vol. 7, no. 1, pp. 1–26, 1979.

[37] Q. McNemar, "Note on the sampling error of the difference between correlated proportions or percentages," *Psychometrika*, vol. 12, no. 2, pp. 153–157, 1947.

[38] J. Cohen, "A coefficient of agreement for nominal scales," *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.

[39] J. R. Landis and G. G. Koch, "The measurement of observer agreement for categorical data," *Biometrics*, vol. 33, no. 1, pp. 159–174, 1977.

[40] X. Tan, X. Wang, Q. Liu, X. Xu, X. Yuan, and W. Zhang, "Paths-over-Graph: Knowledge graph empowered large language model reasoning," *arXiv preprint arXiv:2410.14211*, 2024.

[41] J. Huang, X. Chen, S. Mishra, H. S. Zheng, A. W. Yu, X. Song, and D. Zhou, "Large language models cannot self-correct reasoning yet," in *International Conference on Learning Representations (ICLR)*, 2024.

[42] N. Shinn, F. Cassano, A. Gopinath, K. Narasimhan, and S. Yao, "Reflexion: Language agents with verbal reinforcement learning," in *Advances in Neural Information Processing Systems*, vol. 36, 2023.

[43] S. Min, K. Krishna, X. Lyu, M. Lewis, W.-t. Yih, P. W. Koh, M. Iyyer, L. Zettlemoyer, and H. Hajishirzi, "FActScore: Fine-grained atomic evaluation of factual precision in long form text generation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 12076–12100, 2023.

[44] F. F. Bayat, L. Zhang, S. Munir, and L. Wang, "FactBench: A dynamic benchmark for in-the-wild language model factuality evaluation," in *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 33090–33110, 2025.

[45] Y. Gu, S. Kase, M. Vanni, B. Sadler, P. Liang, X. Yan, and Y. Su, "Beyond I.I.D.: Three levels of generalization for question answering on knowledge bases," in *Proceedings of the Web Conference 2021*, pp. 3477–3488, 2021.

Index

answerer, 40, 50
assertion precision, 71

backtracking, 12, 39, 83
benchmark defect, 6, 94
best-first search, 12
bootstrap, 13
budget, 40, 66, 79, 80

claim extraction, 17, 44
Cohen's kappa, 13, 66, 79
ComplexWebQuestions, 23, 56, 71
controlled-agent pattern, 30

dataset, 22, 56

echo attractor, 6, 93
evaluator, 38, 90
explorer, 36

F1, 10, 64
failure census, 85
Freebase, 9, 22
frontier, 36

gold noise, 58, 94
GraphRAG, 3, 13, 61
groundedness, 38, 65, 78

hallucination, 1, 11, 65, 78
hedge, 64, 87, 88
Hits@1, 10, 64, 70
hop stratum, 57, 76
hybrid scoring function, 37, 83

in-context learning, 11

KGQA, 10, 14
knowledge graph, 3, 8, 23

McNemar, 13
mediator node, 9, 24, 33
mediator traversal, 24, 33
MID, 9, 22
multi-hop question, 2, 10

planner, 35, 82, 89
precision, 64, 71
property graph, 9

ReAct, 11
repair objective, 49
retrieval-augmented generation, 11
retry route, 49, 50
RoG, 74

schema, 9
state machine, 31

Think-on-Graph, 15, 62, 72
traversed adjacency, 45, 47
triple, 8

verification layer, 43, 44, 83, 102
verify_connection, 32, 47

WebQSP, 23, 56, 70

Generated using Postgraduate Thesis LaTeX Template, Version 1.03. Department of Computer Science and Engineering, Bangladesh University of Engineering and Technology, Dhaka, Bangladesh.

This thesis was generated on Thursday 22nd October, 2026 at 3:02am.